

FPGA/IC 秋招笔试/面试题总结

原创

Crazy_M

2021-08-25 10:39:13

3407

★ 收藏 163

版权

分类专栏：

FPGA

FPGA/IC秋招笔试/面试

文章标签：

FPGA

IC



FPGA 同时被 2 个专栏收录 ▾

22 订阅

25 篇文章

持续更新，请根据目录浏览，觉得有用三连支持一下也是一种鼓励~。
未经允许，请勿转载，仅供学习，请勿用于商业用途。

下一章

FPGA/IC 秋招笔试/面试题总结（续）

文章目录

一、FPGA内部资源

1. 什么是FPGA

2.FPGA内部资源

二、同步时钟、同步/异步电路

1.同步时钟

2.同步/异步电路

三、同步复位/异步复位

1.同步复位

2. 异步复位

3. 异步复位同步释放

四、同步FIFO/异步FIFO

1.同步FIFO

2.异步FIFO

五、FIFO最小深度计算

1.FIFO最小深度

2.示例分析

六、逻辑电平

1.TTL电平标准

2.LVTTL电平标准

3.CMOS电平标准

4.LVCOMS电平标准

5.RS232

6.RS485

7.ECL、PECL、LVPECL

8.LVDS

9.CML

10.GTL

11.HSTL

12.SSTL

七、建立时间与保持时间

1.建立时间与保持时间概述及模型

2.寄存器之间数据传输时建立时间与保持时间

3.建立时间余量与保持时间余量计算

4. FPGA允许的最大时钟频率或最小时钟周期

5.计算FPGA时钟所允许的最大频率或最小时钟周期

6.违反建立和保持时间会发生什么？

八、recovery time和removal time

九、亚稳态

十、竞争与冒险

- 1.竞争与冒险产生原因
- 2.判断电路是否存在竞争-冒险现象
3. 消除竞争与冒险

十一、奇数分频

- 1.奇数分频介绍
- 2.通过时钟 相或 实现奇数分频
- 3.通过时钟 相与 实现奇数分频
- 4.通过时钟 相异或 实现奇数分频

十二、时钟抖动 (Clock Jitter) 和时钟偏斜 (Clock Skew)

- 1.时钟抖动 (Clock Jitter)
- 2.时钟偏斜 (Clock Skew)
- 3.时钟抖动和时钟偏斜的区别

十三、Verilog的结构化、数据流、行为级描述方式

- 1.结构化描述方式及RTL视图
- 2.数据流描述方式及RTL视图
- 3.行为级描述方式及RTL视图
- 4.常用门级元件

十四、函数 (function) 和任务 (task)

- 1.函数 function
- 2.任务 task

十五、波特率

十六、锁存器/触发器

十七、组合逻辑/时序逻辑

十八、阻塞/非阻塞赋值

十九、跨时钟域处理

二十、位宽计算（N bit数和M bit数相加、相乘后需要多少bit？）

二十一、IIC

1. IIC接口介绍
2. IIC总线特征
3. IIC总线协议详解
4. 起始条件
5. 从设备地址+R/W位
6. ACK应答
7. 数据传输
8. 停止条件

二十二、SPI

二十三、AXI

二十四、RAM/SRAM/DRAM/SDRAM/DDR

二十五、ROM/PROM/EPROM/E2PROM/FLASH

二十六、硬核/软核/固核

1. 硬核（Hard IP Core）
2. 软核（Soft IP Core）
3. 固核（Firm IP Core）

二十七、浮点数与定点数

1. 32位浮点数
2. 定点数
3. 浮点数转换为定点数
3. 定点数转换为浮点数

一、FPGA内部资源

1. 什么是FPGA

FPGA是一种可以重构电路的芯片，是一种硬件可重构的体系结构。Field Programmable Gate Array，中文名是 **现场可编程门阵列**。通过编程，用户可以随时改变它的应用场景，它可以模拟CPU、GPU等硬件的各种并行运算。通过与目标硬件的高速接口互联，FPGA可以完成目标硬件运行效率比较低的部分，从而在系统层面实现加速。

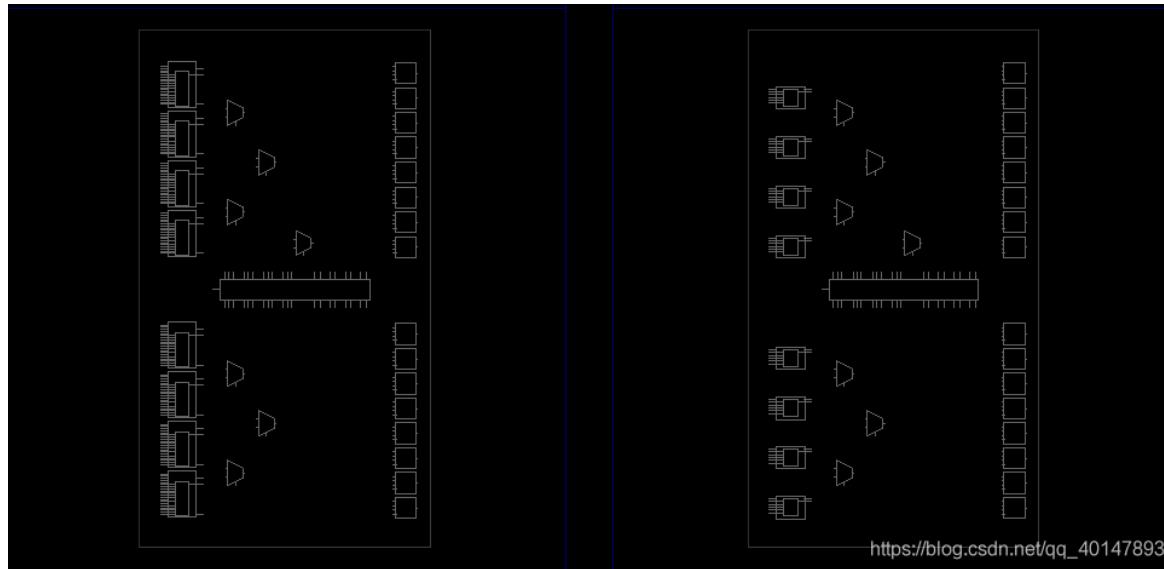
2.FPGA内部资源

目前主流的FPGA都采用了SRAM工艺的查找表（LUT）结构，LUT本质上就是个RAM。FPGA内部组成主要有：**可编程输入/输出块（IOB）**、**可配置逻辑块（CLB）**、**嵌入式块RAM（BRAM）**、丰富的**布线资源**、**底层内嵌功能资源**、内嵌专用**硬核资源**等。

1、可编程输入/输出块（IOB）：为了便于管理和适应多种电气标准，FPGA的IOB被划分为若干个组（bank），每个bank的接口标准由其接口电压VCCO决定，一个bank只能有一种VCCO，但是不同的bank的VCCO可以不同。只有相同标准的端口才能连接在一起，VCCO电压相同是接口标准的基本条件。

2、可配置逻辑块（CLB）：由查找表和可编程寄存器组成，查找表（LUT）完成纯组合逻辑，内部寄存器可配置成触发器或者锁存器。在Xilinx公司的FPGA器件中，CLB由多个（一般为4个或2个）相同的Slice和附加逻辑构成。每个CLB模块不仅可以用于实现组合逻辑、时序逻辑，还可以配置为分布式RAM和分布式ROM。

Slice：Slice是Xilinx公司定义的基本逻辑单位，，一个Slice由两个4输入的函数、进位逻辑、算术逻辑、存储逻辑和函数复用器组成。算术逻辑包括一个异或门（XORG）和一个专用与门（MULTAND），一个异或门可以使一个Slice实现 2bit全加操作，专用与门用于提高乘法器的效率；进位逻辑由专用进位信号和函数复用器（MUXC）组成，用于实现快速的算术加减法操作；4输入函数发生器用于实现4输入LUT、分布式RAM或16比特移位寄存器（Virtex-5系列芯片的Slice中的两个输入函数为6输入，可以实现6输入LUT或 64比特移位寄存器）（根据不同型号去看）；进位逻辑包括两条快速进位链，用于提高CLB模块的处理速度。



上图中左边为SliceM，右边为 SliceL。

SliceM 相比 SliceL 多出的功能在于可以配置成 Distribute RAM（分布式RAM）。

SliceM 相比 SliceL 多了做存储器和移位的功能。

SliceM 相比 SliceL 中含有能够把LUT资源重新整合为RAM和ROM的逻辑。

LUT：目前FPGA中多使用4输入的LUT，所以每一个LUT可以看成是一个有 4位地址线的16x1的RAM。当用户通过原理图或HDL语言描述了一个逻辑电路以后，FPGA开发软件会自动计算逻辑电路的所有可能的结果，并把结果事先写入RAM，这样，每输入一个信号进行逻辑运算就等于输入一个地址进行查表，找出地址对应的内容，然后输出即可。

分布式RAM（DRAM）：查找表存储器

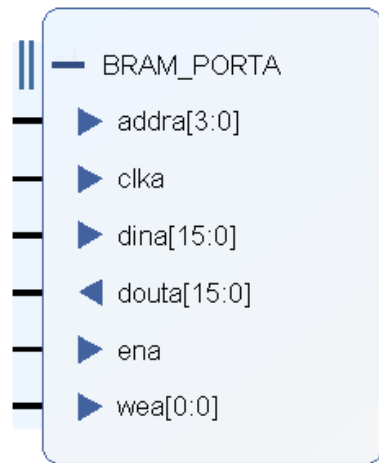
只有成为SLICEM的逻辑块里的查找表（LUT）才可以用做分布式RAM。

利用查找表为电路实现存储器，既可以实现芯片内部存储，又能提高资源利用率。**分布式RAM的特点是可以实现BRAM不能实现的异步访问。**不过使用分布式RAM实现大规模的存储器会占用大量的LUT，可用来实现逻辑的查找表就会减少。因此建议仅在需要小规模存储器时，使用这种分布式RAM。

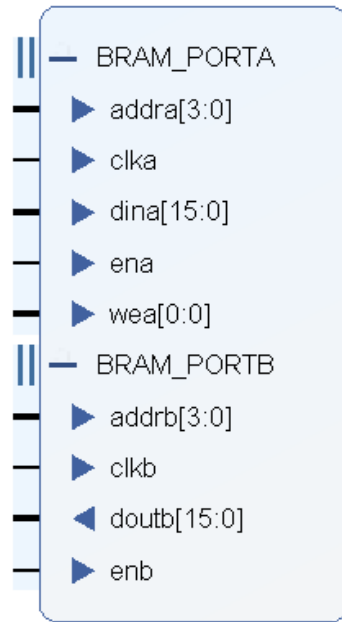
3、嵌入式块RAM：可配置成单端口RAM、双端口RAM、内容地址存储器（CAM）以及FIFO的常用的存储结构。RAM资源使用详解

Block RAM都有两套访问存储器所需的地址总线、数据总线及控制信号灯信号，因此其既可以作为单端口存储器，也可以作为双端口存储器。需要注意的是访问BRAM需要和时钟同步，异步访问不支持的。

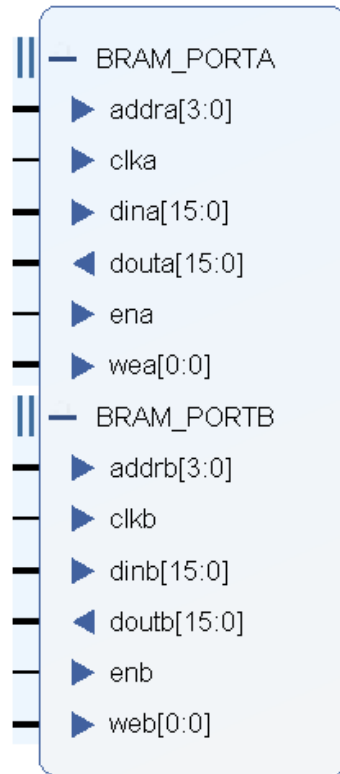
单口RAM (Single Port RAM)：只有一组数据线和地址线，因此读写不能同时进行。



伪双口RAM (Simple Dual Port RAM)：一个端口只读，另一个端口只写。



双口RAM (True Dual Port RAM)：两个端口都可以读写。

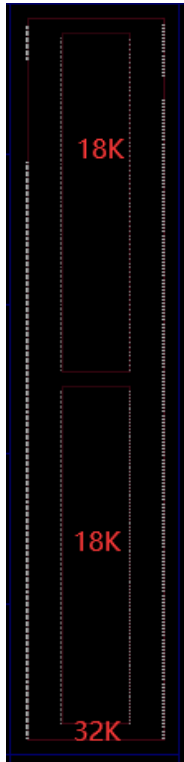


RAM构成:

比如在7系列的Xilinx器件中，每个36Kb的RAM由两个18Kb的RAM组成，每个BRAM有36Kbit的容量，既可以作为一个36Kbit的存储器使用，也可以拆分为两个独立的18Kbit存储器使用。反过来相邻两个BRAM可以结合起来实现72Kbit存储器，而且不消耗额外的逻辑资源。

The 7 series FPGAs block RAM features include:

- Per block memory storage capability where each block RAM can store up to 36 Kbits of data.
- Support of two independent 18Kb blocks or a single 36Kb block RAM.



RAM资源使用：

在器件选择的时候我们可以看到使用器件的资源详细信息，比如XCKU040这款拥有600个RAM块。

Part	I/O Pin C...	Available I...	LUT Elem...	FlipFlo...	Block RAMs	Ultra R...	DS...	Gb Transcei...
xcku040-ffva1156-3-e	1156	520	242400	484800	600	0	19...	20
xcku040-ffva1156-2-e	1156	520	242400	484800	600	0	19...	20
xcku040-ffva1156-2-i	1156	520	242400	484800	600	0	19...	20
xcku040-ffva1156-1-c	1156	520	242400	484800	600	0	19...	20
xcku040-ffva1156-1-i	1156	520	242400	484800	600	0	19...	20
xcku040-ffva1156-1...	1156	520	242400	484800	600	0	19...	20

总容量：600×36Kb=21600Kb=21.1Mb。

我们**例化一个RAM**，看看资源占用情况：

数据深度：20480

宽度：18

占用BRAM内存： $20480 \times 18 = 20 \times 18 \text{ Kbit} = 10 \times 36 \text{ Kbit}$

通过以上的分析正好为10个36Kbit，应该占用10个36Kb的BRAM即10个BRAM。

Memory Size

Write Width	18	Range: 1 to 4608 (bits)
Read Width	18	
Write Depth	20480	Range: 2 to 1048576
Read Depth	20480	

Tcl ConsoleMessagesLogReportsDesign RunsUtilizationTimingDRCPowerMethodologyI/O Ports

Hierarchy

Summary

Slice Logic

Slice LUTs (1%)

LUT as Memory (<1%)

LUT as Shift Register

utilization_1

Hierarchy

Name	Slice LUTs (222600)	Block RAM Tile (715)	Bonded IOB (300)	BUFGCTRL (32)	MMCME2_ADV (6)
top	1430	12	1	3	1
blkram_0 (blk_mem_gen_0)	44	10	0	0	0
clk_0 (clk_wiz_0)	0	0	0	2	1
dbg_hub (dbg_hub)	475	0	0	1	0
ila (ila_0)	857	2	0	0	0

调用ILA也是会消耗BRAM的资源

ila IP配置如下

版权：Crazyy_M

General Options

Probe_Ports(0..5)

Monitor Type

☒ Native

☐ AXI

Number of Probes

6

[1...1024]

Sample Data Depth

1024

☒ Same Number of Comparators for All Probe Ports

Number of Comparators

1

☐ Trigger Out Port

☐ Trigger In Port

Input Pipe Stages

0

General Options		Probe_Ports(0..5)			
Probe Port	Probe Width [1..4096]		Number of Comparators		Probe
PROBE0	1		1		DATA
PROBE1	1		1		DATA
PROBE2	1		1		DATA
PROBE3	16		1		DATA
PROBE4	18		1		DATA
PROBE5	18		1		DATA

总55

ila数据总宽度：55

ila数据深度：1024

$55 \times 1024 = 56320 = 55\text{Kbits}$

需要BRAM 至少 $55/36 = 1.52 \rightarrow 2$ 个(往上取整)。

Name	Slice LUTs (222600)	Block RAM Tile (715)	Bonded IOB (300)	BUFCTRL (32)	MMCM
top	1430	12	1	3	1
> blkram_0 (blk_mem_gen_0)	44	10	0	0	0
> clk_0 (clk_wiz_0)	0	0	0	2	1
> dbg_hub (dbg_hub)	475	0	0	1	0
> ila (ila_0)	857	2	0	0	0

4、丰富的布线资源：布线资源连通FPGA内部的所有单元。而连线的长度和工艺决定信号在连线上的驱动能力和传输速度。主要分为四类：全局布线资源、长线资源、短线资源、分布式布线资源。FPGA中的布局布线资源主要包括三部分：CB、SB和行列连线。布线资源的目的是为了能够让位于不同位置的逻辑资源块、时钟处理单元、BRAM、DSP和接口模块等资源能够相互通信，从而协调合作，完成所需功能。FPGA中的布线资源，就好比绘制PCB板时的连线资源一样，虽然器件A和器件B的位置和连接关系没有任何改变，但很可能因为周边电路的布局、布线的一些变化，使得前后两次A、B之间的连线形态发生很大的变化。因此，与采用固定长度的金属线将所有宏单元连接在一起的CPLD不同，FPGA中任意两点之间的线延迟是无法预测的，必须等到布局、布线完成之后才能确定。**这也是为什么在FPGA的开发中，时序约束和时序分析这两样工作是必不可少的。**

5、底层内嵌的功能单元：主要包括DLL、PLL、DSP、CPU等，现在越来越丰富的内嵌功能单元，使得FPGA称为了系统级的设计工具，使其具备了软硬件联合设计的能力。

PLL：

PLL的英文全称是Phase Locked Loop，即锁相环，是一种反馈控制电路。

锁相环作为一种反馈控制电路，其特点是**利用外部输入的参考信号控制环路内部震荡信号的频率和相位**。因为锁相环可以实现输出信号频率对输入信号频率的自动跟踪，所以锁相环通常用于闭环跟踪电路。锁相环在工作的过程中，当输出信号的频率与输入信号的频率相等时，输出电压与输入电压保持固定的相位差值，即输出电压与输入电压的相位被锁住，这就是锁相环名称的由来。

6、内嵌专用的硬核资源：内嵌的专用硬核是相对底层软核而言的，指FPGA处理能力强大的硬核，等效于ASIC电路。主要有乘法器、高速收发器、PCIe、以太网控制器等。

二、同步时钟、同步/异步电路

1.同步时钟

当两个时钟的相位关系是固定关系，则可以称这个时钟为同步时钟（synchronous clock），经过一个PLL产生相位不同，但是相位固定的两个时钟，他们依旧是同步时钟。而如果是两个晶振产生的时钟，因为两个晶振在上电时相位差是随机的，而且不同晶振时钟漂移抖动也不一样，所以相位是不固定的。当无法判断两个时钟间的相位时，则可以称这两个时钟为异步时钟(asynchronous clocks)。

2.同步/异步电路

异步电路：

- a) 电路核心逻辑是用组合电路实现;
- b) 异步时序电路的最大缺点是容易产生毛刺;
- c) 不利于器件移植;
- d) 不利于静态时序分析(STA)、验证设计时序性能。

同步时序电路：

- a) 电路核心逻辑是用各种触发器实现;
- b) 电路主要信号、输出信号等都是在某个时钟沿驱动触发器产生的;
- c) 同步时序电路可以很好的避免毛刺;
- d) 利于器件移植;
- e) 利于静态时序分析(STA)、验证设计时序性能。

三、同步复位/异步复位

1.同步复位

同步复位只有在时钟沿到来时复位信号才起作用，则复位信号持续的时间应该超过一个时钟周期才能保持系统复位。

同步复位代码：

```

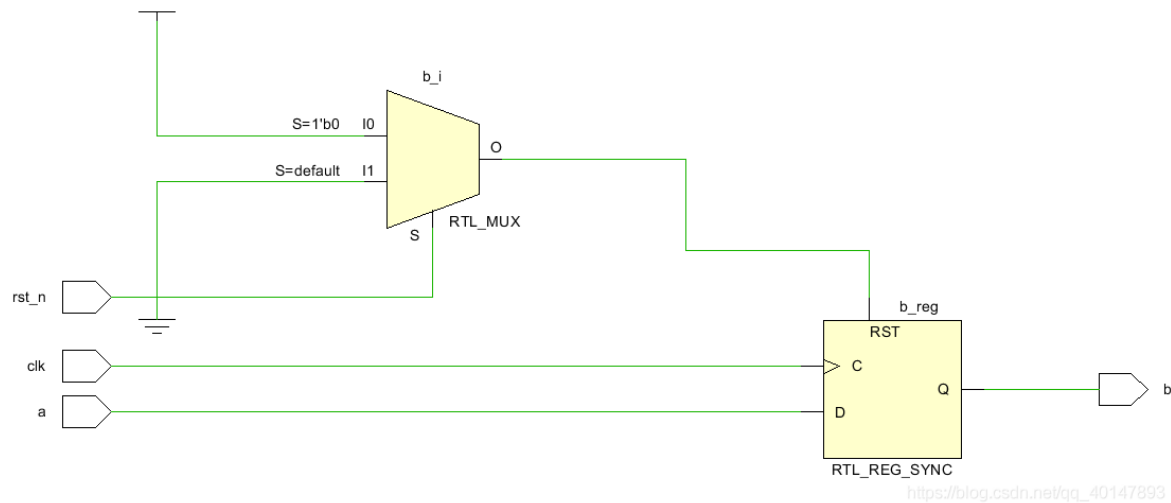
1  //同步复位
2  module top
3  (
4      input    clk,
5      input    rst_n,
```

```

6      input  a,
7
8      output reg b
9
10 );
11
12
13 always@(posedge clk)
14 begin
15     if(!rst_n)
16         b <= 1'b0;
17     else
18         b <= a;
19 end
20
21 endmodule

```

同步复位RTL视图：



同步复位优点： 一般能够保证电路百分百同步。确保复位只发生在有效沿，可以作为过滤掉毛刺的手段

同步复位的缺点： 复位信号的有效时长必须大于时钟周期，才能真正被系统识别并完成复位。同时还

要考虑如：时钟偏斜、组合逻辑路径延时、复位延时等因素。由于大多数的厂商目标库内的触发器都只有异步复位端口，采用同步复位的话，就会耗费较多的逻辑资源。

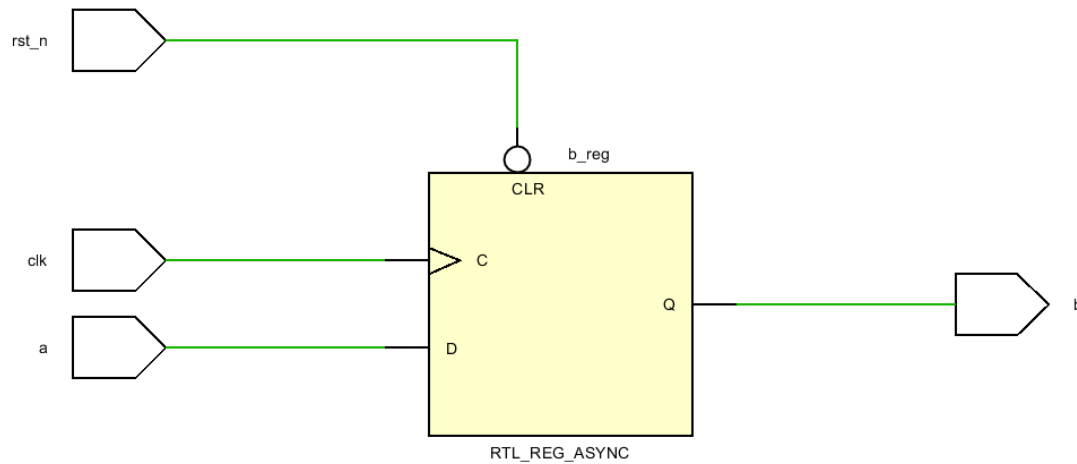
2. 异步复位

只要有复位信号系统马上复位，因此异步复位抗干扰能力差，有些噪声也能使系统复位，因此有时候闲的不够稳定，要想设计一个好的复位最好使用异步复位同步释放。

异步复位代码：

```
1 //异步复位
2 module top
3 (
4     input    clk,
5     input    rst_n,
6     input    a,
7
8     output   reg b
9
10 );
11
12
13 always@(posedge clk or negedge rst_n)
14 begin
15     if(!rst_n)
16         b <= 1'b0;
17     else
18         b <= a;
19 end
20
21 endmodule
22
```

异步复位RTL视图：


https://blog.csdn.net/qq_40147893

异步复位优点：异步复位信号识别方便，而且可以很方便的使用全局复位。由于大多数的厂商目标库内的触发器都有异步复位端口（CLR）（在这个端口一般接低电平有效的复位信号rst_n，即使设计中是高电平复位，实际综合后也会把异步复位信号反向后接到CLR端）可以节约逻辑资源。

异步复位缺点：复位信号容易受到毛刺的影响。复位结束时刻恰在亚稳态窗口内时，无法决定现在的复位状态是1还是0，会导致亚稳态。

通过一下实例分析异步复位存在的隐患：

```

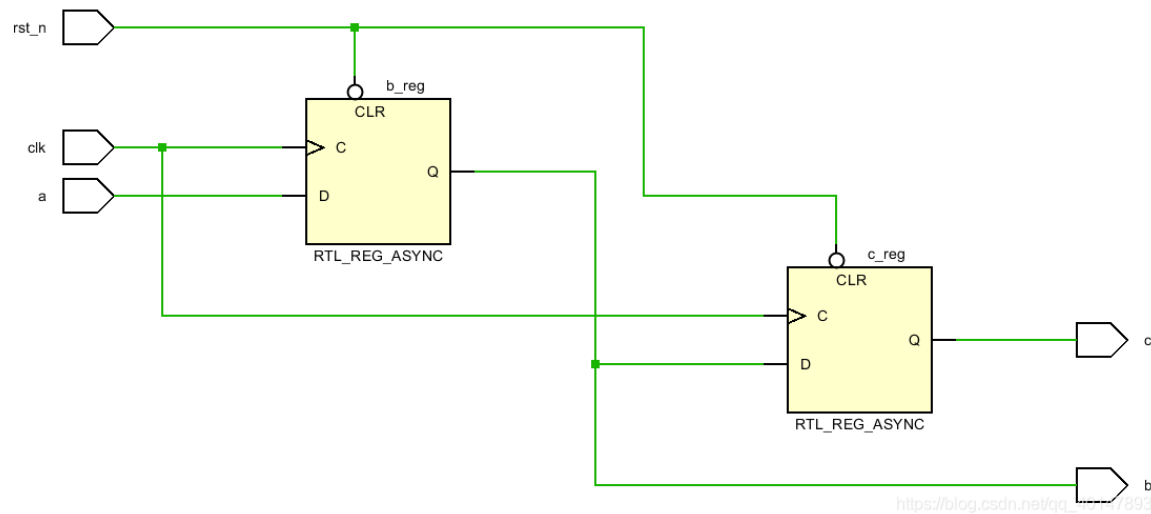
1 //异步复位存在的隐患实例
2 module top
3 (
4     input  clk,
5     input  rst_n,
6     input  a,
7
8     output reg b,
9     output reg c
10
11 );
12

```

```

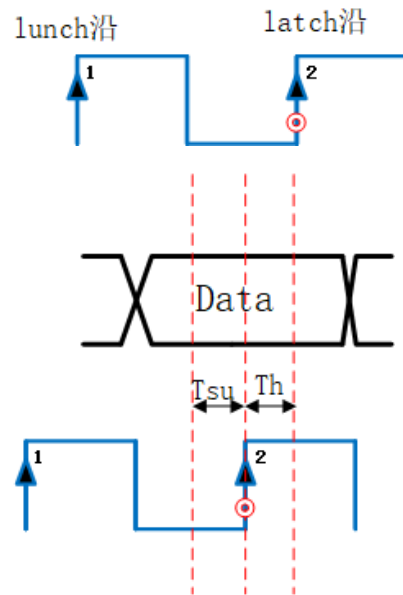
13
14 always@(posedge clk or negedge rst_n)
15 begin
16     if(!rst_n)
17         b <= 1'b0;
18     else
19         b <= a;
20 end
21 always@(posedge clk or negedge rst_n)
22 begin
23     if(!rst_n)
24         c <= 1'b0;
25     else
26         c <= b;
27 end
28
29 endmodule

```



正常情况下，在clk的上升沿将c的值更新为b，b的值更新为a。一旦进入复位，b，c都清零，但是并不能确定复位信号rst_n会在什么时候结束。

涉及到 建立时间和保持时间 参考博客：[建立时间和保持时间](#)



如果结束于b_reg和c_reg的{latch edge-setup, latch edge+hold time}时间之外，那么一切都会正常。如果出现在之内，复位信号的撤销（由低变高）出现在clk锁存数据的建立时间或者保持时间之内，此时clk检测到rst_n的状态就会是一个亚稳态（不确定是0还是1），就会导致输出数据的错误。

也有可能一个reg处于复位，另一个reg跳出了复位，均会影响系统的正常工作，如果更大的项目隐患就更大了。

3. 异步复位同步释放

使用异步复位同步释放就可以消除上述缺点。所谓异步复位，同步释放就是在复位信号到来的时候不受时钟信号的同步，而是在复位信号释放的时候受到时钟信号的同步，为了消除亚稳态的产生，利用两个同一时钟沿触发的层叠寄存器，将异步信号同步化。

```

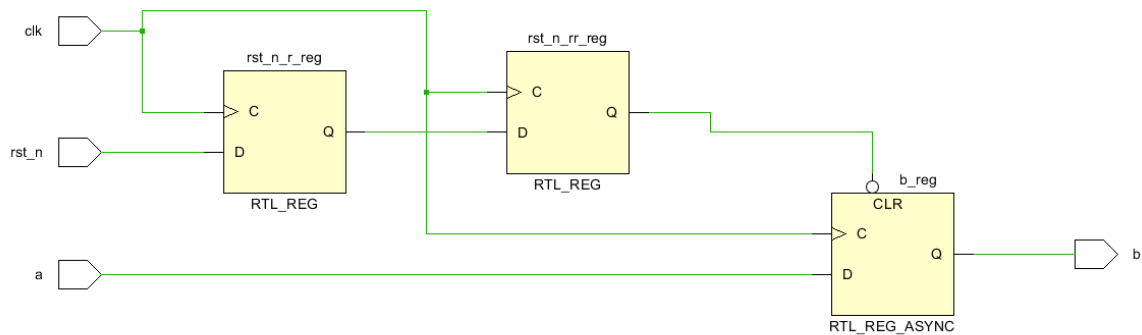
1 //异步复位、同步释放
2 module top
3 (
4     input    clk,
5     input    rst_n,
6     input    a,
7
8     output   reg b

```

```

9
10 );
11 reg rst_n_r;
12 reg rst_n_rr;
13
14 always@(posedge clk)
15 begin
16     {rst_n_rr,rst_n_r} <= {rst_n_r,rst_n};
17 end
18
19 always@(posedge clk or negedge rst_n_rr)
20 begin
21     if(!rst_n_rr)
22         b <= 1'b0;
23     else
24         b <= a;
25 end
26
27 endmodule

```



https://blog.csdn.net/qq_40147893

四、同步FIFO/异步FIFO

FIFO本质上还是RAM，是一种先进先出的数据缓存器(先存入的数据先取出)。它与普通存储器的区别：没有外部读写地址线，只能顺序写入数据，顺序的读出数据，其数据地址由**内部读写指针自动加1**，不像其他存储器可以由地址线决定读取或写入某个指定的地址，异步FIFO读写时钟不同，读写是相互独立的。

一般用途：

- (1)跨时钟域多bit传输：读写可以由不同的时钟控制，使用异步FIFO可以在两个不同时钟系统之间快速方便的传输数据。
- (2)数据匹配：对于不同宽度的数据接口可以使用FIFO，比如写入数据宽度为8bit，读取数据宽度为16bit，通过FIFO数据缓存器就可以达到数据匹配。

1.同步FIFO

同步FIFO的写时钟和读时钟为同一个时钟，FIFO内部所有逻辑都是同步逻辑，常常用于交互数据缓冲。

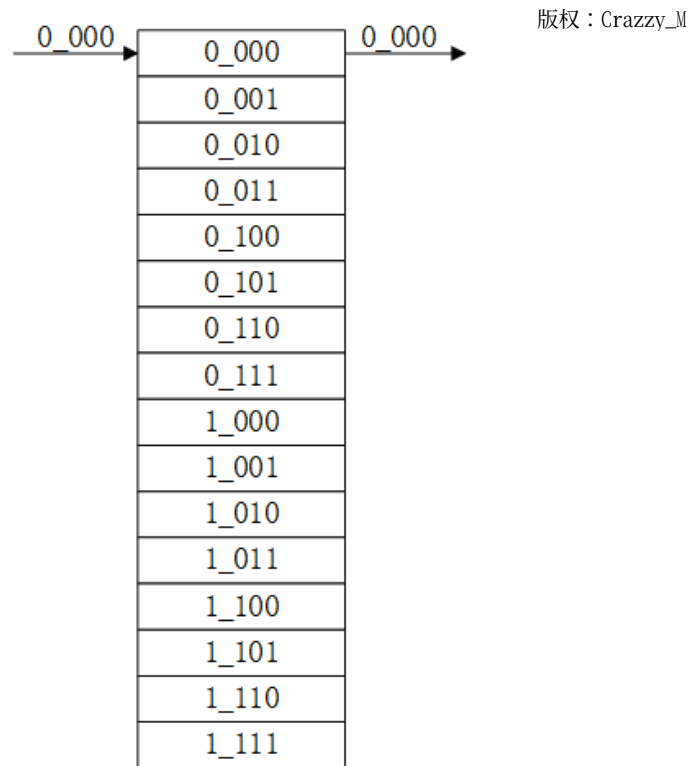
FIFO空、满信号的检测：

在设计FIFO时最重要的就是空满信号的检测，FIFO的主要是通过空(empty)信号、满信号(full)来控制数据的读写的，如果FIFO为空还读取数据势必会出现数据的错误，FIFO已满再写数据就会导致FIFO溢出同样会导致数据出错。那FIFO的空、满信号是怎么产生的呢。首先我们要明白读写FIFO的时候我们不用操作读写地址，但是我们在**写入数据或者读出数据时FIFO内部会通过写入或读出数据的操作在内部进行地址的自动增加**。当FIFO满或者空时我们通过内部读写地址的特征就会判断出FIFO的状态。

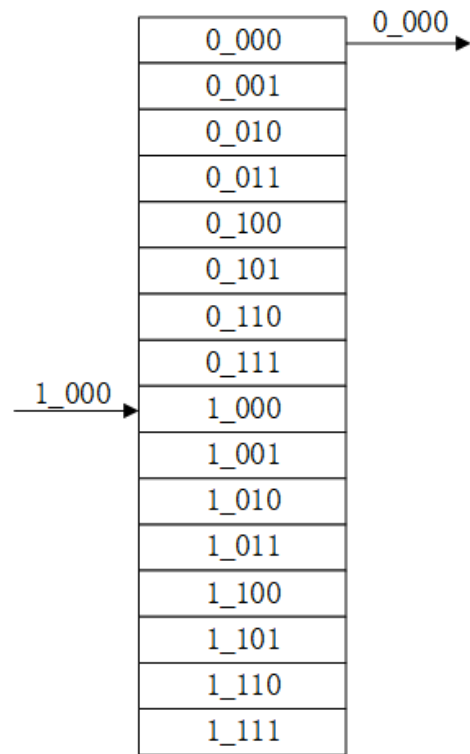
例如我们设计深度为8的FIFO即最大可以存放8个数据，地址所需位数为3，在设计FIFO时我们将地址扩展为4位**根据我们的扩展位可以判断FIFO的空满状态**。

8深度的FIFO经过以下4个操作：（下面方框代表地址）

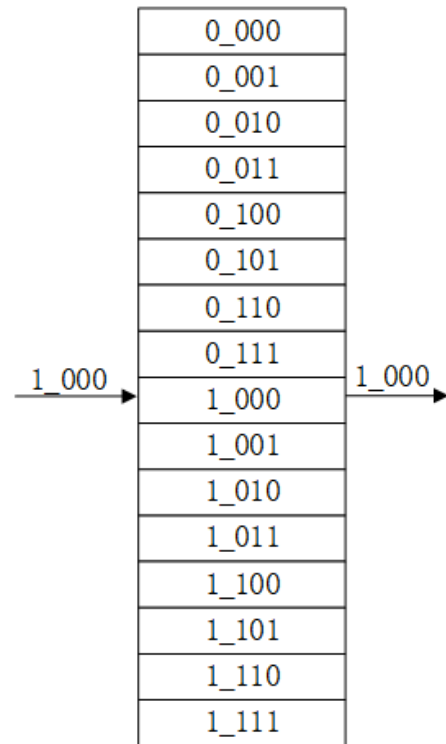
- 1、当FIFO为空时，读指针为0_000B，写指针为0_000B，此时FIFO里面没有数据，FIFO的状态为**空**。



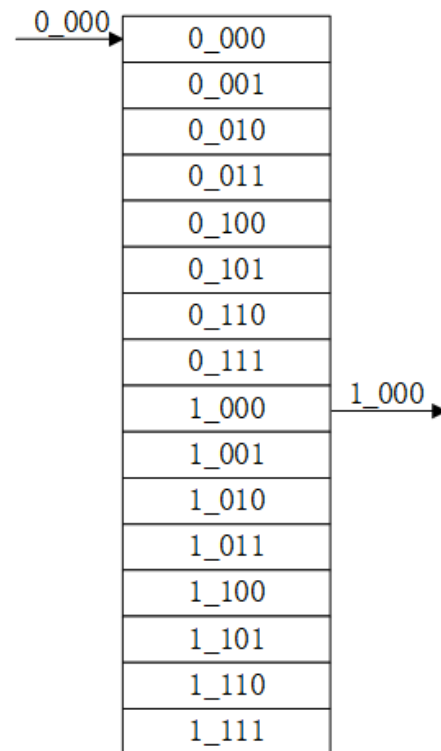
2、当FIFO写入8个数据时，写指针指向1_000B，读指针还为0_000B，此时FIFO里有8个数据，FIFO状态应该为**满**。



3、当FIFO读出8个数据时，写指针还指向1_000B，读指针为1_000B，此时FIFO里面没有数据，FIFO的状态为**空**。



4、向FIFO再写入8个数据，写指针指向**0_000B**，读指针还为**1_000B**，此时FIFO里有8个数据，FIFO状态应该为**满**。



通过上述观察发现地址除了最高位余下的3位为地址0-7的循环。

观察上图FIFO满时读写地址的高2位不同其余位均相同；FIFO空时读写地址完全相同。

所以

判断FIFO空的条件：读写地址完全相同。

判断FIFO满的条件：读写地址的最高位不同其余位均相同。

同步FIFO设计：

设计深度为16，数据位宽为16的同步FIFO

```

1 //深度16 位宽16
2 module top
3 #
4 (
5     parameter    depth = 16,
6     parameter    width = 16

```

```

7  )
8  (
9      input          clk,
10     input          rst_n,
11     input          wr_en,
12     input  [width-1:0] data_in,
13
14     input          rd_en,
15     output  [width-1:0] data_out,
16
17     output          full,
18     output          empty
19
20 );
21
22 reg [width-1:0] mem [depth:0]; //深度16 位宽16的mem
23 reg [4:0]      waddr;          //写指针
24 reg [4:0]      raddr;          //读指针
25 reg [width:0]  temp_rdata;
26
27
28 //写指针
29 always@(posedge clk or negedge rst_n)
30 begin
31     if(!rst_n)
32         waddr <= 5'b0_0000;
33     else if(wr_en == 1'b1)
34         waddr <= waddr + 1;
35     else
36         waddr <= waddr;
37 end
38
39 //读指针
40 always@(posedge clk or negedge rst_n)
41 begin
42     if(!rst_n)
43         raddr <= 5'b0_0000;
44     else if(rd_en == 1'b1)
45         raddr <= raddr + 1;

```

```

46     else
47         raddr <= raddr;
48     end
49
50 //写数据
51 always@(posedge clk)
52 begin
53     mem[waddr] <= data_in;
54 end
55
56 //读数据 1 latency
57 assign data_out = temp_rdata;
58 always@(posedge clk or negedge rst_n)
59 begin
60     if(!rst_n)
61         temp_rdata <= {width{1'b0}};
62     else
63         temp_rdata <= mem[raddr];
64 end
65
66 //空信号判断
67 assign empty = (waddr == raddr)?1'b1:1'b0;
68
69 //满信号判断
70 assign full = ({~waddr[4],waddr[3:0]} == raddr)?1'b1:1'b0;
71
72 endmodule
73

```

testbench:

```

1 module top_test;
2
3 reg      clk;
4 reg      rst_n;
5 reg      wr_en;
6 reg[15:0] data_in;
7 reg      rd_en;
8 wire      full;

```

```

9  wire      empty;
10
11  top u1
12  (
13      .clk      (clk),
14      .rst_n     (rst_n),
15      .wr_en     (wr_en),
16      .data_in   (data_in),
17      .rd_en     (rd_en),
18      .full      (full),
19      .empty     (empty)
20
21  );
22
23  initial
24  begin
25      clk      = 0;
26      rst_n    = 0;
27      wr_en    = 0;
28      rd_en    = 0;
29      data_in  = 16'h0000;
30
31      repeat(5)@(posedge clk);
32      rst_n    = 1;
33      repeat(5)@(posedge clk);
34      drive_w();
35      repeat(5)@(posedge clk);
36      drive_r();
37
38  end
39
40  task    drive_w;
41      integer i;
42      begin
43          for(i=0;i<16;i=i+1)
44              begin
45                  wr_en    <= 1;
46                  data_in <= i[15:0];
47                  @(posedge clk);

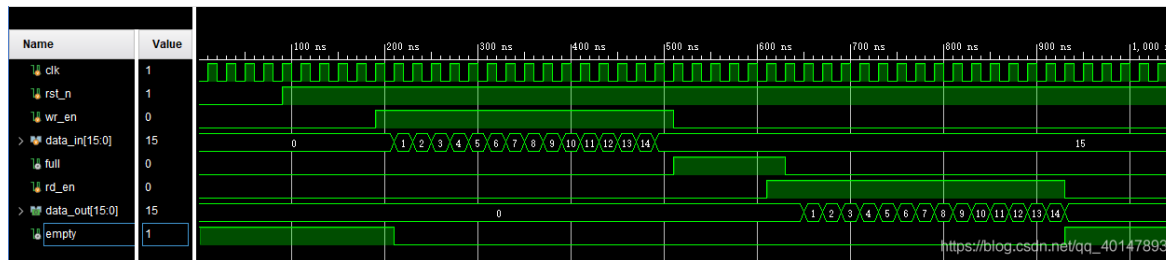
```

```

48         wr_en    <= 0;
49     end
50 end
51 endtask
52
53 task    drive_r;
54     integer i;
55     begin
56         for(i=0;i<16;i=i+1)
57             begin
58                 rd_en    <= 1;
59                 @(posedge clk);
60                 rd_en    <= 0;
61             end
62     end
63 endtask
64
65 always # 10 clk = ~clk;
66
67 endmodule

```

同步FIFO波形：



2.异步FIFO

异步FIFO的写时钟和读时钟为异步时钟，FIFO内部的写逻辑和读逻辑的交互需要异步处理，异步FIFO常用于跨时钟域交互。因为FIFO为异步设计，读写都有不同的时钟，读写是相互独立的，因为空满信号的判断是借助于读写地址进行的，所以涉及到了不同的时钟域，当二进制读地址从0111向1000变化时，地址所有位都要变化，如果写时钟恰好在读地址的变化时刻采样，写所得到的读地址值有可

能是从0000到1111中的任何一个（即**亚稳态的发生**），可以采用格雷码形式。**由于格雷码每次只变化一位，采用格雷码可以降低亚稳态的发生概率。**

方法：两级寄存器同步 + 格雷码

读指针同步到写时钟域、写指针同步到读时钟域时要通过两级D触发器来进行同步处理。

由于设计的时候读写指针用了至少两级寄存器同步，同步会消耗至少两个时钟周期，势必会使得判断空或满有所延迟，这会不会导致设计出错呢？

异步FIFO通过比较读写指针进行满空判断，但是读写指针属于不同的时钟域，所以在比较之前需要先将读写指针进行同步处理，

① **将写指针同步到读时钟域再和读指针比较进行FIFO空状态判断**，因为在同步写指针时需要时间，而在这个同步的时间内有可能还会写入新的数据，因此同步后的写指针一定是小于或者等于当前实际的写指针，所以此时判断FIFO为空不一定是真空，这样更加保守，一共不会出现空读的情况，虽然会影响FIFO的性能，但是并不会出错。

②**将读指针同步到写时钟域再和写指针比较进行FIFO满状态判断**，同步后的读指针一定是小于或者等于当前的读指针，所以此时判断FIFO为满不一定是真满。

这样更保守，这样可以保证FIFO的特性：**FIFO空之后不能继续读取，FIFO满之后不能继续写入。总结来说异步逻辑转到同步逻辑不可避免需要额外的时钟开销**，这会导致满空趋于保守，但是保守并不等于错误，这么写会稍微有性能损失，但是不会出错。

格雷码：

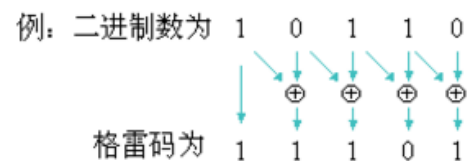
码表：

十进制数	二进制码	格雷码
0	0000	0000
1	0001	0001
2	0010	0011
3	0011	0010
4	0100	0110
5	0101	0111

十进制数	二进制码	格雷码
6	0110	0101
7	0111	0100
8	1000	1100
9	1001	1101
10	1010	1111
11	1011	1110
12	1100	1010
13	1101	1011
14	1110	1001
15	1111	1000

版权：Crazy_M
格雷码

二进制至格雷码的转换：



二进制转格雷码代码：

```

1 module top(
2
3     input  sys_clk,
4     input  rst_n,
5
6     input  [4:0]bin_code,
7     output [4:0]gray_code
8
9 );

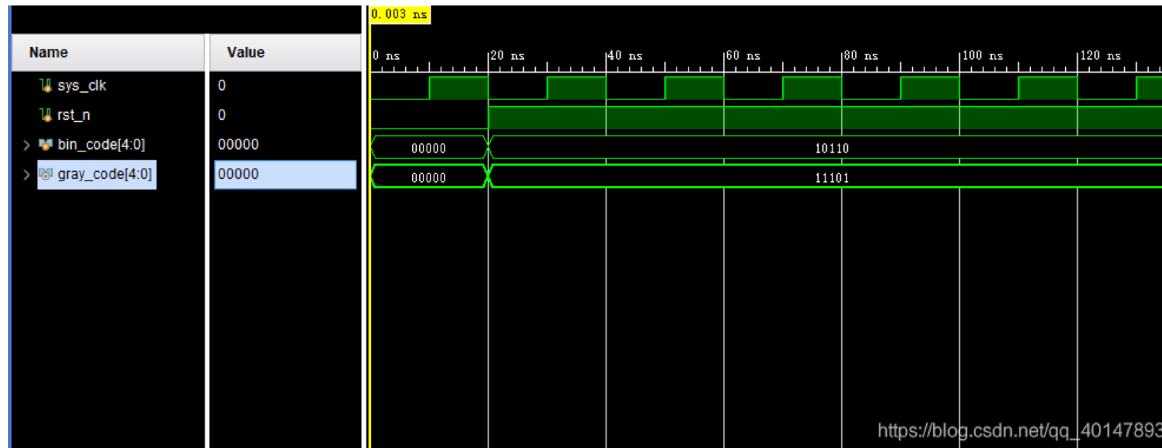
```

```

10
11 assign gray_code = (bin_code>>1)^bin_code;
12
13 endmodule

```

波形：



例如深度为8采用格雷码判断空满的条件：

引入格雷码（原因往下看）：

FIFO操作	写地址指向 (B)	Gray	读地址指向 (B)	Gray	FIFO状态
空FIFO	0_000	0000	0_000	0000	空
写入8个数据	1_000	1100	0_000	0000	满
读出8个数据	1_000	1100	1_000	1100	空
写入8个数据	0_000	0000	1_000	1100	满
读出8个数据	0_000	0000	0_000	0000	空

判断异步FIFO空的条件：读写地址（格雷码）完全相同。

判断异步FIFO满的条件：读写地址（格雷码）的高2位不同，其余位均相同。

异步FIFO代码设计：

```

1 //深度为16 宽度为16
2 module top
3 #
4 (
5     parameter      depth = 16,
6     parameter      width = 16
7 )
8 (
9     input           wr_clk,
10    input           rst_n,
11    input           wr_en,
12    input  [width-1:0] data_in,
13
14    input           rd_clk,
15    input           rd_en,
16    output  [width-1:0] data_out,
17
18    output          full,
19    output          empty
20
21 );
22
23 reg [width-1:0] mem [depth-1:0];
24
25 reg [width-1:0] temp_data;
26
27 reg [4:0]  waddr;
28 wire[4:0]  waddr_gray;
29 reg [4:0]  waddr_w2r1;
30 reg [4:0]  waddr_w2r2;
31
32 reg [4:0]  raddr;
33 wire[4:0]  raddr_gray;
34 reg [4:0]  raddr_r2w1;

```

```

35 reg [4:0]   raddr_r2w2;
36
37 //写指针
38 always@(posedge wr_clk or negedge rst_n)
39 begin
40     if(!rst_n)
41         waddr <= 5'b0_0000;
42     else if(wr_en)
43         waddr <= waddr + 1;
44     else
45         waddr <= waddr;
46 end
47 //写指针转换为格雷码
48 assign waddr_gray = (waddr>>1)^waddr;
49 //写指针同步到读时钟域
50 always@(posedge wr_clk or negedge rst_n)
51 begin
52     if(!rst_n)
53         {waddr_w2r2,waddr_w2r1} <= {2{5'b0_0000}};
54     else
55         {waddr_w2r2,waddr_w2r1} <= {waddr_w2r1,waddr_gray};
56 end
57
58 //读指针
59 always@(posedge rd_clk or negedge rst_n)
60 begin
61     if(!rst_n)
62         raddr <= 5'b0_0000;
63     else if(rd_en)
64         raddr <= raddr + 1;
65     else
66         raddr <= raddr;
67 end
68 //读指针转换为格雷码
69 assign raddr_gray = (raddr>>1)^raddr;
70 //读指针同步到写时钟域
71 always@(posedge rd_clk or negedge rst_n)
72 begin
73     if(!rst_n)

```

```

74         {raddr_r2w2,raddr_r2w1} <= {2{5'b0_0000}};
75     else
76         {raddr_r2w2,raddr_r2w1} <= {raddr_r2w1,raddr_gray};
77 end
78
79
80 //写数据
81 always@(posedge wr_clk)
82 begin
83     mem[waddr] <= data_in;
84 end
85
86 //读数据 1 latency
87 assign data_out = temp_data;
88 always@(posedge rd_clk or negedge rst_n)
89 begin
90     if(!rst_n)
91         temp_data <= {width{1'b0}};
92     else if(rd_en==1)
93         temp_data <= mem[raddr];
94     else
95         temp_data <= temp_data;
96 end
97
98
99 //空信号检测(写指针同步到rd_clk后与读指针进行比较)
100 assign empty = (waddr_w2r2 == raddr_gray)?1'b1:1'b0;
101
102 //满信号检测(读指针同步到wr_clk后与写指针进行比较)
103 assign full = ({~(raddr_r2w2[4:3]),raddr_r2w2[2:0]}==waddr_gray[4:0])?1'b1:1'b0;
104
105 endmodule
106

```

testbench

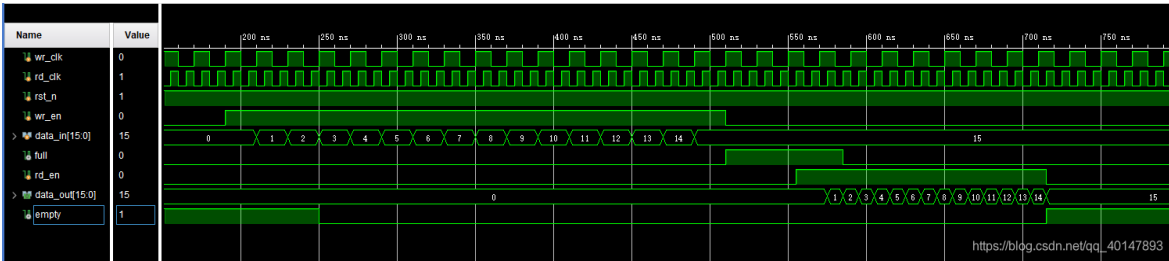
```

1 module top_test;
2
3 reg         wr_clk;

```

```
4 reg      rd_clk;
5 reg      rst_n;
6 reg      wr_en;
7 reg[15:0] data_in;
8 reg      rd_en;
9 wire      full;
10 wire     empty;
11
12 top u1
13 (
14     .wr_clk  (wr_clk),
15     .rd_clk  (rd_clk),
16     .rst_n   (rst_n),
17     .wr_en   (wr_en),
18     .data_in (data_in),
19     .rd_en   (rd_en),
20     .full    (full),
21     .empty   (empty)
22
23 );
24
25 initial
26 begin
27     wr_clk  = 0;
28     wr_clk  = 0;
29     rd_clk  = 0;
30     rst_n   = 0;
31     wr_en   = 0;
32     rd_en   = 0;
33     data_in = 16'h0000;
34
35     repeat(5)@(posedge wr_clk);
36     rst_n   = 1;
37     repeat(5)@(posedge wr_clk);
38     drive_w();
39     repeat(5)@(posedge rd_clk);
40     drive_r();
41
42 end
```

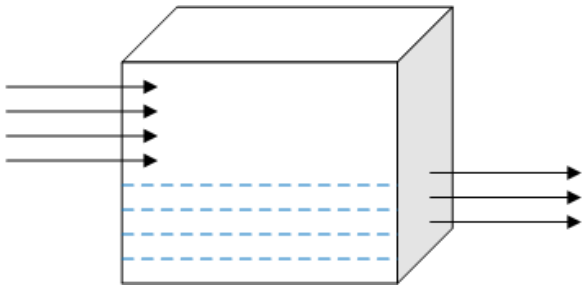
```
43
44 task    drive_w;
45     integer i;
46     begin
47         for(i=0;i<16;i=i+1)
48             begin
49                 wr_en    <= 1;
50                 data_in <= i[15:0];
51                 @(posedge wr_clk);
52                 wr_en    <= 0;
53             end
54         end
55     endtask
56
57 task    drive_r;
58     integer i;
59     begin
60         for(i=0;i<16;i=i+1)
61             begin
62                 rd_en    <= 1;
63                 @(posedge rd_clk);
64                 rd_en    <= 0;
65             end
66         end
67     endtask
68
69 always # 10 wr_clk = ~wr_clk;
70 always # 5  rd_clk = ~rd_clk;
71
72 endmodule
```



五、FIFO最小深度计算

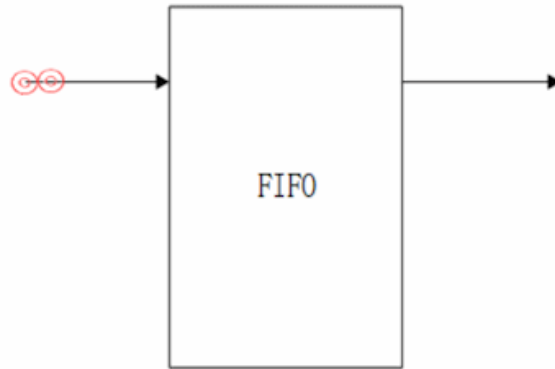
1.FIFO最小深度

在数据的传输中当读速率慢于写速率时，FIFO便可被用作系统中的缓冲元件或队列，类似于水流经过一个缓冲区，上游水流速度快，下游水流速度慢，部分水就可以被缓存在中间的缓冲区，但是上游与下游的水速差距过大，就会导致流入中间缓存区的水超出了缓存区承受能力导致水的溢出，不能顺利到达下游。在上游水流速度最大，下游水流速度最小的情况下，中间缓冲区不能溢出的大小就是我们关系的问题。



同理，读写速率差距过大，FIFO深度太小，就会导致一段时间内向FIFO灌输的数据超出了FIFO的承受能力，就会导致数据的丢失，该容量取决于读写数据的速率。所以在写速率最大，读速率最小时，这时正好不导致数据溢出的FIFO的深度就是我们要计算的FIFO最小深度。

FIFO写入速率快，读出速度慢模型：



最简单的写快读慢的实例，FIFO写入一个数据需要1s，读取一个数据需要2s，一共传输6个数据。(下图)

6个数据写入需要时间为： $6 \times 1 = 6 \text{ s}$

6s读取的数据为： $6 / 2 = 3$ 个

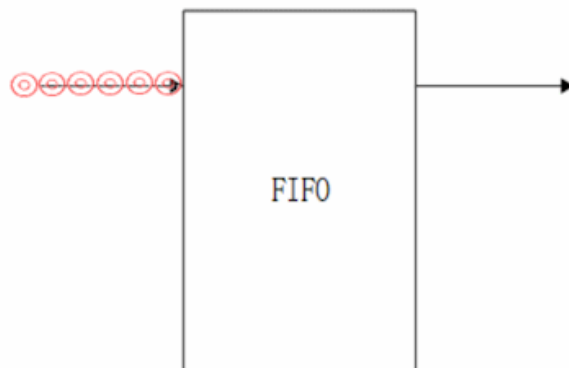
6s FIFO内剩余数据的个数： $6 - 3 = 3$ 个

6s才能读取3个数据，剩余的数据3个在FIFO中缓冲，在6s的之后的时间才能被缓慢读出。

仔细观察下图(写快读慢)，FIFO中最多存在3个数据，该容量就是FIFO的最小深度。

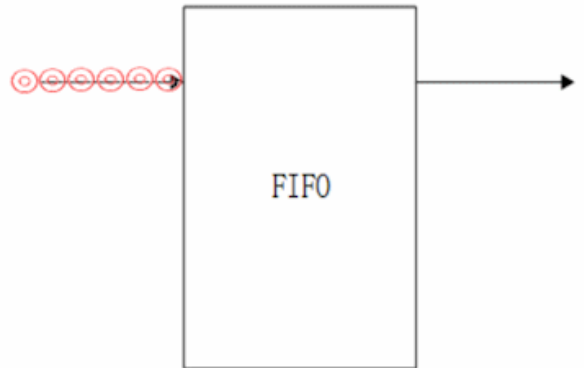
该FIFO的最小深度 = $6 - 3 = 3$ (如果深度小于3，则会导致数据丢失)。

总结：要确定FIFO的深度，关键在于计算出在突发读写的这段时间内有多少数据没有被读走。也就是说FIFO的最小深度就等于没有被读走的数据个数。



FIFO写入与读取速度相同：

读数据速率比写数据快，因此FIFO只起到变换时钟域的作用，FIFO最下深度为1即可。

**2.示例分析****示例一：****写入速率 > 读取速率**

A/D采样率为50MHz，读A/D的速率为40MHz，要不丢失地将10万个采样数据送入FPGA，在A/D和FPGA之间至少加多大容量（深度）的FIFO才可以？

写一个数据所需的时间为：

$$1/50MHz = 0.02 \times 10^{-6}s$$

写完所有的数据所需的时间为：

$$100,000 \times 0.02 \times 10^{-6} = 0.002s$$

读一个数据所需的时间为：

$$1/40MHz = 0.025 \times 10^{-6}s$$

0.002S内读走的数据为：

$$0.002 / (0.025 \times 10^{-6}) = 0.08 \times 10^6 = 80,000$$

则FIFO的最小深度为：

$$100,000 - 80,000 = 20,000 \text{ (0.002s还没有被读走的数据的个数)}$$

(如果读比写慢几拍，则深度比20,000略大)

示例二：

FIFO写入时钟为100MHz，读取时钟为80Mhz，分以下情况分析。

①：传输2000个数据，FIFO最小深度为多少？

100MHz时钟写入一个数据需要的时间为

$$1/100MHz = 10ns$$

写入2000个数据所需的时间为

$$2000 \times 10 = 20000ns$$

80MHz时钟读出一个数据所需的时钟

$$1/80MHz = 12.5ns$$

20000ns读出的数据个数为

$$20000ns/12.5ns = 1600$$

剩余FIFO中没读走的数据个数为

$$2000 - 1600 = 400$$

所以FIFO的最小深度为400。

②：100个时钟写入80个数据，1个时钟读1个数据，求FIFO的最小深度

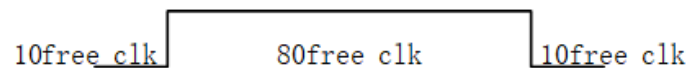
100MHz时钟写入一个数据需要的时间为

$$1/100MHz = 10ns$$

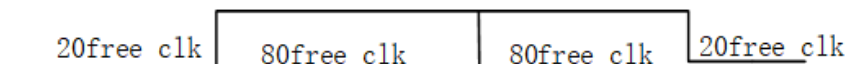
80MHz时钟读出一个数据所需的时钟

$$1/80MHz = 12.5ns$$

100个时钟写入80个数据，可以以下理解，100个时钟包含80个有效写时钟，20个空闲时钟。



但是在什么情况下会出现突发写入数据最多呢，在两个100个时钟，前一个100个时钟的后80个时钟在写数据，后100个时钟的前80个数据在写数据，也就是如下所示：



在这种情况下，以100MHz的时钟一次写入了160个数据。

写入160个数据所需的时间为

$$160 \times 10ns = 1600ns$$

1600ns读取的数据个数为

$$1600ns / 12.5ns = 128$$

FIFO内剩余的数据为

$$160 - 128 = 32$$

所以FIFO最小深度为32

③：100个时钟写入80个数据，3个时钟读1个数据，求FIFO的最小深度

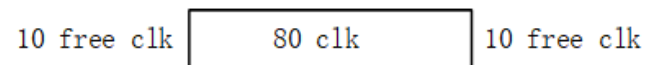
100MHz时钟写入一个数据需要的时间为

$$1/100MHz = 10ns$$

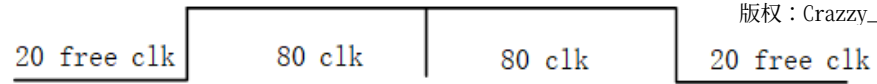
80MHz时钟读出一个数据所需的时钟

$$(1/80MHz) \times 3 = 12.5 \times 3ns$$

100个时钟写入80个数据，可以以下理解，100个时钟包含80个有效写时钟，20个空闲时钟。



但是在什么情况下会出现突发写入数据最多呢，在两个100个时钟，前一个100个时钟的后80个时钟在写数据，后100个时钟的前80个数据在写数据，也就是如下所示：



在这种情况下，以100MHz的时钟一次写入了160个数据。

写入160个数据所需的时间为

$$160 \times 10ns = 1600ns$$

1600ns读取的数据个数为

$$1600ns / (12.5 \times 3)ns = 42.66$$

FIFO内剩余的数据为

$$160 - 42.66 = 117.33$$

所以FIFO最小深度为118(FIFO一般设置为2的n次幂(128))。

④：100个时钟写入80个数据，3个时钟读2个数据，求FIFO的最小深度

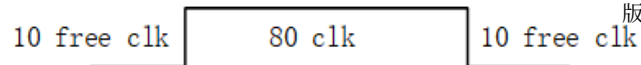
100MHz时钟写入一个数据需要的时间为

$$1/100MHz = 10ns$$

80MHz时钟读出一个数据所需的时钟

$$(1/80MHz) \times 3/2 = 12.5 \times 3/2ns$$

100个时钟写入80个数据，可以以下理解，100个时钟包含80个有效写时钟，20个空闲时钟。



但是在什么情况下会出现突发写入数据最多呢，在两个100个时钟，前一个100个时钟的后80个时钟在写数据，后100个时钟的前80个数据在写数据，也就是如下所示：



在这种情况下，以100MHz的时钟一次写入了160个数据。

写入160个数据所需的时间为

$$160 \times 10ns = 1600ns$$

1600ns读取的数据个数为

$$1600ns / (12.5 \times 3/2ns)ns = 85.33$$

FIFO内剩余的数据为

$$160 - 42.66 = 74.66$$

所以FIFO最小深度为75(FIFO一般设置为2的n次幂(128))。

六、逻辑电平

1.TTL电平标准

TTL(Transistor-Transistor Logic)，晶体管-晶体管逻辑电平。

工作于TTL接口标准下的数字电路，其内部的有源器件的标准电源供给应为5V。输出输入如下：

输入	逻辑1 的电压要求为大于或等于2.0V，逻辑0 的电压要求为小于或等于0.8V
----	---

输入	逻辑1 的电压要求为大于或等于2.0V，逻辑0 的电压要求为小于或等于0.8V
输出	逻辑1 的电压要求为大于或等于2.4V，逻辑0 的电压要求为小于或等于0.5V

对于输出、输入端的电压要求，可以看出输出端的电压输出要求要比输入端的双阈值判定标准更加严格，这样做主要是考虑到噪声的干扰以及电信号在输出与输入间的传输速度，从而让双阈值判定更加可靠，下同。

2.LVTTL电平标准

由于2.4V到5V之间还有很大空间，这对改善噪声干扰并没有明显的提高，而且会增加系统的功耗，并且由于数字逻辑1、0之间电平相差较大，还会影响到数字电路的响应速度。所以产生了LVTTL-Low Voltage Transistor-Transistor Logic，即低压TTL电平标准。LVTTL又有LVTTL3V3和LVTTL2V5：

(1).LVTTL3V3

内部有源器件的标准电源供给为3.3V，如我们常用的部分单片机，输出、输入情况如下：

输入	逻辑1 的电压要求为大于或等于2.0V，逻辑0 的电压要求为小于或等于0.8V
输出	逻辑1 的电压要求为大于或等于2.4V，逻辑0 的电压要求为小于或等于0.4V

(2).LVTTL2V5

内部有源器件的标准电源供给为2.5V，输出、输入情况如下：

输入	逻辑1 的电压要求为大于或等于1.7V，逻辑0 的电压要求为小于或等于0.7V
输出	逻辑1 的电压要求为大于或等于2.0V，逻辑0 的电压要求为小于或等于0.2V

TTL使用注意：TTL电平一般过冲都会比较严重，可能在始端串22欧或33欧电阻；TTL电平输入脚悬空时是内部认为是高电平。要下拉的话应用1k以下电阻下拉。TTL输出不能驱动CMOS输入。

3.CMOS电平标准

CMOS(Complementary Metal Oxide Semiconductor)，这种接口电平标准初衷是用于基于CMOS、PMOS组成的MOS管结构的数字系统之间的。工作于CMOS接口标准下的数字电路，其内部有源器件电源供给为5V，输入输出如下：

输入	逻辑1 的电压要求为大于或等于1.45V， 逻辑0 的电压要求为小于或等于0.5V
输出	逻辑1 的电压要求为大于或等于3.5V， 逻辑0 的电压要求为小于或等于1.5V

COMS与TTL接口相比，有了更大的噪声容限，并且其输入阻抗也远大于TTL输入阻抗。

4.LVCOMS电平标准

同TTL一样，鉴于功耗和响应速度的考虑，CMOS也同样衍生出了LVCOMS接口标准，并且由于COMS管相对于三极管的导通门限更加低，因此LVCOMS比LVTTTL更容易使用较低的电压进行通信。

(1).LVCOMS3V3

内部有源器件的标准电源供给为Vcc=3.3V

输入	逻辑1 的电压要求为大于或等于2.0V， 逻辑0 的电压要求为小于或等于0.7V
输出	逻辑1 的电压要求为大于或等于3.2V， 逻辑0 的电压要求为小于或等于0.4V

(2).LVCOMS2V5

内部有源器件的标准电源供给为Vcc=2.5V

输入	逻辑1 的电压要求为大于或等于1.7V， 逻辑0 的电压要求为小于或等于0.7V
输出	逻辑1 的电压要求为大于或等于2.0V， 逻辑0 的电压要求为小于或等于0.4V

(3).LVCOMS1V8

内部有源器件的标准电源供给为Vcc=1.8V，当然这是由一定容忍度的，与之前介绍的电平标准不同，这个容忍度会影响它的输入输出情况：

输入	逻辑1 的电压要求为大于或等于0.65倍的Vcc， 逻辑0 的电压要求为小于或等于0.35倍的Vcc
输出	逻辑1 的电压要求为大于或等于Vcc-0.45V， 逻辑0 的电压要求为小于或等于0.45V

如果Vcc精确为1.8V：

输入	逻辑1 的电压要求为大于或等于1.17V， 逻辑0 的电压要求为小于或等于0.63V
----	--

输出	逻辑1 的电压要求为大于或等于1.35V，逻辑0 的电压要求为小于或等于0.65V
----	---

(4).LVCOMS1V5

内部有源器件的标准电源供给为Vcc=1.5V

输入	逻辑1 的电压要求为大于或等于0.65倍的Vcc，逻辑0 的电压要求为小于或等于0.35倍的Vcc
输出	LVCOMS1V5没有明确要求，但是肯定是逻辑1越接近Vcc越好，逻辑0越接近0V越好

如果Vcc精确为1.5V：

输入	逻辑1 的电压要求为大于或等于0.975V，逻辑0 的电压要求为小于或等于0.525V
输出	LVCOMS1V5没有明确要求，但是肯定是逻辑1越接近Vcc越好，逻辑0越接近0V越好

(5).LVCOMS1V2

内部有源器件的标准电源供给为Vcc=1.2V

输入	逻辑1 的电压要求为大于或等于0.65倍的Vcc，逻辑0 的电压要求为小于或等于0.35倍的Vcc
输出	LVCOMS1V2没有明确要求，但是肯定是逻辑1越接近Vcc越好，逻辑0越接近0V越好

如果Vcc精确为1.2V：

输入	逻辑1 的电压要求为大于或等于0.78V，逻辑0 的电压要求为小于或等于0.42V
输出	LVCOMS1V2没有明确要求，但是肯定是逻辑1越接近Vcc越好，逻辑0越接近0V越好

CMOS使用注意：CMOS结构内部寄生有可控硅结构，当输入或输入管脚高于VCC一定值(比如一些芯片是0.7V)时，电流足够大的话，可能引起闩锁效应，导致芯片的烧毁。

5.RS232

RS232总线标准共设有25条信号线，RS232的标准电源供给为±12V或±15V

逻辑1	-15~3V
逻辑0	3~15V

6.RS485

RS485采用差分的形式来传递信息的，因此抗干扰要优于RS232。

逻辑1	两线之间的电压差要求为2~6V之间
逻辑0	两线之间的电压差要求为-6~-2V之间

7.ECL、PECL、LVPECL

ECL：Emitter Coupled Logic 发射极耦合逻辑电路(差分结构)：

速度快，驱动能力强，噪声小，很容易达到几百M的应用。但是功耗大，需要负电源。

Vcc	0V
Vee	-5.2V
VOH	-0.88V
VOL	-1.72V
VIH	-1.24V
VIL	-1.36V

为简化电源，出现了PECL(ECL结构，改用正电压供电)和LVPECL。

PECL：

Vcc	5V
VOH	4.12V
VOL	3.28V

Vcc	5V
VIH	3.78V
VIL	3.64V

LVPECL:

Vcc	3.3V
VOH	2.42V
VOL	1.58V
VIH	2.06V
VIL	1.94V

ECL、PECL、LVPECL使用注意：不同电平不能直接驱动。中间可用交流耦合、电阻网络或专用芯片进行转换。以上三种均为射随输出结构，必须有电阻拉到一个直流偏置电压。(如多用于时钟的LVPECL：直流匹配时用130欧上拉，同时用82欧下拉；交流匹配时用82欧上拉，同时用130欧下拉。但两种方式工作后直流电平都在1.95V左右。)

8.LVDS

前面的电平标准摆幅都比较大，为降低电磁辐射，同时提高开关速度又推出LVDS电平标准。

LVDS：Low Voltage Differential Signaling 差分对输入输出，内部有一个恒流源3.5-4mA，在差分线上改变方向来表示0和1。通过外部的100欧匹配电阻(并在差分线上靠近接收端)转换为±350mV的差分电平。LVDS使用注意：可以达到600M以上，PCB要求较高，差分线要求严格等长，差最好不超过10mil(0.25mm)。100欧电阻离接收端距离不能超过500mil，最好控制在300mil以内。

9.CML

CML：是内部做好匹配的一种电路，不需再进行匹配。三极管结构，也是差分线，速度能达到3G以上。只能点对点传输。

10.GTL

GTL：类似CMOS的一种结构，输入为比较器结构，比较器一端接参考电平，另一端接输入信号。1.2V电源供电。

Vcc	1.2V
VOH	$\geq 1.1V$
VOL	$\leq 0.4V$
VIH	$\geq 0.85V$
VIL	$\leq 0.75V$

PGTL/GTL+：

Vcc	1.5V
VOH	$\geq 1.4V$
VOL	$\leq 0.46V$
VIH	$\geq 1.2V$
VIL	$\leq 0.8V$

11.HSTL

HSTL 是主要用于QDR存储器的一种电平标准：一般有 $V_{CCIO}=1.8V$ 和 $V_{CCIO}=1.5V$ 。和上面的GTL相似，输入为输入为比较器结构，比较器一端接参考电平($V_{CCIO}/2$)，另一端接输入信号。对参考电平要求比较高(1%精度)。

12.SSTL

SSTL 主要用于DDR存储器。和HSTL基本相同。 $V_{CCIO}=2.5V$ ，输入为输入为比较器结构，比较器一端接参考电平1.25V，另一端接输入信号。对参考电平要求比较高(1%精度)。HSTL和SSTL大多用在300M以下。

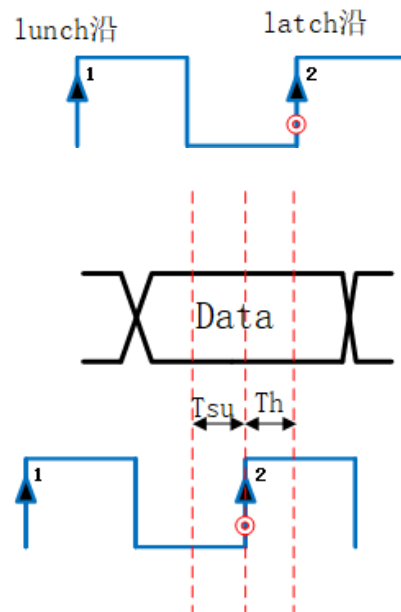
七、建立时间与保持时间

1.建立时间与保持时间概述及模型

建立时间 (Tsu) : 在时钟采样沿之前, 数据必须保持稳定的时间, 该时间量称为建立时间。

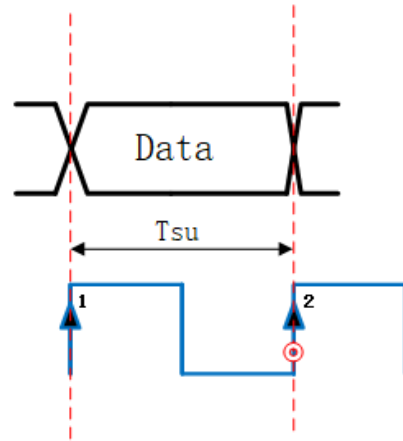
保持时间 (Th) : 在时钟采样沿之后, 数据必须保持稳定的最短时间。

理想最优的建立时间和保持时间出现在数据中间采样的位置, 如下所示, 实质就是使触发器在采样沿得到稳定的数据, 如果数据在时钟上升沿的建立保持时间内{[latch edge-setup](#), [latch edge+hold time](#)}发生跳变, 则会产生亚稳态输出, 即输出值在短时间内处于不确定态, 有可能是1, 有可能是0, 也可能什么都不是, 处于中间态。

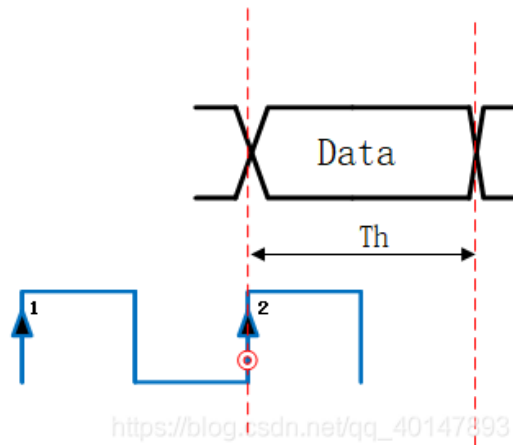


在出现沿打沿的现象时会出现建立时间最大值或者保持时间最大值。

理想建立时间最大值:



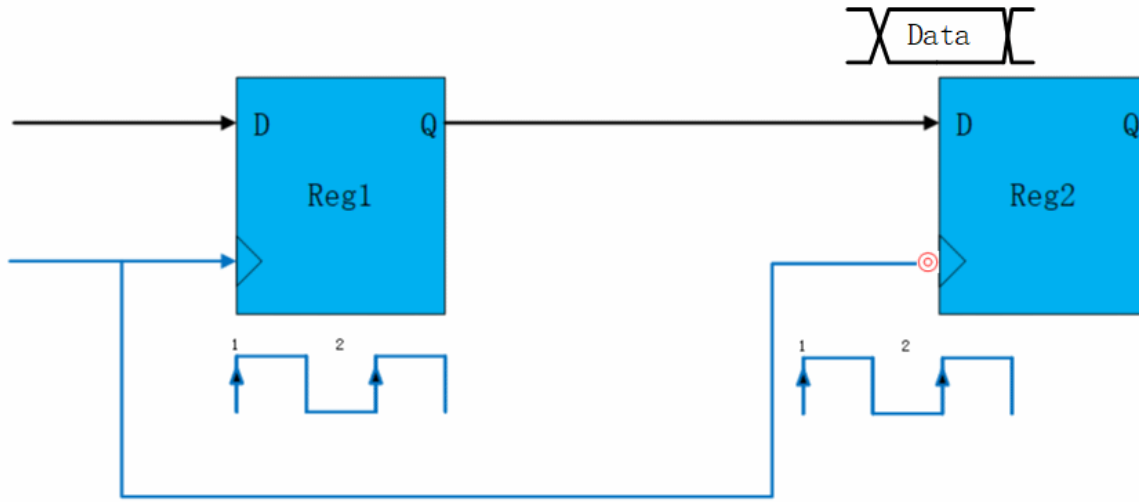
理想保持时间最大值：



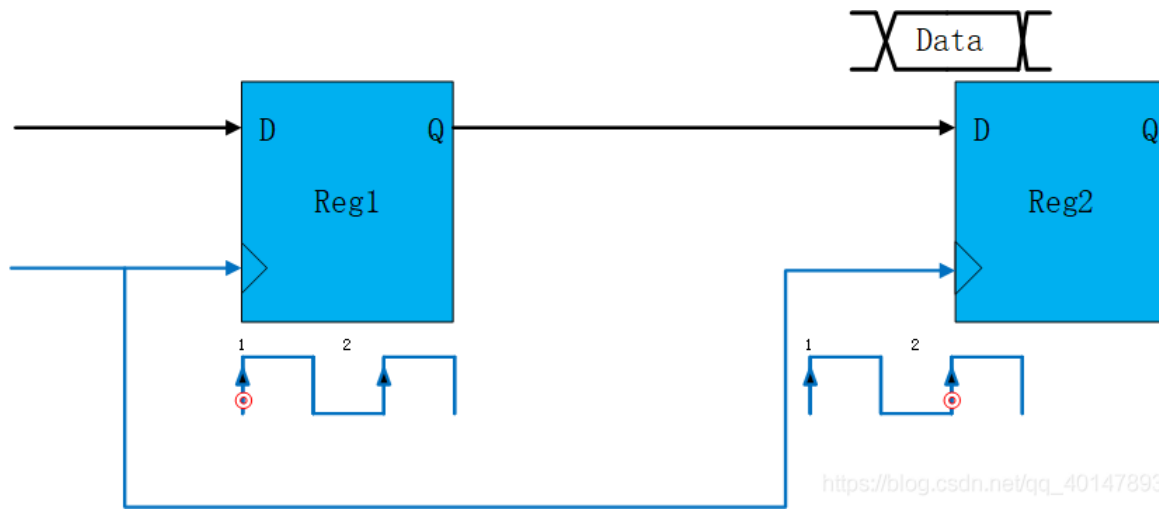
https://blog.csdn.net/qq_40147493

2.寄存器之间数据传输时建立时间与保持时间

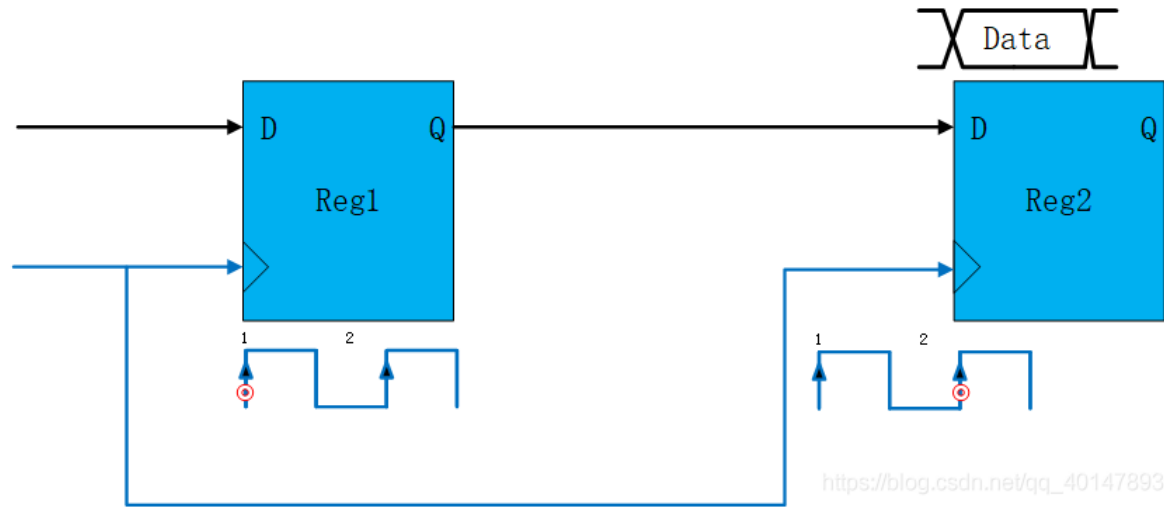
寄存器数据传输的模型：数据从在Reg1到Reg2，理想情况下采样沿到达Reg2时，数据的中间位置到达Reg2的数据端，这样出现最优的建立时间和保持时间，此时采样的数据也是最稳定的，如果时钟早到或者数据早到就会产生余量，严重的情况下就会产生时序为例。



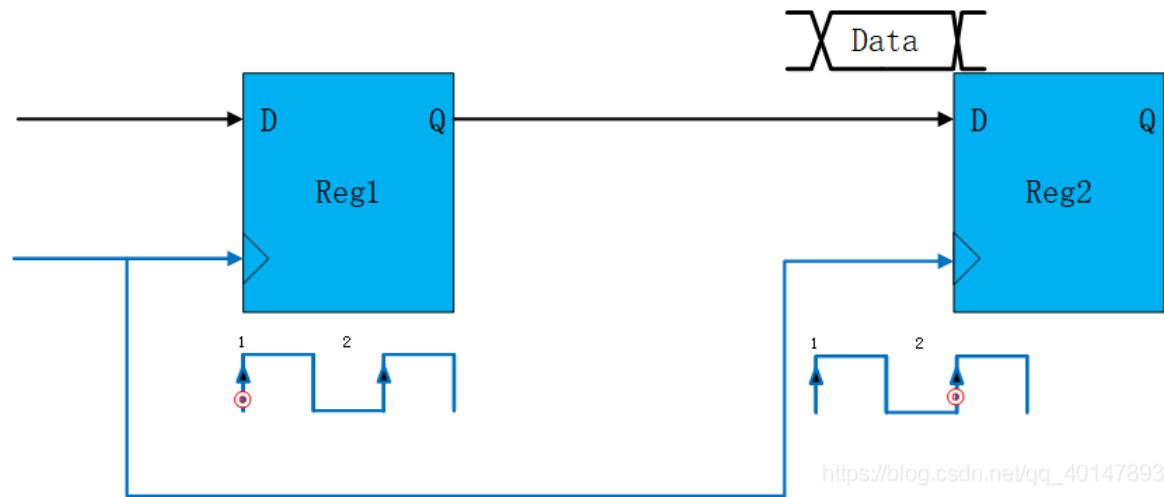
数据和时钟到达Reg2最优情况，采样沿位于数据的中间位置：



出现保持时间最大值采样沿，采样沿与数据起始位置对齐：

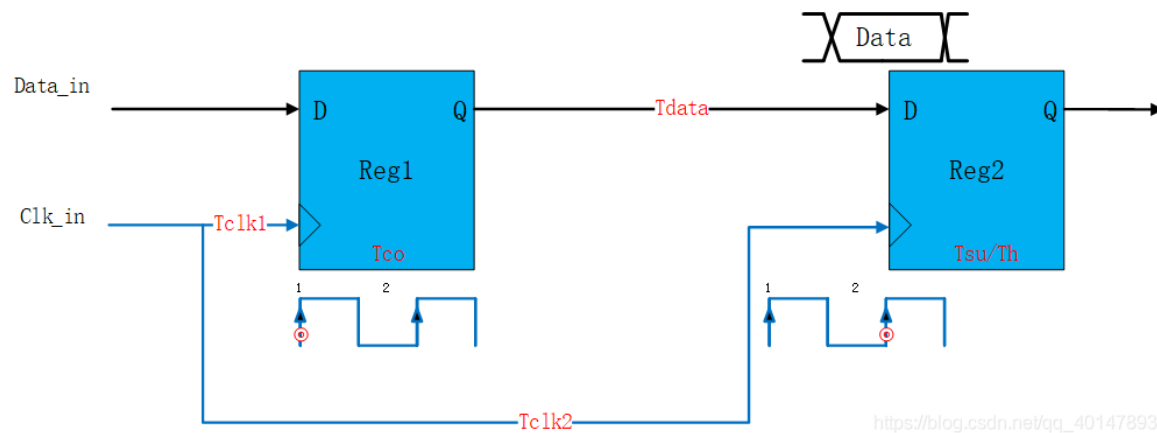


出现建立时间最大值，采样沿与数据结束为止对齐：



3.建立时间余量与保持时间余量计算

在实际的数据传输中器件和线路均会存在延迟，寄存器间数据传输模型如下所示：



https://blog.csdn.net/qq_40147893

Tclk1：时钟到达Reg1的时钟延时。

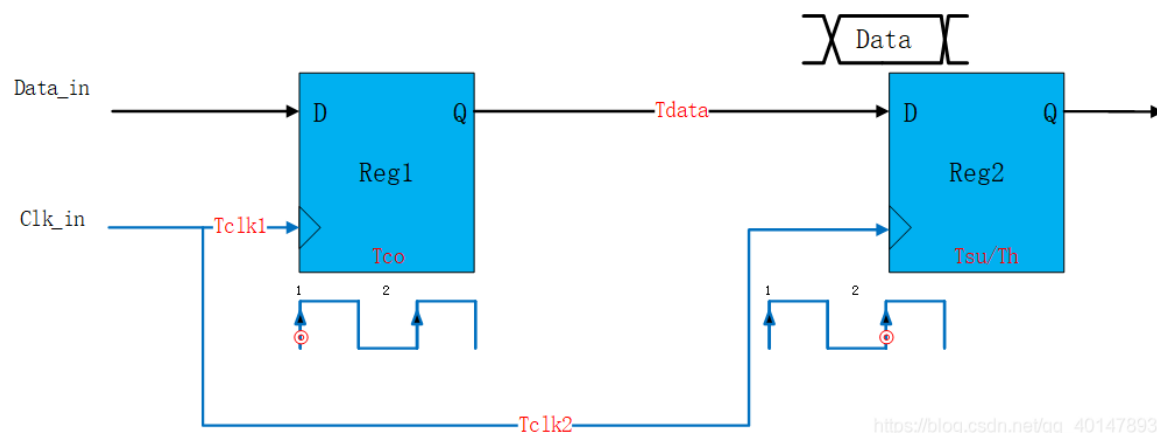
Tco：数据从Reg1器件D到Q的物理延时。

Tclk2：时钟到达Reg2的时钟延时。

Tdata：数据从Reg1的Q端到Reg2的D端的线路延时。

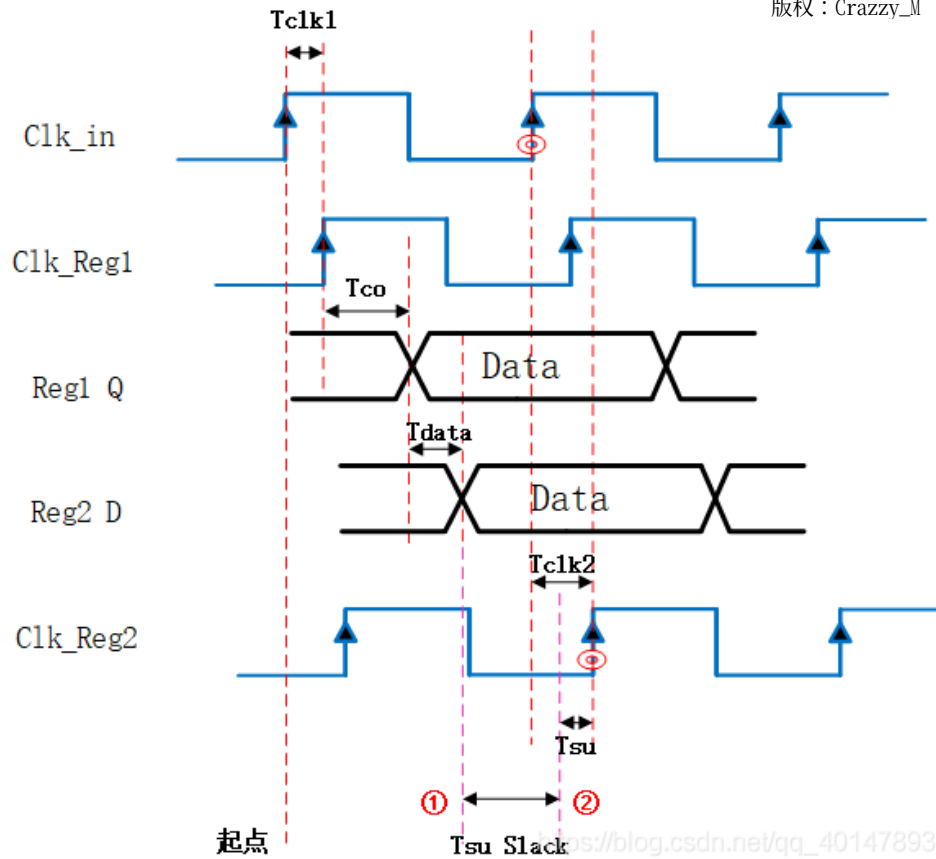
Tsu/Th：保持时间/建立时间。

建立时间余量(Setup Slack)： 要求数据到达时间和数据实际到达时间的差值。



https://blog.csdn.net/qq_40147893

实际考虑线路延时数据从Reg1到Reg2的时序图如下所示：



Clk_in: 输入时钟。

Clk_Reg1: Clk_in经过 T_{clk1} 时延到达Reg1的时钟。

Reg1 Q : 数据从Reg1的D端进入到Q端经过 T_{co} 时延的数据。

Reg1 D : 数据从Reg1的Q端输出经过 T_{data} 时延到达Reg2的D端的数据。

Clk_Reg2: Clk_in经过 T_{clk2} 时延到达Reg2的时钟。

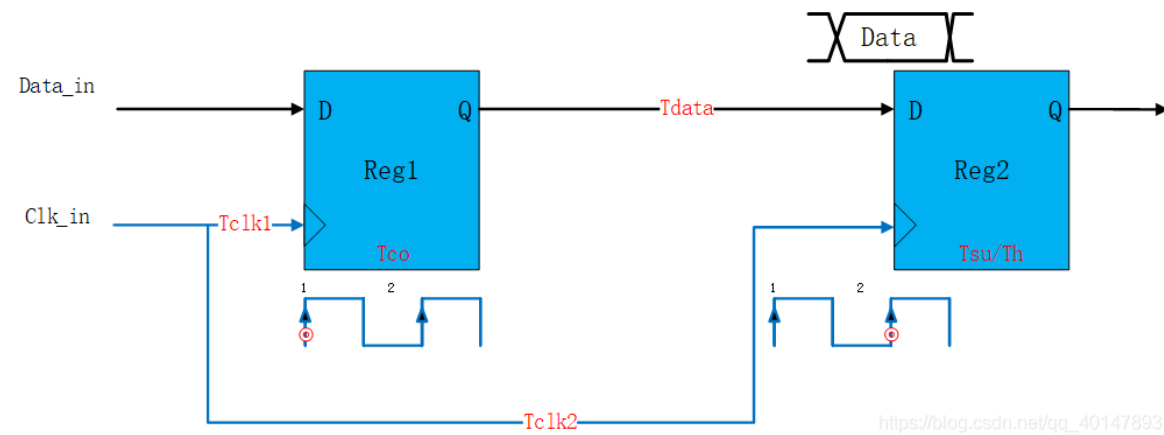
通过上图观察：（Tcycle为一个时钟周期）

要求数据达到的时间 = $T_{cycle} + T_{clk2} - T_{su}$ 。（图中②）

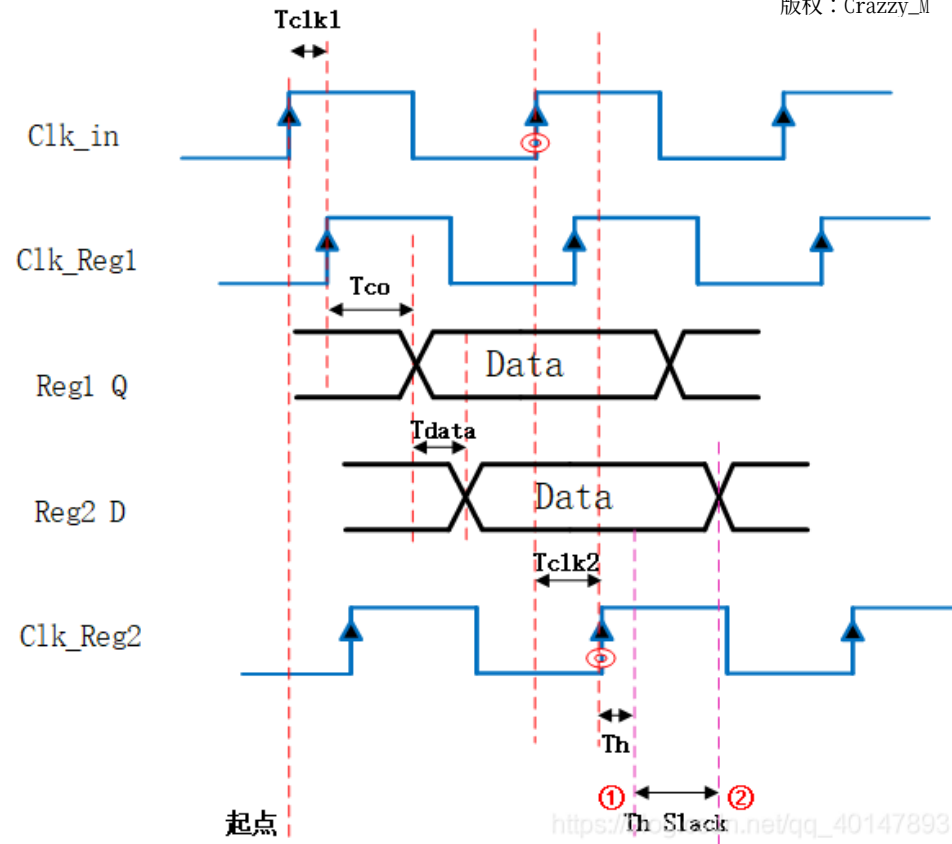
实际数据到达的时间 = $T_{clk1} + T_{co} + T_{data}$ 。（图中①）

建立时间余量（Setup Slack） = $T_{cycle} + T_{clk2} - T_{su} - (T_{clk1} + T_{co} + T_{data})$ 。

保持时间余量(Hold Slack): 数据实际结束位置和要求数据结束位置的差值。 版权：Crazy_M



实际考虑线路延时数据从Reg1到Reg2的时序图如下所示：



Clk_in：输入时钟。

Clk_Reg1：Clk_in经过Tclk1时延到达Reg1的时钟。

Reg1 Q：数据从Reg1的D端进入到Q端经过Tco时延的数据。

Reg1 D：数据从Reg1的Q端输出经过Tdata时延到达Reg2的D端的数据。

Clk_Reg2：Clk_in经过Tclk2时延到达Reg2的时钟。

通过上图观察：（Tcycle为一个时钟周期）

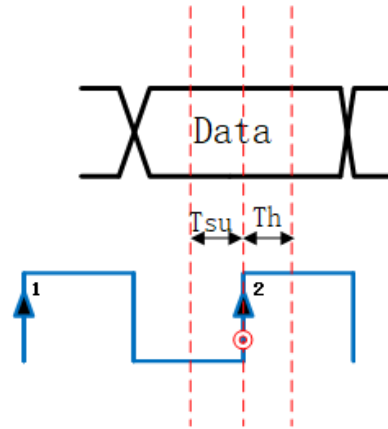
实际数据结束的时间 = Tcycle + Tclk1 + Tco + Tdata。（图中②）

要求数据结束的时间 = Tcycle + Tclk2 + Th。（图中①）

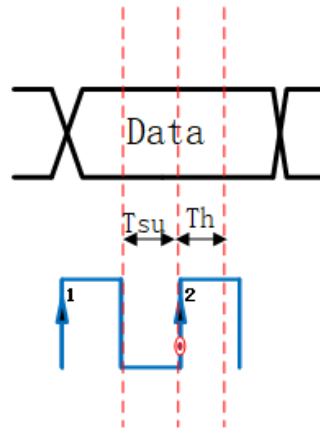
保持时间余量（Told Slack） = Tcycle + Tclk1 + Tco + Tdata - （Tcycle + Tclk2 + Th）。

从上面的公式中可以看出建立时间余量（Told Slack）与时钟周期Tcycle 无关，所以时钟频率也与保持时间余量无关。

4. FPGA允许的最大时钟频率或最小时钟周期



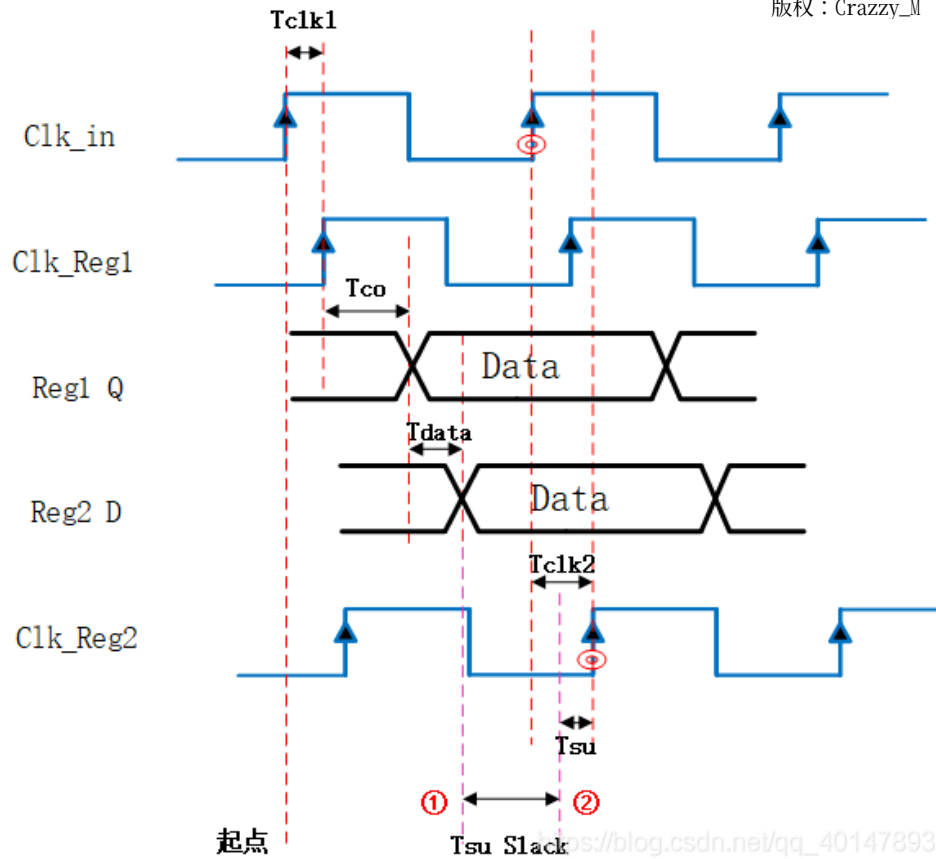
通过上面的推导时钟周期和频率与保持时间无关，所以在 **建立时间余量为0时取得最大时钟频率即最小时钟周期。**



5. 计算FPGA时钟所允许的最大频率或最小时钟周期

例： 计算建立时间余量Setup Slack 与 系统时钟允许的最大时钟频率 F_{max} 。

时钟周期为10ns；Clk in到Reg1的时延为3ns；Clk in到Reg2的时延为2ns；Reg1 Q端到Reg2 D端的时延为3ns； $T_{us}=0.5ns$ ； $T_{co}=1ns$ ；建立时间余量？FPGA允许的最大时钟频率？



建立时间余量 (Setup Slack) = Tcycle + Tclk2 - Tsu - (Tclk1 + Tco + Tdata)。

$$\text{Setup Slack} = 10 + 2 - 0.5 - (3 + 1 + 3) = 4.5\text{ns}$$

建立时间余量为0时取得最大时钟频率即最小时钟周期。

$$F_{\max} = 1 / (\text{Tcycle} - \text{Setup Slack}) = 1 / (10 - 4.5) \text{。}$$

例： 计算保持时间余量Hold Slack 与 系统时钟允许的最大时钟频率 Fmax。

时钟周期为10ns；Clk in到Reg1的时延为3ns；Clk in到Reg2的时延为2ns；Reg1 Q端到Reg2 D端的时延为3ns；Th=0.5ns；Tco=1ns；保持时间余量？FPGA允许的最大时钟频率？

$$\begin{aligned} \text{保持时间余量 (Hold Slack)} &= \text{Tcycle} + \text{Tclk1} + \text{Tco} + \text{Tdata} - (\text{Tcycle} + \text{Tclk2} + \text{Th}) \\ &= \text{Tclk1} + \text{Tco} + \text{Tdata} - \text{Tclk2} - \text{Th}。 \end{aligned}$$

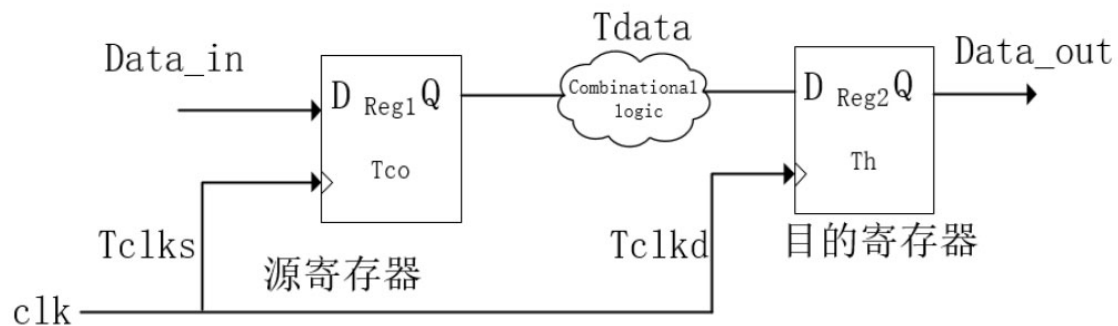
Told Slack = 3 + 1 + 3 - 2 - 0.5 = 4.5。

版权：Crazyy_M

Fmax与保持时间余量无关。

另一种模型（给出时钟偏斜）：

其实和上面一样， $T_{skew} = T_{clk2} - T_{clk1}$



T_{co}：寄存器更新延迟。clock output delay，时钟触发到数据输出的最大延迟时间

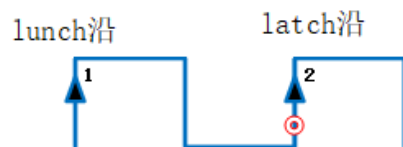
最小时钟周期： $T_{min} = T_{co} + T_{data} + T_{su} - T_{skew}$ 。最快频率 $F_{max} = 1/T_{min}$

$T_{skew} = T_{clk_d} - T_{clk_s}$ 。

CSDN @Crazyy_M

6.违反建立和保持时间会发生什么？

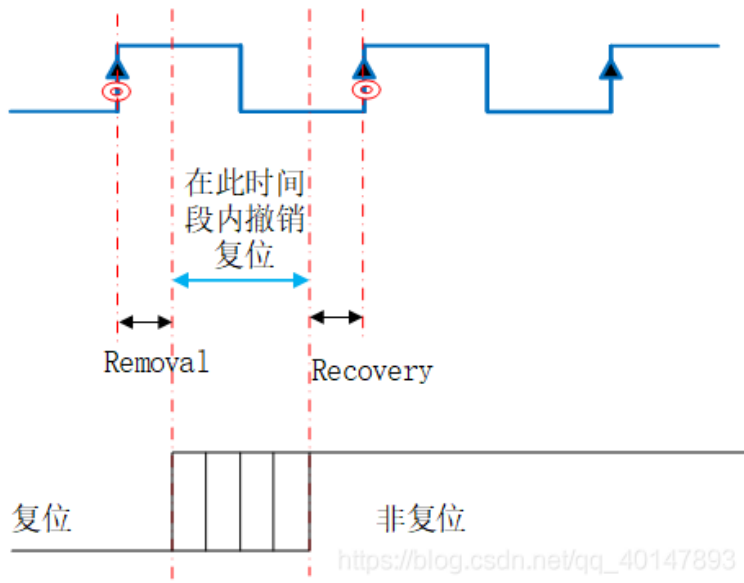
在触发器建立期间，如果数据在时钟上升沿的建立保持时间内{latch edge-setup, latch edge+hold time}发生跳变，则会产生亚稳态输出，即输出值在短时间内处于不确定态，有可能是1，有可能是0，也可能什么都不是，处于中间态。



八、recovery time和removal time

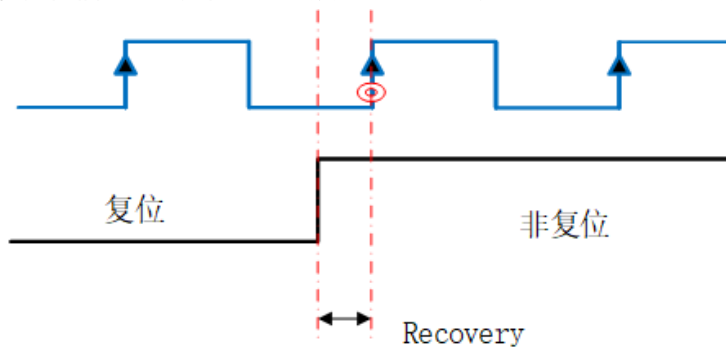
在**同步**电路中，为了防止亚稳态，输入数据需要与时钟满足**setup time**和**hold time**才能进行数据的正常传输。

对于**异步**复位寄存器来说，为了防止亚稳态，同样异步复位信号和时钟满足**recovery time**和**removal time**才能有效进行复位操作和复位释放操作。简而言之，**复位置位信号不要在clk的有效跳边沿发生变化**。



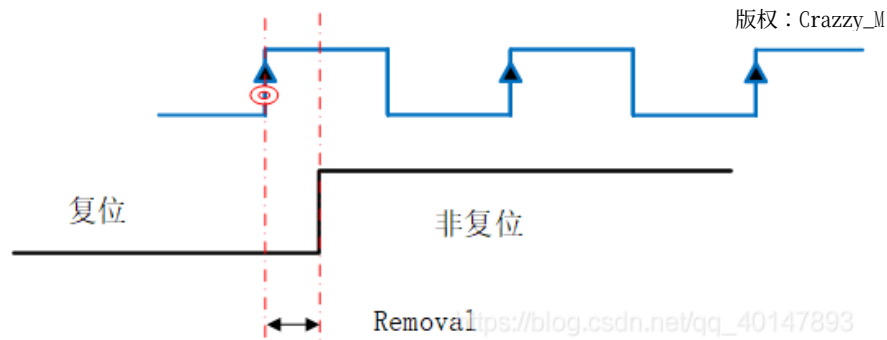
1、recovery time: 恢复时间（不要影响下一个有效时钟沿）

在撤销复位时，复位信号到非复位状态必须在时钟有有效沿到来之前一段时间到达。



2、removal time: 去除时间（不要影响上一个有效时钟沿）

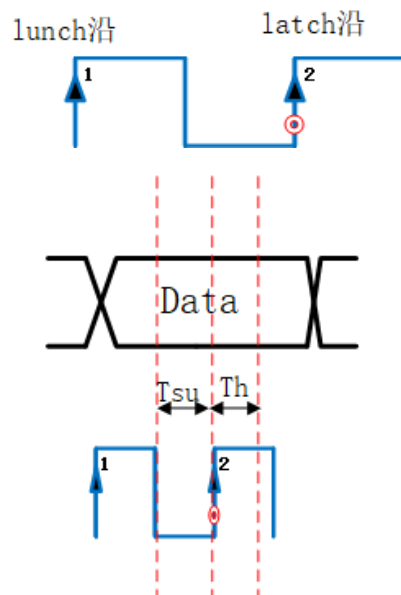
在撤销复位时，在时钟沿来临时之后复位信号还要保持的时间（异步复位信号撤销时离上一个有效时钟沿的最小时间长度）。



九、亚稳态

亚稳态，是因为违反寄存器的建立时间和保持时间产生的。

逻辑电路中绝大多数的时序问题基本都是因为这个原因产生的。由于寄存器在任何信号通路中都有特定的建立和保持时间，这要求输入信号要保持稳定，在触发器建立期间，如果数据在时钟上升沿的建立保持时间内{latch edge-setup, latch edge+hold time}发生跳变，则会产生亚稳态输出，即输出值在短时间内处于不确定态，有可能是1，有可能是0，也可能什么都不是，处于中间态，这个未知的状态便称为**亚稳态**。



常见的情况：

1、输入信号是异步信号。

- 2、时钟偏移/抖动高于容限值（时钟信号质量不好）。
- 3、跨时钟域信号传输。
- 4、复位电路设计。

降低亚稳态的概率：

其实就是避免余量出现负值。

建立时间余量 (Setup Slack) = $T_{\text{cycle}} + T_{\text{clk2}} - T_{\text{clk1}} - T_{\text{su}} - T_{\text{co}} - T_{\text{data}}$ 。

保持时间余量 (Hold Slack) = $T_{\text{clk1}} + T_{\text{co}} + T_{\text{data}} - T_{\text{clk2}} - T_{\text{h}}$ 。

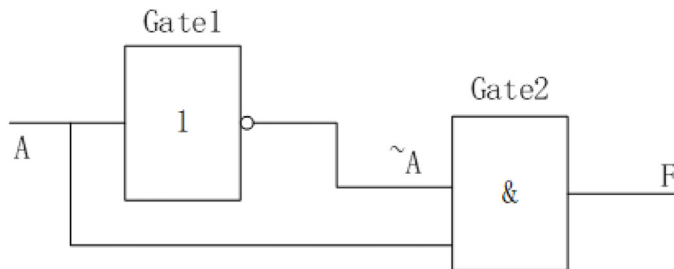
- 1、跨时钟域信号传输可以使用FIFO进行数据缓冲
- 2、复位电路采用异步复位、同步释放处理。
- 3、用反应更快的FF。
- 4、改善时钟质量，用边沿变化快速的时钟信号。
- 5、降低系统时钟频率。

十、竞争与冒险

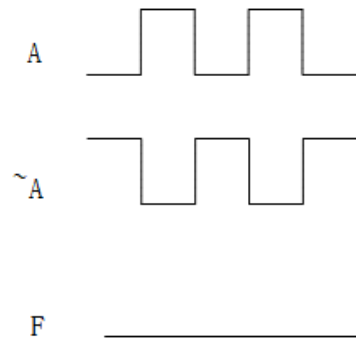
1.竞争与冒险产生原因

观察以下门电路，Gate1为 非门，Gate2为 与门，实现了逻辑

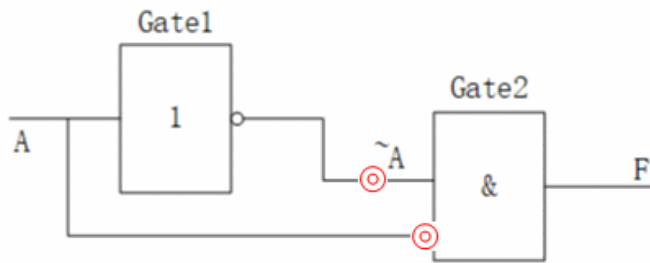
$$F = A \& \bar{A}$$



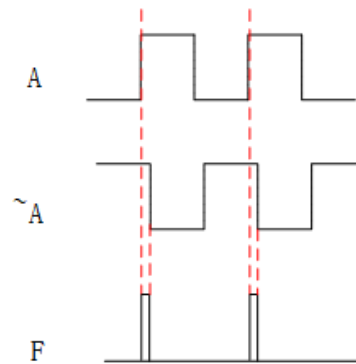
理想情况下输入与输出的关系为：



但是由于门电路的输入到输出是一定会有时间延迟的，这个时间通常叫做电路的开关延迟，就会导致出现以下毛刺。（Gate1输入到输出有一定的延迟，导致 $\sim A$ 信号就会滞后于A一段时间）



$\sim A$ 信号滞后于A一段时间就会导致尖峰脉冲的出现：



以上现象就产生了竞争-冒险现象。

竞争： 信号由于不同路径传输到达某一汇合点的时间有先有后的现象。（当一个门的输入有两个或两个以上的变量发生改变时，由于这些变量是经过不同路径产生的，使得他们的状态改变的時刻有先有后，这种时差引起的现象称为竞争。）

冒险： 由于竞争现象所引起的电路输出发生瞬间错误（产生干扰脉冲）的现象。

2.判断电路是否存在竞争-冒险现象

代数法： 只要输出函数中出现互补信号，就存在竞争-冒险的可能。

产生负向尖峰脉冲：

$$F = A + \overline{A}$$



产生正向尖峰脉冲：

$$F = A\overline{A}$$



卡诺图法： 观察卡诺图中是否有两个圈相切但不相交的情况，如有则存在竞争-冒险的现象。

以下卡诺图代表组合逻辑电路的逻辑表达式为

$$F = AB + \overline{A}C$$

BC \ A	00	01	11	10
0	0	1	1	0
1	0	0	1	1

如果B=C=1时，表达式就会变为

$$F = A + \overline{A}$$

就会产生竞争-冒险现象。

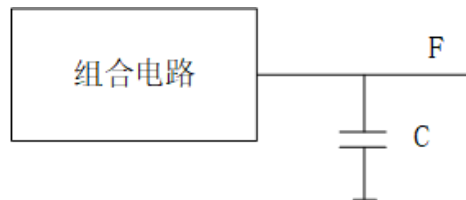
实际中：实际上逻辑电路级数多、输入变量多，判断竞争冒险非常复杂，广泛采用仿真和实际测量逻辑电路来观察是否存在竞争冒险的现象。

3. 消除竞争与冒险

当组合电路的某一部分存在竞争冒险现象时，可能危及电路的正常工作。因此必须消除竞争-冒险，以提高电路的可靠性。消除竞争-冒险的方法有以下几种：

1、接入滤波电容

在逻辑电路较慢的速度下工作，为了消除竞争冒险，可以在输出端并联一个电容器，其容量为4-20pF，这时在输出端不会出现逻辑错误，该方法只能用于对波形和延迟时间要求不严格的情况。

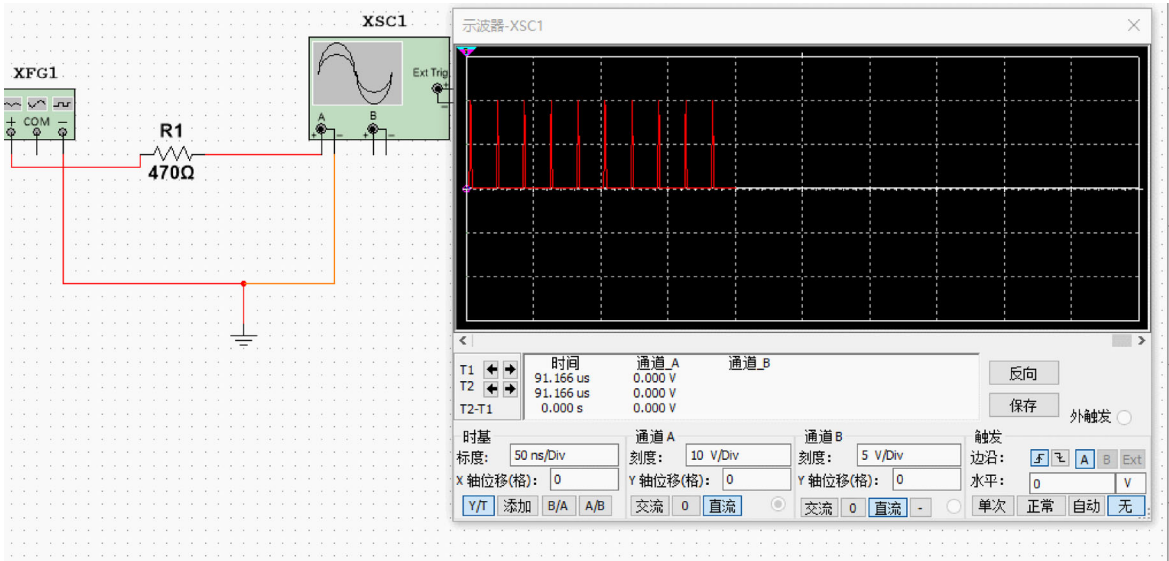


为什么在输出端接入滤波电容就会消除冒险现象呢？

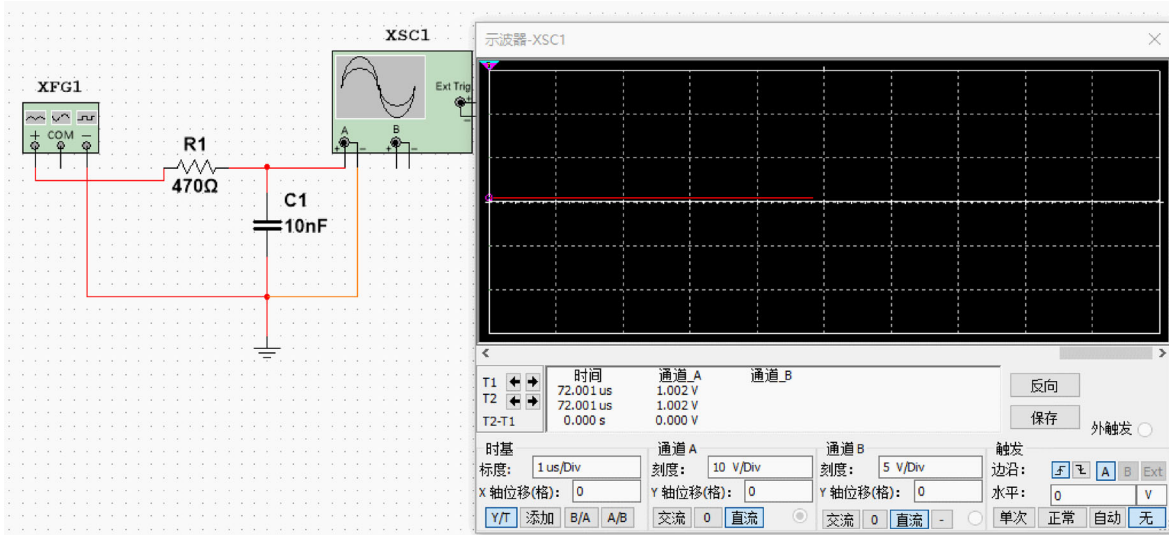
简单可以理解为电容通高频阻低频，尖脉冲直接对地短路，消除了尖脉冲，稳定了输出。

通过电路来理解一下，通过信号源模拟产生尖脉冲信号：

版权：Crazy_M

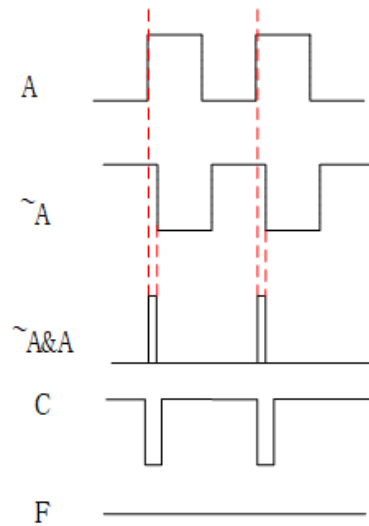
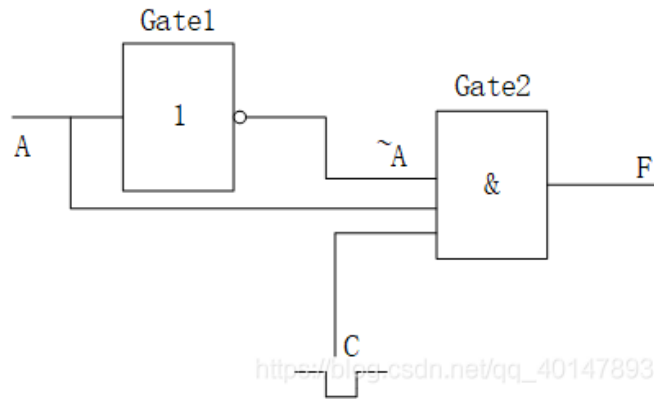


通过滤波电容后的信号：（因为尖脉冲是模拟的，不是门电路工作时候产生的尖脉冲，所以选用了10nF电容，电容要根据实际情况来选择）



2、引入选通信号

在输入信号变化，并有可能出现竞争冒险，选用一个选通信号将输出门封锁，等到所有输入信号都变为稳态后，再去掉封锁输出信号。这样就避免了电路输出端出现瞬时尖峰脉冲。



3、更改逻辑设计（利用卡诺图增加冗余项）

适当的逻辑表达式中增加一些冗余项，它们的增加不改变逻辑功能，但是可以消除互补信号产生的竞争-冒险现象。

例如

$$F = AB + \overline{A}C$$

B=C=1时

$$F = A + \overline{A}$$

有竞争-冒险现象，但是增加冗余项BC

$$F = AB + \overline{A}C + BC$$

就可以消除竞争冒险，因为当B=C=1时无论A如何变化，输出都是1。

冗余项是怎么来的呢？

上面提到卡诺图相切就会带来竞争-冒险的现象，我们人为的去破坏卡诺图相切现象，还不改变逻辑电路的功能。

原表达式：

$$F = AB + \overline{A}C$$

BC \ A	00	01	11	10
0	0	1	1	0
1	0	0	1	1

增加冗余项：

$$F = AB + \overline{A}C + BC$$

A \ BC	00	01	11	10
0	0	1	1	0
1	0	0	1	1

从卡诺图中可以看出，我们实现的逻辑功能是没有变的。

稍微复杂一点举例：

原表达式：

$$F = \overline{A}\overline{B}\overline{C} + \overline{A}BD + A\overline{D}$$

AB \ CD	00	01	11	10
00	1	1	0	0
01	0	1	1	0
11	1	0	0	1
10	1	0	0	1

增加冗余项：

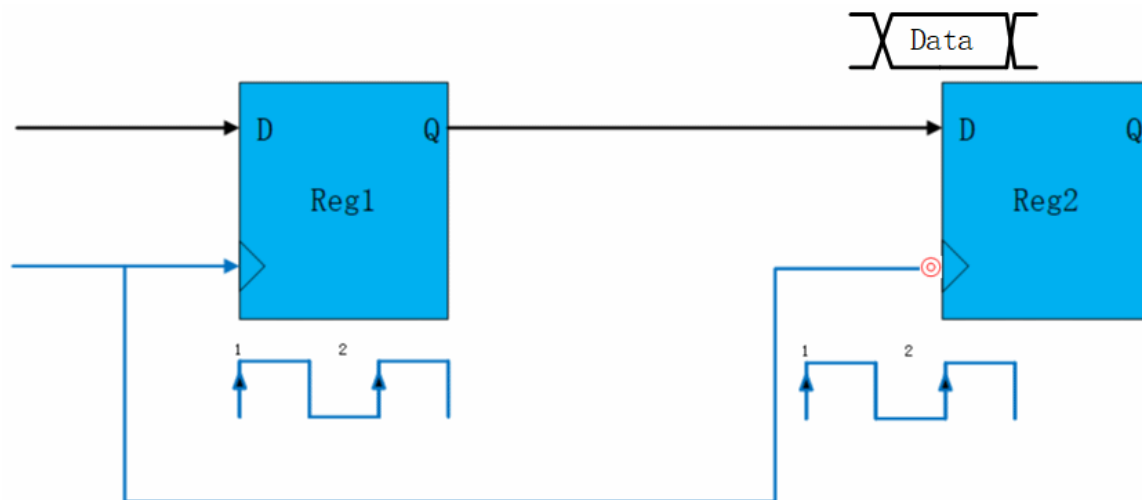
$$F = \overline{A}\overline{B}\overline{C} + \overline{A}BD + A\overline{D} + \overline{B}\overline{C}\overline{D} + \overline{A}CD$$

CD \ AB	00	01	11	10
00	1	1	0	0
01	0	1	1	0
11	1	0	0	1
10	1	0	0	1

4、信号延时法

因为毛刺最终是由于延迟造成的，所以可以找出产生延迟的支路。对于延迟相对较小的支路加上毛刺宽度的延迟可以消除毛刺。在分离电路中可以加一些非门或者其他门。在FPGA中，开发软件在综合设计时会把这些门当做冗余逻辑去掉，达不到延时的效果。可以用高频时钟来驱动移位寄存器，待延时信号作为数据输入，按所需延时正确设置移位寄存器的级数，移位寄存器的输出即为延时后的信号。

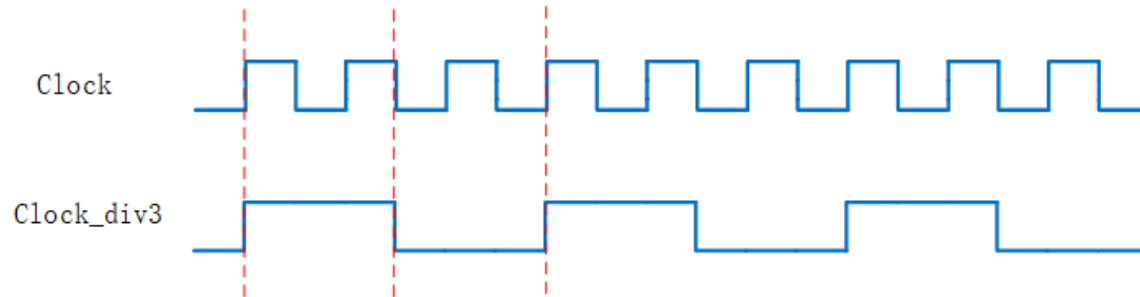
数据以接入的时钟周期通过通过寄存器，我们通过设置移位寄存器的级数就可以控制延时。



十一、奇数分频

1.奇数分频介绍

奇数分频 ($2N+1$)，以三分频 ($N=1$) 为例，三分频即分频后的时钟半周期（周期）是原时钟半周期（周期）的3倍宽度。



偶数分频我们只需要借助原时钟的上升沿或者下降沿进行计数分频，但是要实现奇数分频我们就要同时利用原时钟的上升沿和下降沿。可以通过原时钟的上升沿和下降沿计数产生clk1和clk2，再通过clk1和clk2的**关系**实现奇数分频。这种关系可以是或、与、异或，详细如下所述。

2.通过时钟 相或 实现奇数分频

原时钟clk通过上升沿和下降沿计数产生时钟clk1和clk2，clk1和clk2 相或 产生clk三分频的clk_div3时钟。

clk1：原时钟clk的上升沿产生。

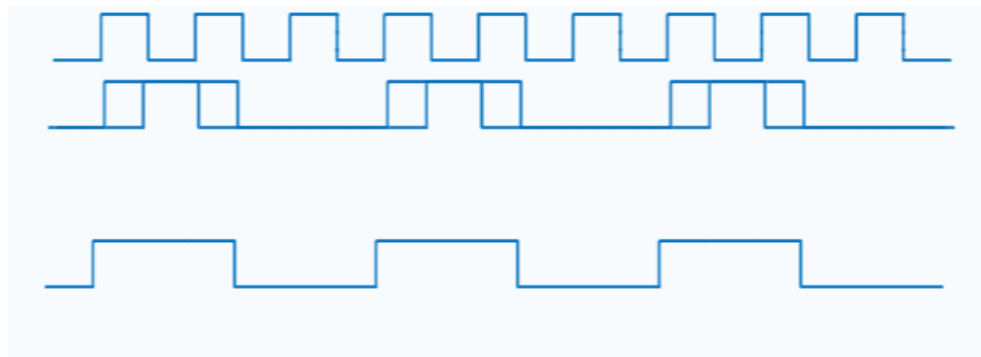
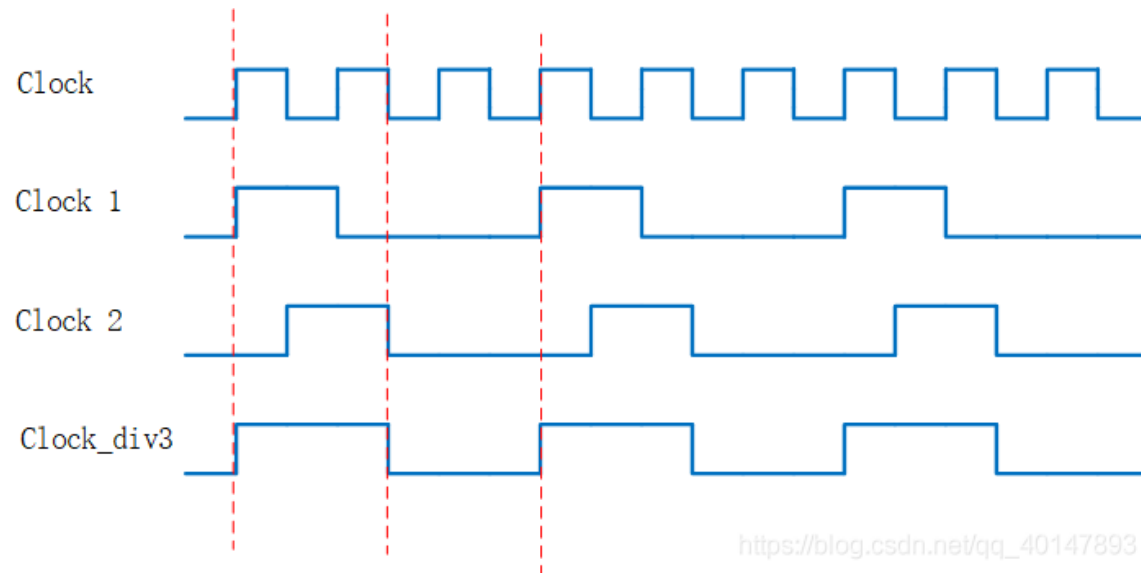
clk2：原时钟clk的下降沿产生。

clk1和clk2特征：

3分频：高电平:低电平 = 1:2。

2N+1分频：高电平:低电平 = N:N+1。

版权：Crazy_M



代码：

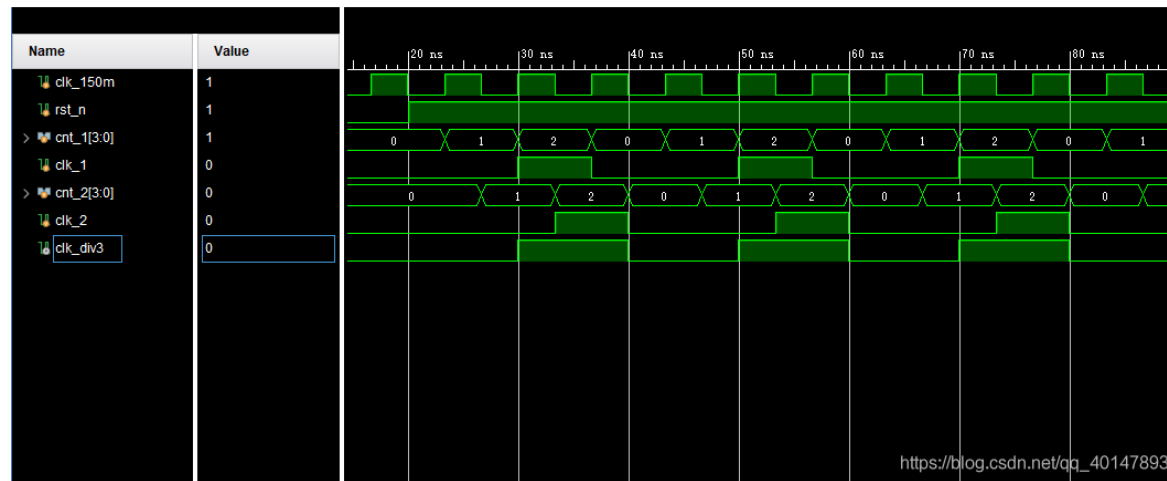
```
1 //时钟 相或
2 module top
3 (
4     input    clk_150m,
5     input    rst_n,
6
```

```

7      output      clk_div3
8
9  );
10
11  reg [3:0]  cnt_1;//上升沿计数
12  reg [3:0]  cnt_2;//下降沿计数
13  reg        clk_1;
14  reg        clk_2;
15
16  assign      clk_div3 = clk_1 | clk_2;
17
18  always@(posedge clk_150m or negedge rst_n)
19  begin
20      if(!rst_n)
21      begin
22          cnt_1      <= 4'd0;
23          clk_1      <= 1'b0;
24      end
25      else
26      begin
27          cnt_1      <= (cnt_1 == 4'd2)?4'd0:cnt_1+4'd1;
28          clk_1      <= (cnt_1 == 4'd1)?1'b1:1'b0;
29      end
30  end
31
32  always@(negedge clk_150m or negedge rst_n)
33  begin
34      if(!rst_n)
35      begin
36          cnt_2      <= 4'd0;
37          clk_2      <= 1'b0;
38      end
39      else
40      begin
41          cnt_2      <= (cnt_2 == 4'd2)?4'd0:cnt_2+4'd1;
42          clk_2      <= (cnt_2 == 4'd1)?1'b1:1'b0;
43      end
44  end
45

```

仿真波形：



3.通过时钟 相与 实现奇数分频

原时钟clk通过上升沿和下降沿计数产生时钟clk1和clk2，clk1和clk2 相与 产生clk三分频的clk_div3时钟。

clk1：原时钟clk的上升沿产生。

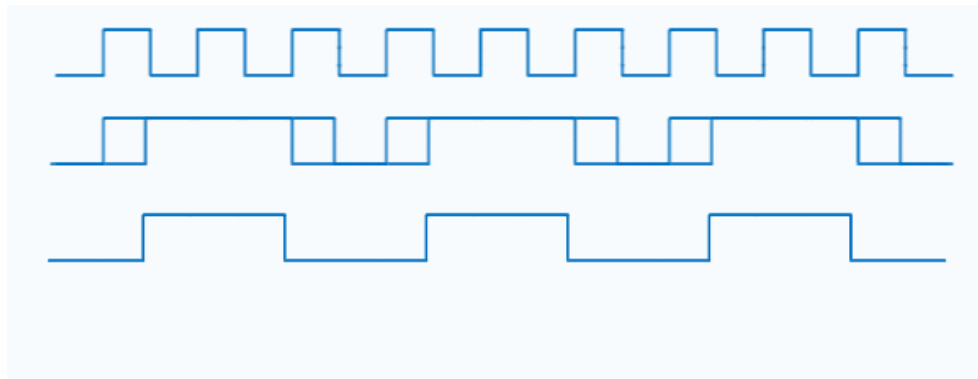
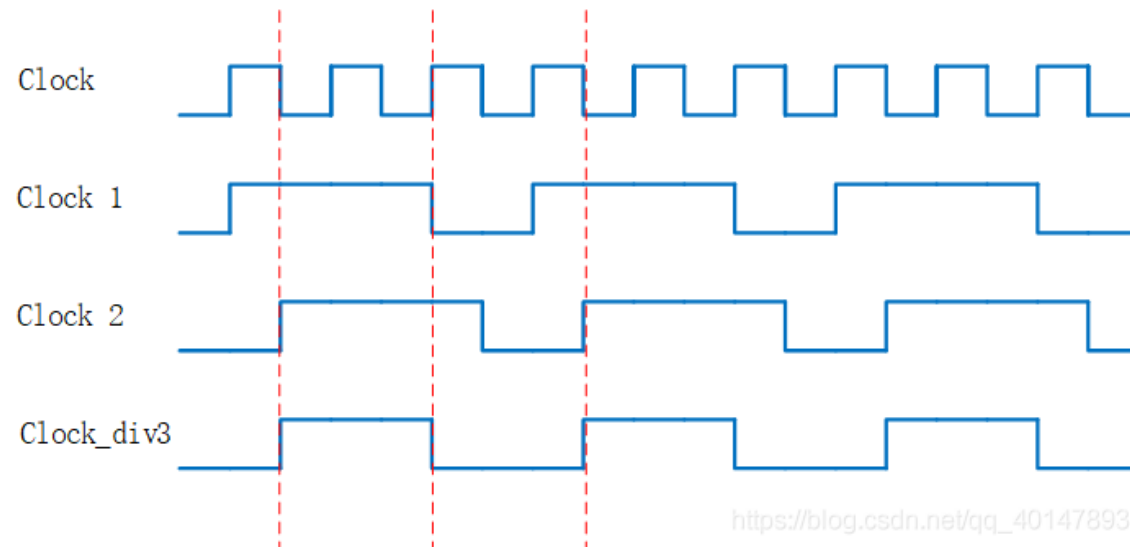
clk2：原时钟clk的下降沿产生。

clk1和clk2特征：

3分频：高电平:低电平 = 1:2。

2N+1分频：高电平:低电平 = N+1:N。

版权：Crazy_M



代码:

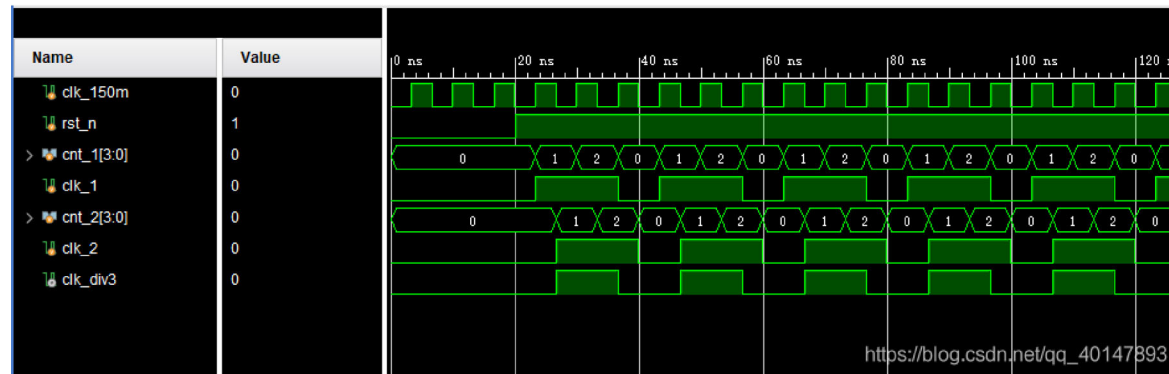
```
1 //时钟 相与
2 module top
3 (
4     input    clk_150m,
5     input    rst_n,
6
```

```

7      output      clk_div3
8
9  );
10
11  reg [3:0]  cnt_1;//上升沿计数
12  reg [3:0]  cnt_2;//下降沿计数
13  reg        clk_1;
14  reg        clk_2;
15
16  assign      clk_div3 = clk_1 & clk_2;
17
18  always@(posedge clk_150m or negedge rst_n)
19  begin
20      if(!rst_n)
21      begin
22          cnt_1      <= 4'd0;
23          clk_1      <= 1'b0;
24      end
25      else
26      begin
27          cnt_1      <= (cnt_1 == 4'd2)?4'd0:cnt_1+4'd1;
28          clk_1      <= (cnt_1 <= 4'd1)?1'b1:1'b0;
29      end
30  end
31
32  always@(negedge clk_150m or negedge rst_n)
33  begin
34      if(!rst_n)
35      begin
36          cnt_2      <= 4'd0;
37          clk_2      <= 1'b0;
38      end
39      else
40      begin
41          cnt_2      <= (cnt_2 == 4'd2)?4'd0:cnt_2+4'd1;
42          clk_2      <= (cnt_2 <= 4'd1)?1'b1:1'b0;
43      end
44  end
45

```

仿真波形：



4.通过时钟 相异或 实现奇数分频

原时钟clk通过上升沿和下降沿计数产生时钟clk1和clk2，clk1和clk2 相异或 产生clk三分频的clk_div3时钟。

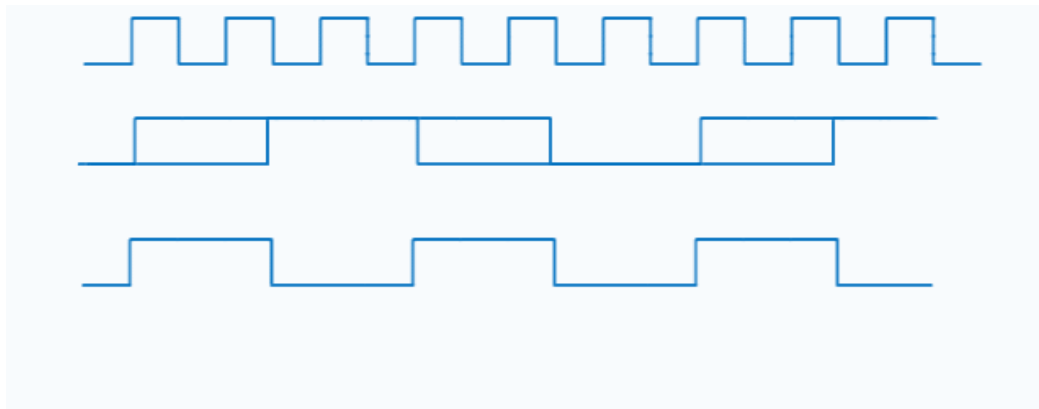
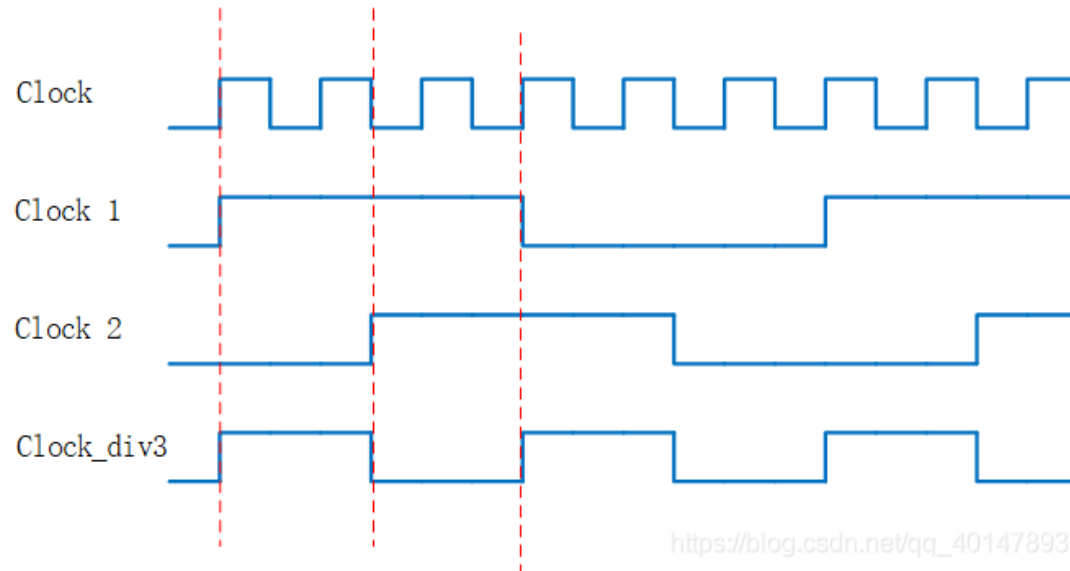
clk1：原时钟clk的上升沿产生。

clk2：原时钟clk的下降沿产生。

clk1和clk2特征：

3分频：clk1以原时钟clk的3个周期为单位翻转，clk2在clk1时钟的中间点发生翻转。

2N+1分频: clk1以原时钟clk的2N+1个周期为单位翻转, clk2在clk1时钟的中间点发生翻转。



代码:

```
1 //时钟 相异或
2 module top
3 (
4     input    clk_150m,
```

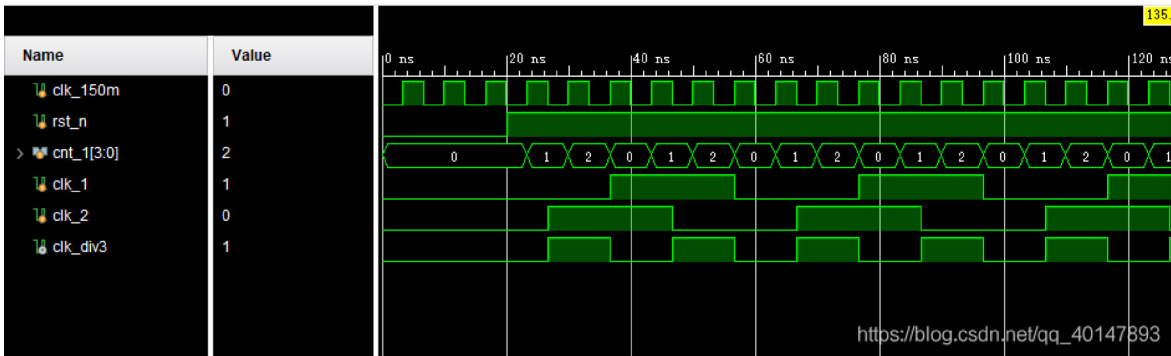
```

5      input      rst_n,
6
7      output     clk_div3
8
9  );
10
11 reg [3:0]  cnt_1;//上升沿计数
12 reg       clk_1;
13 reg       clk_2;
14
15 assign     clk_div3 = clk_1 ^ clk_2;
16
17 always@(posedge clk_150m or negedge rst_n)
18 begin
19     if(!rst_n)
20     begin
21         cnt_1      <= 4'd0;
22         clk_1      <= 1'b0;
23     end
24     else
25     begin
26         cnt_1      <= (cnt_1 == 4'd2)?4'd0:cnt_1+4'd1;
27         clk_1      <= (cnt_1 == 4'd2)?~clk_1:clk_1;
28     end
29 end
30
31 always@(negedge clk_150m or negedge rst_n)
32 begin
33     if(!rst_n)
34     begin
35         clk_2      <= 1'b0;
36     end
37     else
38     begin
39         clk_2      <= (cnt_1 == 4'd1)?~clk_2:clk_2;
40     end
41 end
42
43 endmodule

```

仿真波形：

版权：Crazy_M



测试代码：

```
1 module test;
2
3   reg    clk_150m;
4   reg    rst_n;
5
6
7   wire    clk_div3;
8
9   top u1
10  (
11    .clk_150m(clk_150m),
12    .rst_n(rst_n),
13    .clk_div3(clk_div3)
14
15
16  );
17
18
19  initial
20  begin
21    clk_150m = 0;
22    rst_n    = 0;
23
```

```

24     #20;
25
26     rst_n    = 1;
27 end
28
29 always # 3.3333 clk_150m = ~clk_150m;
30
31 endmodule
32

```

十二、时钟抖动 (Clock Jitter) 和时钟偏斜 (Clock Skew)

1. 时钟抖动 (Clock Jitter)

时钟抖动指的在某一个给定的点上时钟周期发生短暂性变化，使得时钟周期在不同的周期上可能加长或者缩短。（两个时钟周期存在差值，是时钟发生器内部产生的，和晶振或者PLL内部电路有关，布线对其没有影响）；它是频率上的不确定。

时钟边沿的变化不可能总是理想的瞬变，它总会有一个从高到低或者从低到高的变化过程；这就导致某一个给定的点上时钟周期发生短暂性变化，使得时钟周期在不同的周期上可能加长或者缩短，产生时钟抖动。

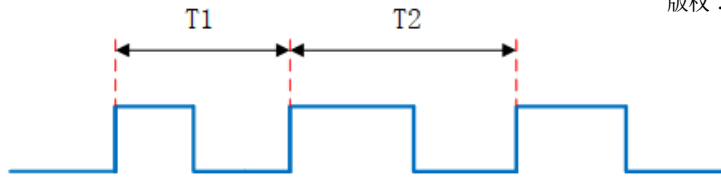
参考之前的建立时间与保持时间，严格的说建立时间应该是 $T_{su}+T_1$ ，保持时间应该是 T_h+T_1 。



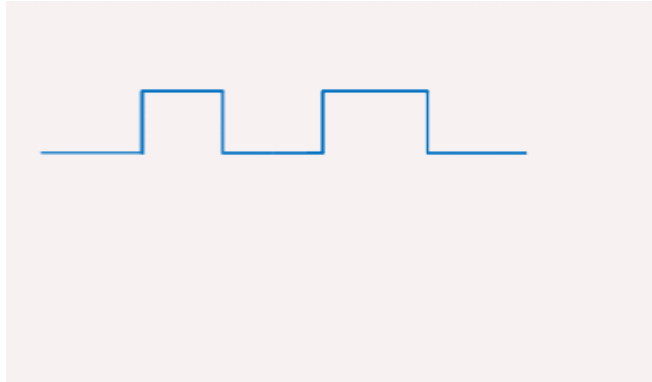
时钟频率具有随环境温度变化的特性。理想的方波是不存在的，在不影响系统性能的情况下，允许时钟周期在一个很小的时间范围内变化（这个变化的范围往往是微乎其微，有的甚至可以去考虑）。时钟抖动是源时钟的不确定性因素之一。

如下图所示：

时钟抖动 (Clock Jitter) = $T_2 - T_1$ 。

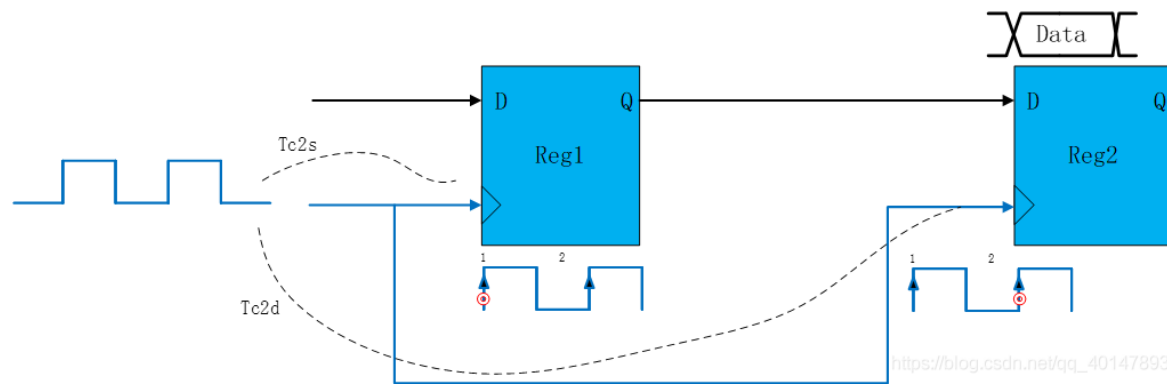


它是频率上的不确定：（不同的周期有所差异）



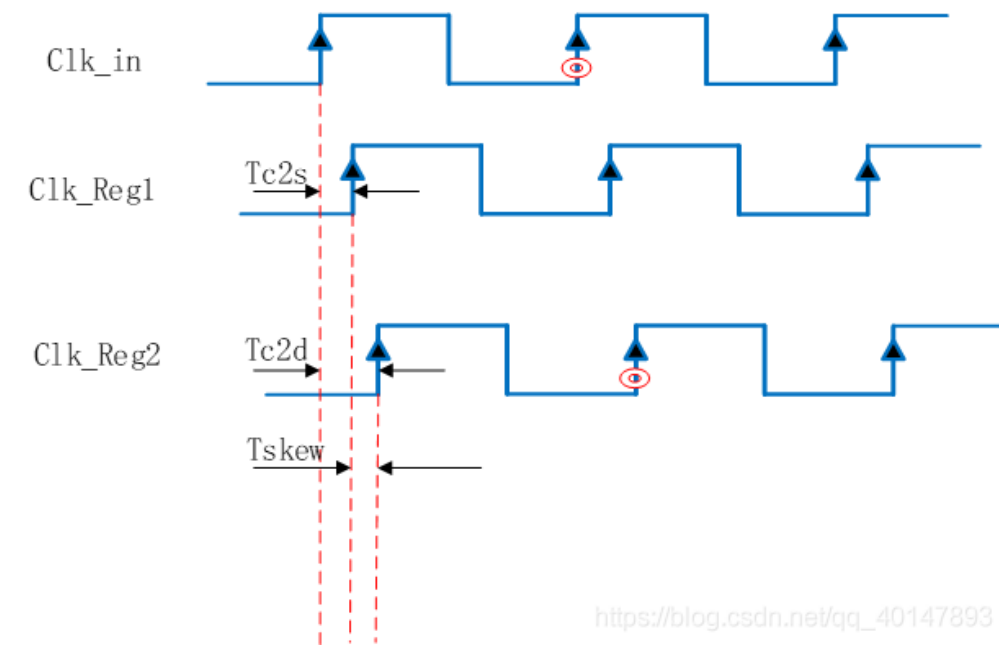
2.时钟偏斜 (Clock Skew)

时钟偏斜，由于布线长度以及负载不同引起的，导致同一个时钟信号到达相邻两个时序单元的时间不一致。（同样的时钟产生的多个子时钟之间的延时差异），（同一时钟信号到达两个不同寄存器之间的时间差值）（**相位上的不确定**）。



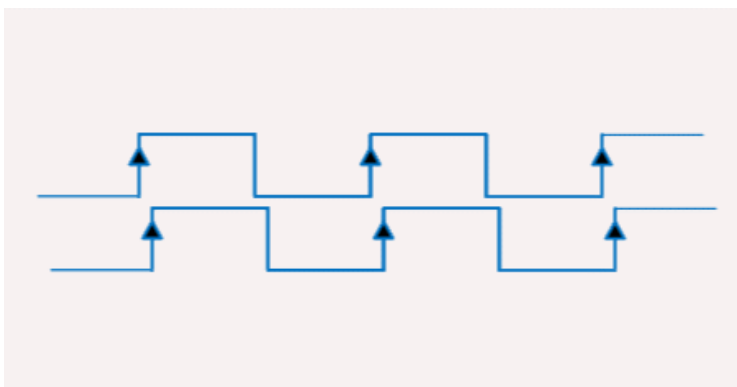
时序分析的起点是源寄存器（Reg1），终点是目的寄存器（reg2）。

如上图所示Clk_in为PLL或者源时钟产生的时钟。Clk_in经过Tc2s到达源寄存器Reg1、经过Tc2d到达目的寄存器Reg2。时序图如下所示：



时钟偏斜 (Clock Skew) = $T_{c2d} - T_{c2s}$ 。

它是相位上的不确定：（下图分别为Clk_in到Reg1和Reg2的时钟信号，同一个时钟信号到达相邻两个时序单元的时间不一致）



3.时钟抖动和时钟偏斜的区别

时钟抖动（Clock Jitter）是在时钟发生器内部产生的，和晶振或者PLL内部电路有关，布线对其没有影响；它是频率上的不确定。

时钟偏斜（Clock Skew）是由不同布线长度导致的不同路径的时钟上升沿到来的时延不同；它是相位上为不确定。

十三、Verilog的结构化、数据流、行为级描述方式

说明：

以表达式 $L=AB+BC+AC$ ，进行verilog代码编写以及RTL视图的观察来描述三种方式。

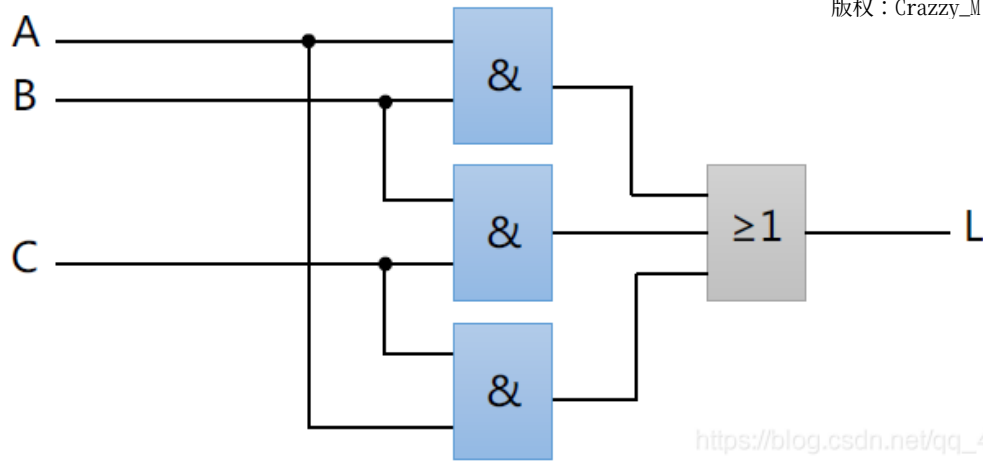
输入输出：

输入	输出
A、B、C	L

真值表：

A	B	C	L
0	0	0	0
0	0	1	0
0	1	0	0
0	1	1	1
1	0	0	0
1	0	1	1
1	1	0	1
1	1	1	1

逻辑图(下面三种方式都是为了实现这个逻辑)：



1.结构化描述方式及RTL视图

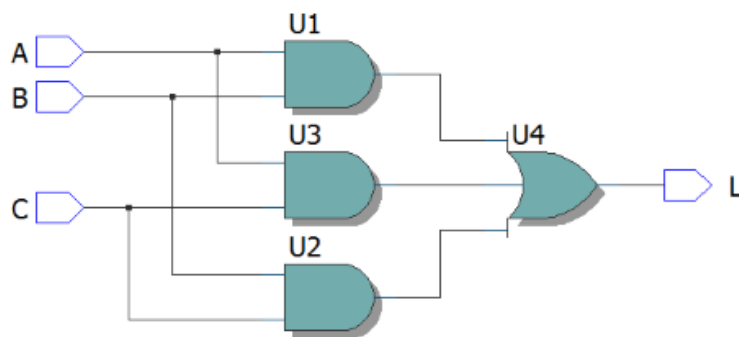
●**结构化描述方式：** 是使用实例化低层次模块的方法，即调用其他已经定义过的低层次模块对整个电路的功能进行描述，或者直接调用Verilog内部预先定义的基本门级元件描述电路的结构。

●**结构化描述代码：**

```

1  module Top
2  (
3      //输入端口
4      input A,
5      input B,
6      input C,
7      //输出端口
8      output L
9  );
10
11  wire AB,BC,AC;//内部信号声明
12
13  and U1(AB,A,B);//与门
14  and U2(BC,B,C);//与门
15  and U3(AC,A,C);//与门
16
17  or U4(L,AB,BC,AC);//或门
  
```

•结构化描述RTL视图：



https://blog.csdn.net/qq_40147893

2.数据流描述方式及RTL视图

•**数据流描述方式：** 是使用连续赋值语句(assign)对电路的逻辑功能进行描述，该方式特别便于对组合逻辑电路建模。

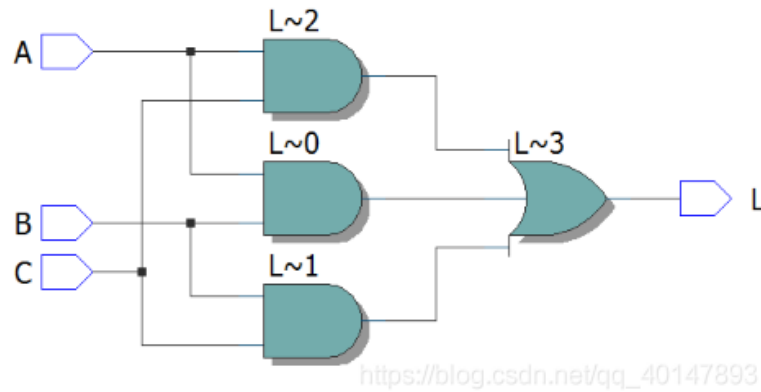
•**数据流描述代码：**

```

1 module Top
2 (
3     //输入端口
4     input A,
5     input B,
6     input C,
7     //输出端口
8     output L
9 );
10
11 assign L = (A&&B) || (B&&C) || (A&&C);
12
13 endmodule
  
```

•数据流描述RTL视图:

版权：Crazyy_M



3.行为级描述方式及RTL视图

•**行为级描述方式:** 是使用过程块语句结构(always)和比较抽象的高级程序语句对电路的逻辑功能进行描述。

•行为级描述代码:

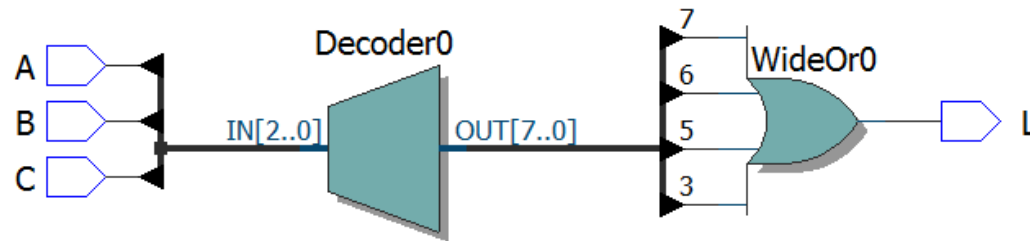
```
1 module Top
2 (
3     //输入
4     input A,
5     input B,
6     input C,
7     //输出
8     output reg L
9 );
10
11 always@(A,B,C)
12 begin
13     case({A,B,C})
14         3'b000: L = 1'b0;
15         3'b001: L = 1'b0;
16         3'b010: L = 1'b0;
17         3'b011: L = 1'b1;
```

```

18      3'b100: L = 1'b0;
19      3'b101: L = 1'b1;
20      3'b110: L = 1'b1;
21      3'b111: L = 1'b1;
22      default: L = 1'bx;
23  endcase
24 end
25
26 endmodule

```

●行为级描述RTL视图：



https://blog.csdn.net/qq_40147893

4.常用门级元件

verilog拓展知识：VerilogHDL内置26个基本元件，其中14个门级元件，12个开关元件。以下为我们常用的门级元件。

版权: Crazy_M

类别	关键字	符号示意图	门名称
多输入门	and		与门
	nand		与非门
	or		或门
	nor		或非门
	xor		异或门
	xnor		异或非门
多输出门	buf		缓冲器
	not		非门
三态门	bufif1		高电平使能三态缓冲器
	buiif0		低电平使能三态缓冲器
	notif1		高电平使能三态非门
	notif0		低电平使能三态非门

十四、函数 (function) 和任务 (task)

1.函数 function

function函数的目的是返回一个用于表达式的值。

verilog中的function只能用于组合逻辑。

1.定义函数的语法

```

1  function <返回值的类型或者范围> <函数名>
2  <端口说明语句>
3  <变量类型说明>
4  begin
5  <语句>
6  
```

① <返回值的类型或范围>这一项为可选项，如果缺失，则返回值为一位寄存器类型数据。

②函数的返回值：函数的定义蕴含声明了与函数同名、位宽一致的内部寄存器。

③函数的调用：函数的调用是通过将函数作为表达式中的操作数来实现的。其调用格式：

<函数名> (<表达式> ,..., <表达式>) ;

④函数使用的规则

1.函数定义不能包含有任何的时间控制语句。即任何用#、@、wait来标识的语句。

2 函数不能调用“task”。

3 定义函数时至少要有一个输入参数。

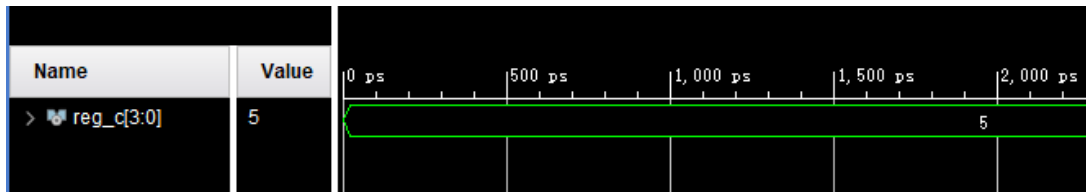
4 在函数的定义中必须有一条赋值语句给函数中与函数同名、位宽相同的内部寄存器赋值。

5 verilog中的function只能用于组合逻辑。

1. 具体实例

利用函数计算简单的加法。

```
1 module top_test;
2
3   wire    [3:0]reg_c;
4
5   assign  reg_c = add(4'd2,4'd3);
6
7   function [3:0]add;
8   input   [3:0]a;
9   input   [3:0]b;
10  begin
11      add = a + b;
12  end
13  endfunction
14
15  endmodule
16
```



2.任务 task

任务就是一段封装在“task-endtask”之间的程序。任务是通过调用来执行的，而且只有在调用时才执行，如果定义了任务，但是在整个过程中都没有调用它，那么这个任务是不会执行的。调用某个任务时可能需要它处理某些数据并返回操作结果，所以任务应当有接收数据的输入端和返回数据的输出端。另外，**任务可以彼此调用，而且任务内还可以调用函数。**

1、任务定义

任务定义形式：

```

1 task task_id;
2     [declaration]
3     procedural_statement
4 endtask

```

其中，关键词 task 和 endtask 将它们之间的内容标志成一个任务定义，task 标志着一个任务定义结构的开始；task_id 是任务名；可选项 declaration 是端口声明语句和变量声明语句，任务接收输入值和返回输出值就是通过此处声明的端口进行的；procedural_statement 是一段用来完成这个任务操作的过程语句，如果过程语句多于一条，应将其放在语句块内；endtask 为任务定义结构体结束标志。

定义任务要注意：

- (1) 在第一行“task”语句中不能列出端口名称；
- (2) 任务的输入、输出端口和双向端口数量不受限制，甚至可以没有输入、输出以及双向端口。
- (3) 任务定义的描述语句中，可以使用/出现不可综合操作语句（使用最频繁的就是延迟控制语句），但是这样会造成该任务不可综合。
- (4) 在任务中可以调用其他任务或函数、也可以调用自身。
- (5) 在任务定义结构内不能出现initial和always过程块。

(6) 在任务定义中可以出现“disable 中止语句”，将中断正在执行的任务，但真是不可综合的。当任务被中断后，程序流程将返回到调用任务的地方继续向下执行。

2. 具体实例

通过任务写一个简单的比较器。

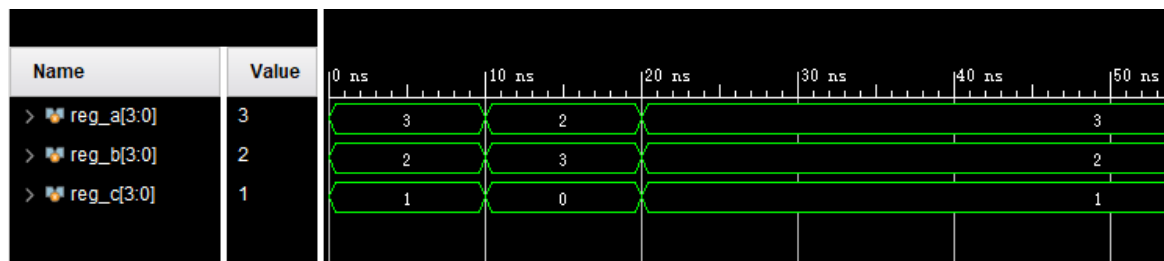
```

1  module top_test;
2  reg [3:0] reg_a;
3  reg [3:0] reg_b;
4  reg [3:0] reg_c;
5
6
7  initial
8  begin
9      reg_a = 4'd3;
10     reg_b = 4'd2;
11     task_temo(reg_a,reg_b,reg_c);
12     #10;
13
14     reg_a = 4'd2;
15     reg_b = 4'd3;
16     task_temo(reg_a,reg_b,reg_c);
17     #10;
18
19     reg_a = 4'd3;
20     reg_b = 4'd2;
21     task_temo(reg_a,reg_b,reg_c);
22
23 end
24
25 task task_temo;
26 input  [3:0]a;
27 input  [3:0]b;
28 output      y;
29
30 if(a>b)
31     y = 1'b1;
32 else
33     y = 1'b0;

```

```
34 | endtask
35 |
36 | endmodule
```

版权：Crazy_M



虽然任务中不能出现 initial 语句和 always 语句语句，但任务调用语句可以在 initial 语句和 always 语句中使用，其语法形式如下：

```
task_id[(端口 1, 端口 2, ..., 端口 N)];
```

其中 task_id是要调用的任务名，端口 1、端口 2, ...是参数列表。参数列表给出传入任务的数据（进入任务的输入端）和接收返回结果的变量（从任务的输出端接收返回结果）。**任务调用语句中，参数列表的顺序必须与任务定义中的端口声明顺序相同。**任务调用语句是过程性语句，所以任务调用中接收返回数据的变量必须是寄存器类型。

在调用任务时，要注意以下几点：

- (1) 任务调用语句只能出现在过程块内；
- (2) 任务调用语句和一条普通的行为描述语句的处理方法一致；
- (3) 当被调用输入、输出或双向端口时，**任务调用语句必须包含端口名列表，且信号端口顺序和类型必须和任务定义结构中的顺序和类型一致。**需要说明的是，任务的输出端口必须和寄存器类型的数据变量对应。
- (4) **可综合任务只能实现组合逻辑**，也就是说调用可综合任务的时间为“0”。而在面向仿真的任务中可以带有时序控制，如时延，因此面向仿真的任务的调用时间不为“0”。

十五、波特率

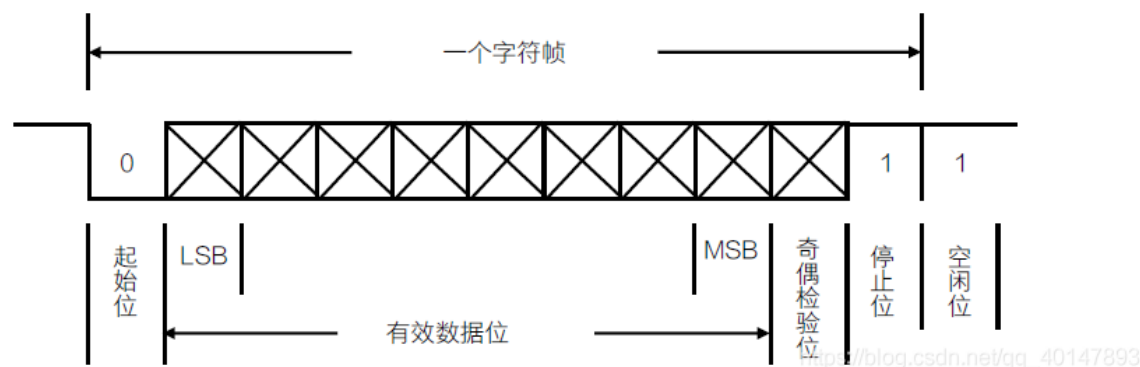
基础知识：

因	果
系统时钟(为串口提供时钟的时钟频率) - 50M	时钟周期 $\frac{1}{50 * 10^6}$
1个时钟周期可以计数1次	50M时钟1s计数50 000 000个
系统时钟 - 50M	计1个数需要 $\frac{1}{50 * 10^6}$ s
波特率-115200	1s传输二进制的位数115200bit
波特率-115200	传输1bit需要 $1/115200s=8.68us$
50M系统时钟-----波特率为115200	传输1bit需要计数 $(50 * 10^6 / 115200) = 434$

以上重点是推导出50M系统时钟—波特率为115200条件下传输1bit需要计数个数为434。

串口传输格式

- 我们通常用的串口传输格式为：1bit起始位+8bit数据位+1bit停止位(无奇偶校验位),如下图所示：

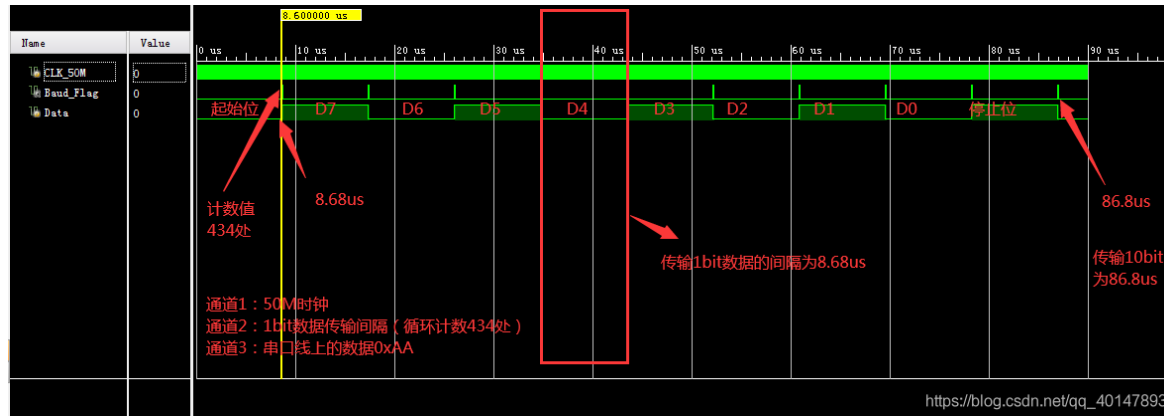


所以传输1Byte数据串口需要传输10bit数据。上面计算得传输1Bit需要的时间为8.68us，则传输1Byte需要时间为 $8.68 \times 10 = 86.8\mu s$ 。

波特率

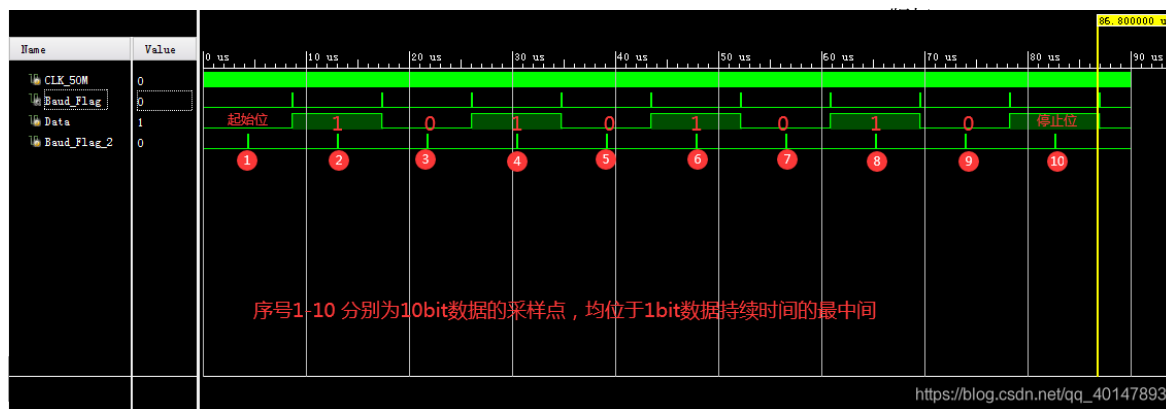
由基础知识知50M系统时钟—波特率为115200条件下传输1bit需要计数个数为434。那么1Byte(串口传输格式为：1bit起始位+8bit数据位+1bit停止位)是不是循环计数10次434就可以传输完毕。

直接上图：图中描述了1Byte数据传输的示意图，重点都在图里！！！。



什么时候去采样串口线上的数据呢？

★观察上图，Baud_Flag信号(通道2)表示了传输1Bit传输的间隔，每遇到1个Baud_Flag=1的信号，数据线上切换1次数据，所以两个Baud_Flag=1之间的数据是稳定的数据，根据抽样定理是不是应该在两个Baud_Flag=1信号的中间去采样数据呢，其实就是在1bit数据持续期间的中间点采样，才能得到最稳定的数据。见下图，重点都在图里！！！



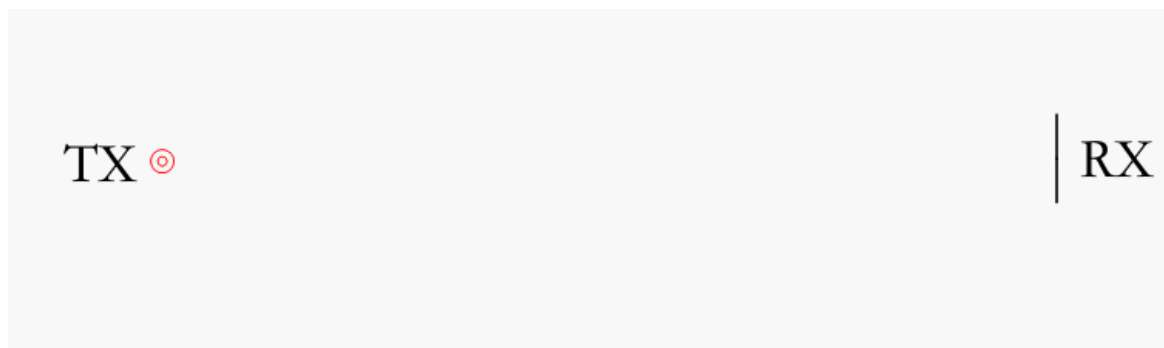
★图中序号①-⑩分别为10bit数据的采样点，采样点处提取数据为1010_1010(0xAA)。

★★★至此，从下往上再看一遍，就明白了我们所熟悉的波特率（115200/9600/...）怎么来的。

☞然后再想想为什么51单片机会会有一个定时器专门服务于串口。

☞再想想为什么9600波特率和115200波特率不能对接了，原理很简单我这边2秒发一次数据，你那边1秒接一次数据，怎么能成功接收呢。

收发波特率一致：



收发波特率不一致，导致RX端不能正常接收：

版权：Crazyy_M

TX

RX

十六、锁存器/触发器

latch是电平触发，register是边沿触发，register在同一时钟边沿发下动作，符合同步电路的设计思想，而latch则属于异步电路设计，往往会导致时序分析困难，不适当的应用latch则会大量浪费芯片资源。

一般信号经过两级触发器就可以消除毛刺。

为什么锁存器不好？

- (1) 锁存器对毛刺不敏感，很容易在信号上产生毛刺。
- (2) 没有时钟信号，不容易进行静态时序分析。

正是因为这两个原因，我们在FPGA设计时，尽量不用锁存器。

注意：“FPGA中只有LUT和FF的资源，没有现成的Latch，所以如果要用Latch，需要更多的资源搭出来”这个观点是不完全正确的。

在Xilinx的FPGA中，6系列之前的器件中都有Latch；6系列和7系列的FPGA中，一个Slice中有50%的storage element可以被配置为Latch或者Flip-Flop，另外一半只能被配置为Flip-Flop。比如7系列FPGA中，一个Slice中有8个Flip-Flop，如果被配置成了Latch，该Slice的另外4个Flip-Flop就不能用了。这样确实造成了资源的浪费。

但在UltraScale的FPGA中，所有的storage element都可以被配置成Flip-Flop和Latch。

十七、组合逻辑/时序逻辑

1 组合逻辑

组合逻辑的特点是任意时刻的输出仅仅取决于该时刻的输入，与电路原本的状态无关，逻辑中不涉及条边沿信号处理。

组合逻辑的verilog描述方式有两种：

(1) always@(电平敏感信号列表)

always模块的敏感列表为所有判断条件信号和输入信号，但一定要注意**敏感列表的完整性**。在always模块中可以使用if、case和for等各种RTL关键字结构。由于赋值语句有阻塞赋值和非阻塞赋值两类，建议读者使用阻塞赋值语句“=”。always模块中的信号必须定义为reg型，不过最终的实现结果中并没有寄存器。这是由于在组合逻辑电路描述中，将信号定义为reg型，只是为了满足语法要求。

(2) assign描述的赋值语句。

信号只能被定义为wire型

2 时序逻辑

时序逻辑的特点是任意时刻的输出不仅取决于该时刻的输入，而且还和电路原来的状态有关。电路里面有存储元件（各类触发器，在FPGA芯片结构中只有D触发器）用于记忆信息，从电路行为上讲，不管输入如何变化，仅当时钟的沿（上升沿或下降沿）到达时，才有可能使输出发生变化。

与组合逻辑不同的是：

(1) 在描述时序电路的always块中的reg型信号都会被综合为寄存器，这是和组合电路说不同的。

(2) 时序逻辑电路中推荐使用非阻塞赋值“<=”。

(3) 时序逻辑的敏感信号列表只需要加入所用的时钟触发沿即可，其余所有的输入和条件判断信号都不用加入，这是因为时序逻辑是通过时钟信号的跳变沿来控制的。

十八、阻塞/非阻塞赋值

1.阻塞赋值

阻塞赋值使用赋值运算符为“=”。阻塞赋值的过程是立刻执行的，即阻塞赋值运算符右侧表达式求值完后立刻会更新至运算符左侧，并且这个执行的过程不受其他语句执行的影响，其后的语句只有当前的赋值操作执行完成后才能顺序执行。

所谓阻塞的概念是指在同一个always块中，其后面的赋值语句从概念上是在前一条赋值语句结束后开始赋值的。

2.非阻塞赋值

非阻塞赋值使用的赋值运算符为“<=”。非阻塞赋值执行过程为：在当前仿真时间槽（time-slot）开始分析计算获得右侧表达式的值，在当前时间槽执行结束时更新左侧表达式的值，在右侧表达式分析计算和左侧表达式被更新之间，任何其他事件都可以执行，同时也有可能修改已经计算完成的右侧表达式的值，即非阻塞赋值的过程不影响其他语句的执行。

根据非阻塞赋值的特点,其赋值运算符左侧操作数只能为寄存器类型,因此非阻塞赋值只能用于过程性语句中（initial和always），不允许在连续赋值语句中使用非阻塞赋值。

verilog模块编程的8个原则：

- (1) 时序电路建模时，用非阻塞赋值。
- (2) 锁存器电路建模时，用非阻塞赋值。
- (3) 用always块建立组合逻辑模型时，用阻塞赋值。
- (4) 在同一个always块中建立时序和组合逻辑电路时，用非阻塞赋值。
- (5) 在同一个always块中不要既用非阻塞赋值又用阻塞赋值。
- (6) 不要在一个以上的always块中为同一个变量赋值。
- (7) 用\$strobe系统任务来显示用非阻塞赋值的变量值。
- (8) 在赋值时不要使用#0延时。

非阻塞赋值：

执行以下语句

```

1  module top_test;
2
3  reg      clk;
4  reg [3:0] in;
5  reg [3:0] q;
6  reg [3:0] q_r;
7  reg [3:0] q_rr;
8
9  initial
10 begin
11     clk = 0;
12
13     in  = 0; q = 1; q_r = 2; q_rr = 3;
```

```

14 end
15
16 always@(posedge clk)
17 begin
18     q      <= in;
19     q_r    <= q;
20     q_rr   <= q_r;
21 end
22
23 always #10 clk = ~clk;
24
25 endmodule

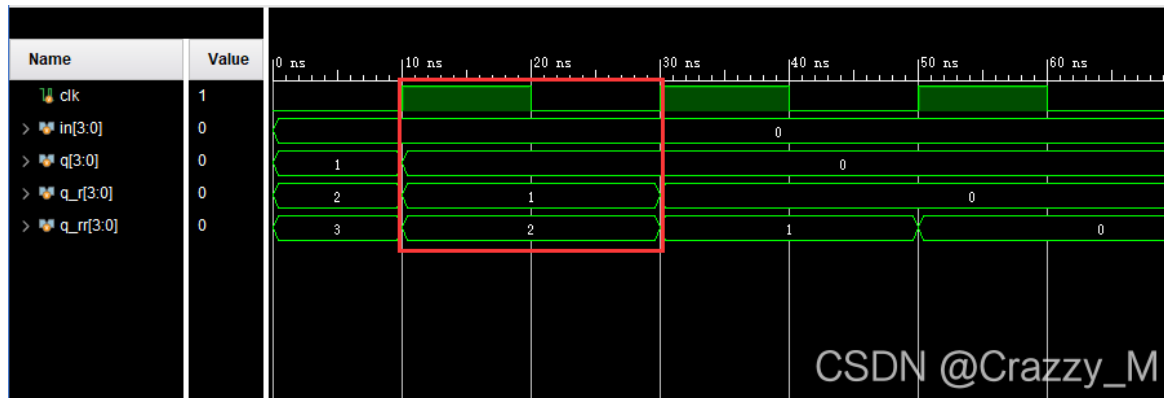
```

一个时钟周期后

q=0;

q_r=1;

q_rr=2;



**阻塞赋值： **执行以下语句

```

1 module top_test;
2
3 reg      clk;
4 reg [3:0] in;
5 reg [3:0] q;

```

```
6  reg [3:0]  q_r;
7  reg [3:0]  q_rr;
8
9  initial
10 begin
11     clk = 0;
12
13     in  = 0; q = 1; q_r = 2; q_rr = 3;
14 end
15
16 always@(posedge clk)
17 begin
18     q      = in;
19     q_r    = q;
20     q_rr   = q_r;
21 end
22
23
24 always #10 clk = ~clk;
25
26 endmodule
27
```

一个时钟周期之后（下面的语句一直在等待上面的语句赋值完成再执行）

q=0;

q_r=0;

q_rr=0;

版权：Crazyy_M



十九、跨时钟域处理

1 单bit信号

1) 电平检测

最为简单的方法。通过寄存器打两拍进行同步，也就是所谓的电平同步器。

存在问题为，输入信号必须保持两个接收时钟周期，每次同步完，输入信号要恢复到无效状态。所以，如果是从快到慢，信号很有可能被滤除。

****两级触发器可防止亚稳态传播的原理：****假设第一级触发器的输入不满足其建立保持时间，它在第一个脉冲沿到来后输出的数据就为亚稳态，那么在下一个脉冲沿到来之前，其输出的亚稳态数据在一段恢复时间后必须稳定下来，而且稳定的数据必须满足第二级触发器的建立时间，如果都满足了，在下一个脉冲沿到来时，第二级触发器将不会出现亚稳态，因为其输入端的数据满足其建立保持时间。

更确切地说，输入脉冲宽度必须大于同步时钟周期与第一级触发器所需的保持时间之和。

最保险的脉冲宽度是两倍同步时钟周期。所以，这样的同步电路对于从较慢的时钟域来的异步信号进入较快的时钟域比较有效，对于进入一个较慢的时钟域，则没有作用。

为什么是打两拍呢，打一拍、打三拍行不行呢？

两级并不能完全消除亚稳态危害，但是提高了可靠性减少其发生概率。总的来讲，就是一级概率很大，三级改善不大。如果再加上第三级寄存器，**由于第二级寄存器对于亚稳态的处理已经起到了很大的改善作用，第三级寄存器在很大程度上可以说只是对于第二级寄存器的延拍，所以意义是不大的。**

适用于慢时钟域向快时钟域。

2) 边沿检测:

在电平同步器的基础上，通过输出端的组合逻辑，可以完成对于信号边沿的提取，**识别上升沿，下降沿以及双边沿，并发出相应的脉冲。**

适用于慢时钟域向快时钟域。

3) 脉冲同步:

先将原时钟下的脉冲信号，转化为电平信号（使用异或门），在进行同步，同步完成之后再
把新时钟域下的电平信号转换为脉冲信号（边沿检测器的功能）。

这就从快时钟域的取出一个单时钟宽度脉冲，在慢时钟域建立新的单时钟宽度脉冲。

结合了之前所提到的两种同步器。

但存在一个问题，输入脉冲的时间间距必须在两个接收时钟周期以上，否则新的脉冲会变宽，这就不再是单时钟脉冲了。

适用于快时钟域向慢时钟域。**2 多bit信号**

1) 采用保持寄存器加握手信号的方法（多数据，控制，地址）。

2) 异步FIFO

握手:

握手指的是两个设备之间通信的一种方式，用来通信的信号就是握手信号。**最简单的握手信号是 valid 和 ready**，也可以叫 request 和 grant。假设设备1向设备2发送数据，设备1不知道设备2什么时候可以接收数据，设备2也不知道设备1什么时候会发送数据，那么它们之间如果用握手通信可能是这样的顺序:

设备1将 valid 信号置1，告诉设备2，数据准备就绪了，请查收

设备2此刻正处于忙碌状态无法接收数据，设备2将 ready 信号保持为0

设备2空闲了，将 ready 信号置1接收设备1的数据

设备1看到设备2的 ready 为1，它知道设备2已经接收好数据了，将 valid 置0同时撤销数据，准备下一次发送。

可以看到因为有握手控制，可以确保数据的正确传输，不会丢失。跨时钟域的握手设计就是利用握手控制这种优势，从而避免因为跨时钟域引起的数据传输错误。

二十、位宽计算（N bit数和M bit数相加、相乘后需要多少bit?）

有符号数（补码）： $-2^{(N-1)} \sim 2^{(N-1)}-1$

如N = 8

则范围为：-128~127（因为0算在正数范围中）

无符号数： $0 \sim 2^N-1$

如N = 8

则表示范围是：0~255。

有符号定点数：

如N = 8

3Q5：意思为共8位，3位整数位，5位小数位。

如果有符号，那么整数位被占去一位。

精度（最小的小数值）

N bit数和M bit数相加、相乘后需要多少bit?

相加相乘后需要的数据位宽，若无已知数据范围，按照基本规律：

相加位宽+1，相乘位宽为N+M（保证宽度足够）。

若已知操作数范围，根据运算操作数所能表示数的绝对值最大值，求出运算结果极限值。

① 例如，两个8bit有符号数**相乘**，其结果需要的位宽是多少？

8bit有符号数：-128~127

最大绝对值的数为-128，所以极限下

$$(-128) \times (-128) = 16384$$

$$= 2^7 \times 2^7 = 2^{7+7} = 2^{14}$$

15bit有有符号数能表示的范围是：

— (2^{14}) 到 $2^{14} - 1$

并不能表示

2^{14}

综上，需要**16bit位宽**，正好是8+8。

② **相加** $\text{sum}[7:0] + b[7:0] + c[7:0] + d[7:0] + e[8:0]$ 为了sum没有溢出，sum 的最小位宽为多少

$$= (2^8 - 1) + (2^8 - 1) + (2^8 - 1) + (2^8 - 1) + (2^9 - 1)$$

$$= 255 * 4 + 511 = 1531$$

$$= 101_1111_1011(\text{B})$$

所以需要**11bit**。

二十一、IIC

I2C 总线在物理连接上非常简单，分别由**SDA(串行数据线)**和**SCL(串行时钟线)**及上拉电阻组成。通信原理是通过对SCL和SDA线高低电平时序的控制，来产生I2C总线协议所需要的信号进行数据的传递。在总线空闲状态时，这两根线一般被上面所接的上拉电阻拉高，保持着高电平。

I2C通信方式为半双工，只有一根SDA线，同一时间只可以单向通信。

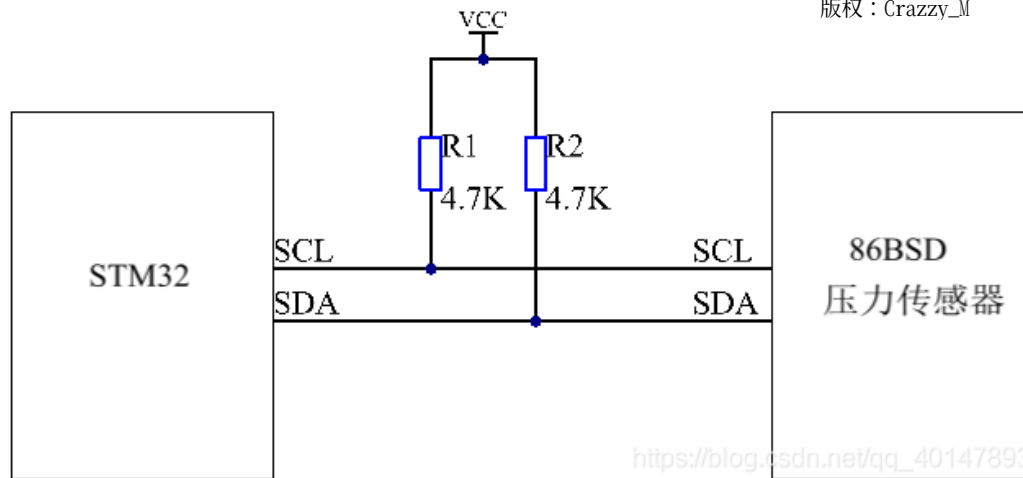
I2C总线上的每一个设备都可以作为主设备或者从设备

1. IIC接口介绍

SDA：串行数据线，总线空闲时为高电平

SCL：串行时钟线，总线空闲时为高电平

★注意：SDA、SCL接口电路为开漏输出，需要接上拉电阻,本实验采用了4.7k的上拉电阻，电路图如下。



2. IIC总线特征

- 1)、IIC总线是真正的多主机总线，每个连接到总线上的器件都有唯一的地址,任何器件既可以作为主机也可以作为从机，但同时时刻只允许有一个主机。
- 2)、连接到相同总线上的IC数量只受总线最大电容的限制，串行的8位双向数据传输位速率在标准模式下可达100Kbit/s，快速模式下可达400Kbit/s，高速模式下可达3.4Mbit/s。
- 3)、主设备和从设备之间以8 bit为单位进行双向传输。

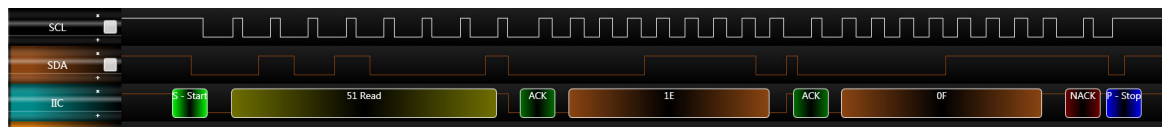
3. IIC总线协议详解

• IIC总线协议的构成

IIC总线协议构成主要构成如下图所示：

起始条件	从设备地址(7bit)	W/R (1bit)	ACK	数据(8bit)	ACK	数据(8bit)	ACK/NACK	停止条件
------	-------------	------------	-----	----------	-----	----------	----------	------

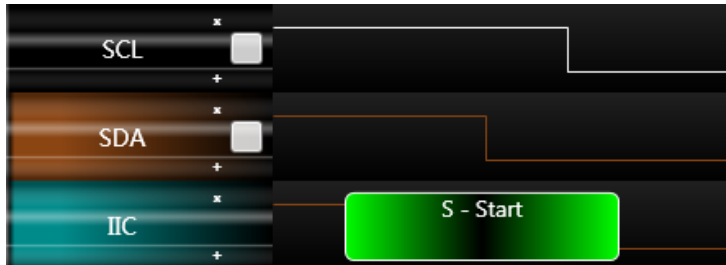
用逻辑分析仪捕捉的传输2Byte数据的IIC协议如下图所示：



下面依次分解。

4. 起始条件

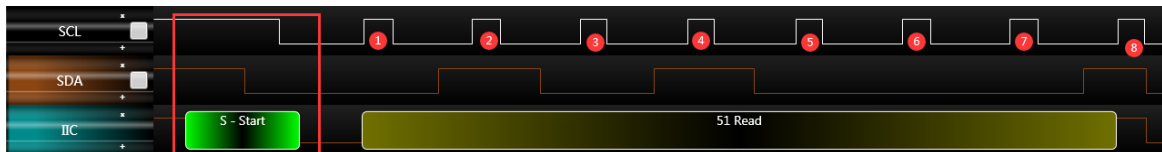
总线上必须以一个起始条件作为数据传输的起点，当SCL为高电平时，SDA由高电平变为低电平时为起始条件，如下图所示。



说明：起始信号和停止信号总是由主设备发出的。

5. 从设备地址+R/W位

之前介绍过每个连接到IIC总线上的器件都有唯一的地址，主设备要想与从设备通信就必须先校验从设备地址，7bit从设备地址+1bit读/写位共同构成了1Byte的数据，这1Byte数据在起始条件之后，并在SCL的8个周期内将其放置在了SDA数据线上，如下图所示。

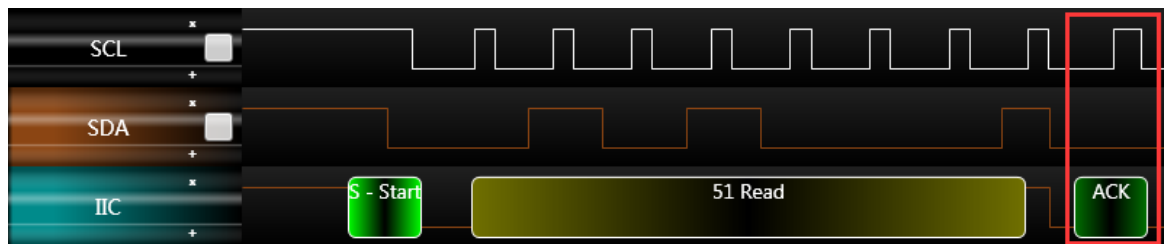


★★★注意：8bit地址数据的D0位读/写位，D0为1时为主设备向从设备读模式，D0为0时为主设备向从设备写模式，上图中可以看出从设备地址为0x51(0101_0001)，实际上该传感器设备的出厂地址为0x28(0010_1000)，由此看出D7-D1为设备的出厂地址D0为1，所以此时的模式是向传感器读取数据，同时观察逻辑分析仪第三通道同样识别为Read模式。

6. ACK应答

应答这个是为了后续通信能继续下去的一个机制，凡是接收数据或命令方，都要给一个应答位，发送方只有收到应答位后才继续发送。只有一个特殊情况不用应答位，就是主机（给时钟信号的一方）作为接收方时，在收到最后一个字节信息时不用应答，所以应答分为主机应答和从机应答。

从机应答：当一个字节的数据从高到低传输完毕后，从设备将SDA数据线拉低，这样主设备检测到SDA被拉低就相当于收到了一个来自从设备的应答，这样主设备才知道一个字节被真正传输完毕。如下图所示：

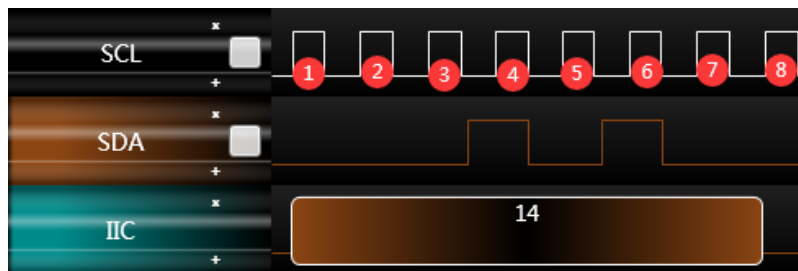


主机应答：主机向从机发送了读数据的指令后，从机必然要向主机返回数据，那么什么作为主机接收到从机数据的标志呢，也是一个应答位，看下图：



7. 数据传输

SDA的数据在SCL高电平期间被写入从机。所以SDA的数据变化要发生在SCL低电平期间。下图为1Byte数据传输的波形图：



8. 停止条件

总线上必须以一个停止条件作为数据传输的终点，**当SCL为高电平时，SDA由低电平变为高电平时为停止条件**，如下图所示。



说明：起始信号和停止信号总是由主设备发出的。

IIC时序的SCL和SDA之间具体的时间差要参考不同的芯片手册。

二十二、SPI

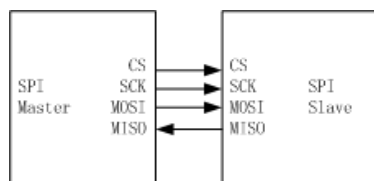
SPI，是一种高速的，全双工，同步的通信总线。

SCK： 时钟信号，由主设备产生，所以主设备SCK信号为输出模式，从设备的SCK信号为输入模式。

CS： 使能信号，由主设备控制从设备，，所以主设备CS信号为输出模式，从设备的CS信号为输入模式。

MOSI： 主设备数据输出，从设备数据输入，所以主设备MOSI信号为输出模式，从设备的MOSI信号为输入模式。

MISO： 主设备数据输入，从设备数据输出，所以主设备MISO信号为输入模式，从设备的MISO信号为输出模式。

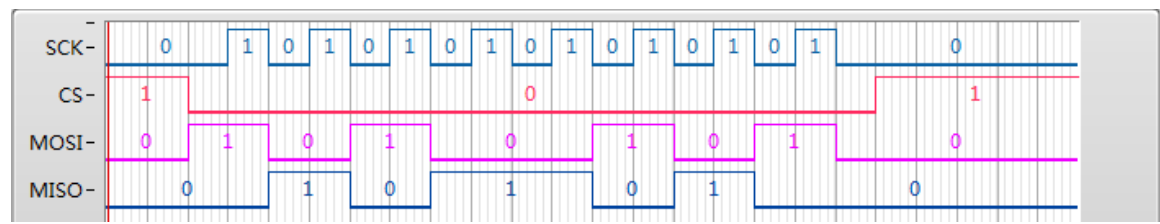


注意：MOSI和MISO不能交叉连接(可以把主从机理解为一个整体系统，MOSI为系统主机发送从机接收的数据线，MISO为主机接收从机发送的数据线)。

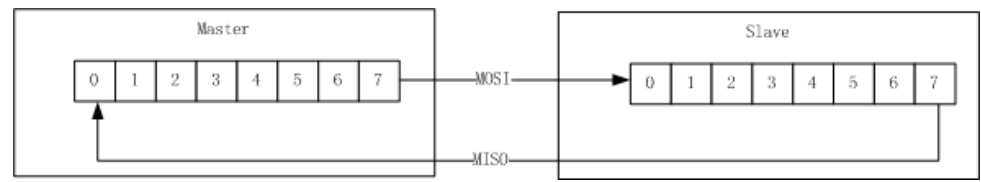
SPI数据传输方向

SPI作为全双工的串行通信协议，数据传输时高位在前，低位在后。主机和从机公用由主机产生的SCK信号，所以在**每个时钟周期内主机和从机有1bit的数据交换**(因为MOSI和MISO数据线上的数据都是在时钟的边沿处被采样)。

如下图：



SPI协议规定数据**采样是在SCK的上升沿或下降沿时刻**(由SPI模式决定，下面会说到)，观察上图，在SCK的边沿处(上升沿或下降沿)，主机会在MISO数据线上采样(接收来自从机的数据)，从机会在MOSI数据线上采样(接收来自主机的数据)，所以每个时钟周期中会有一bit的数据交换。



SPI总线传输一共有 4 种模式，这** 4 种模式分别由时钟极性(CPOL)和时钟相位(CPHA)**来定义。

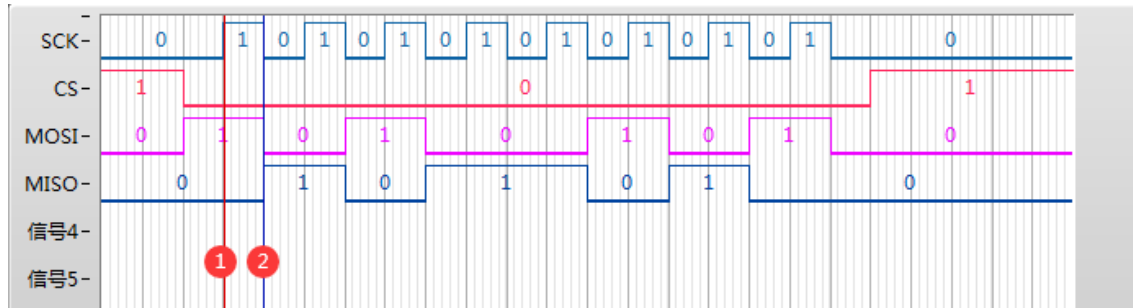
位1	CPOL: 时钟极性⁽¹⁾ 0: 空闲状态时，SCK保持低电平； 1: 空闲状态时，SCK保持高电平。
位0	CPHA: 时钟相位⁽¹⁾ 0: 数据采样从第一个时钟边沿开始； 1: 数据采样从第二个时钟边沿开始。

CPOL	CPHA
规定了SCK时钟信号空闲状态的电平	规定了数据是在SCK时钟的上升沿还是下降沿被采样
-----	-----

CPOL	CPHA
模式0: CPOL=0, CPHA =0	SCK空闲为低电平，数据在SCK的上升沿被采样(提取数据)
模式1: CPOL=0, CPHA =1	SCK空闲为低电平，数据在SCK的下降沿被采样(提取数据)
模式2: CPOL=1, CPHA =0	SCK空闲为高电平，数据在SCK的下降沿被采样(提取数据)
模式3: CPOL=1, CPHA =1	SCK空闲为高电平，数据在SCK的上升沿被采样(提取数据)

以模式0为例：

SCK空闲为低电平，数据在SCK的上升沿被采样(提取数据)，在SCK的下降沿切换数据线的的数据。



在时钟的第1个上升沿(游标1处)(采样点)

MOSI上数据为1，则在此边沿从机采样(提取)数据为1，采样点在MOSI数据线的中间。

MISO上数据为0，则在此边沿主机采样(提取)数据为0，采样点在MISO数据线的中间。

在时钟的第1个下降沿(游标2处)(切换点)

MOSI上数据由1切换为0，，数据在时钟下降沿时切换数据。

MISO上数据由0切换为1，，数据在时钟下降沿时切换数据。

在时钟的第2~8个上升沿(采样点)，主机在MISO上采样数据，从机在MOSI上采样数据。

在时钟的第2~8个下降沿(切换点)，主机在MISO上切换数据，从机在MOSI上切换数据。

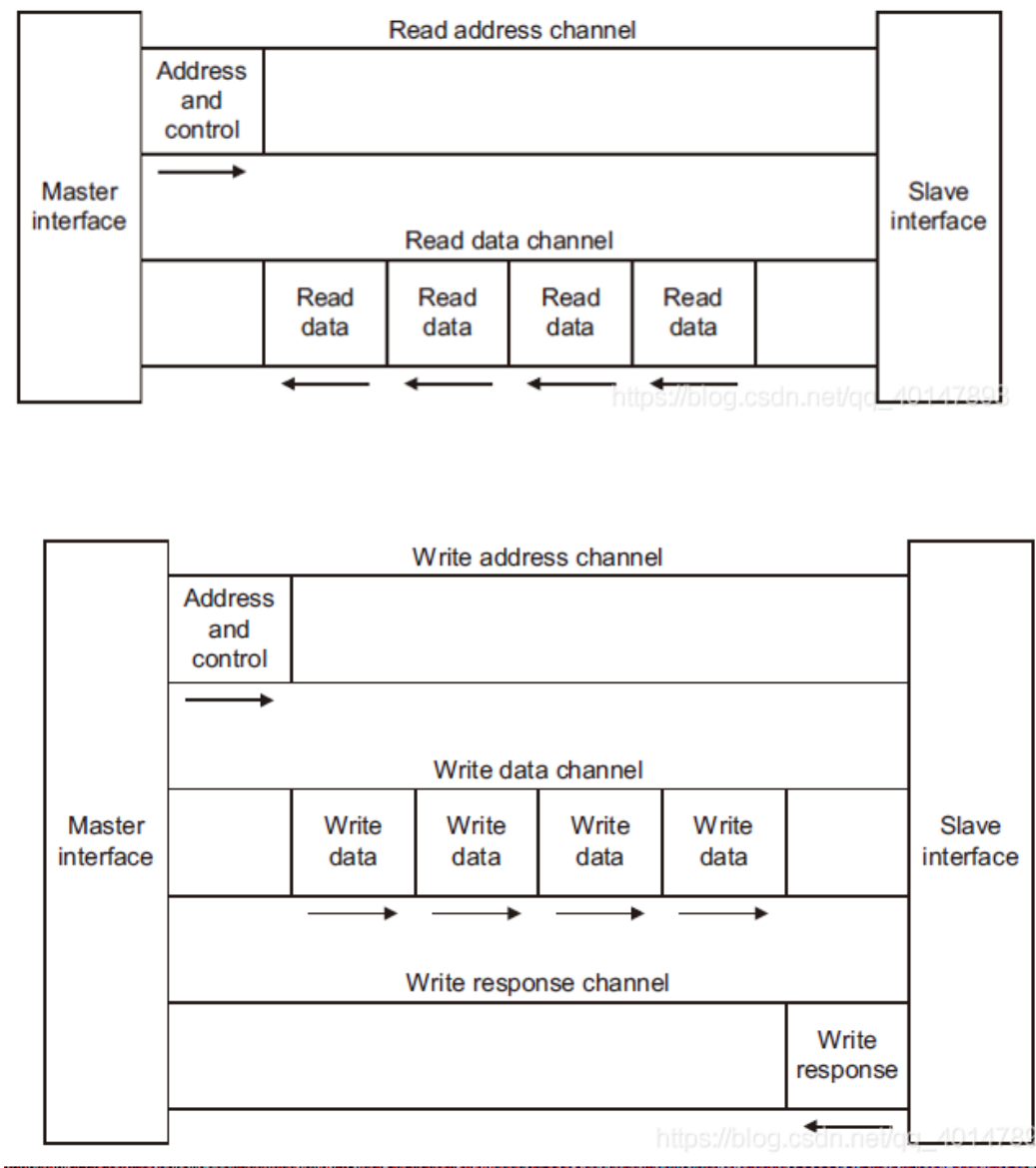
二十三、AXI

AXI (Advanced eXtensible Interface) 是一种总线协议，该协议是ARM公司提出的AMBA3.0中最重要的部分，是一种面向高性能、高带宽、低延迟的片内总线。AMBA4.0将其修改升级为AXI4.0。

AMBA4.0 包括AXI4.0、AXI4.0-lite、ACE4.0、AXI4.0-stream。

AXI协议是基于burst的传输，并且定义了以下5个独立的传输通道：
读地址通道、读数据通道、写地址通道、写数据通道、写响应通道。

版权：Crazyy_M



读地址通道：

每个读事务都有自己的地址通道。适当的地址通道携带通信所需的所有地址和控制信息。AXI协议支持以下机制：

可变的突发长度，每个突发长度从1-16个数据。

(连续传输的周期数就是突发长度。)

突发传送的大小为8-1024bit

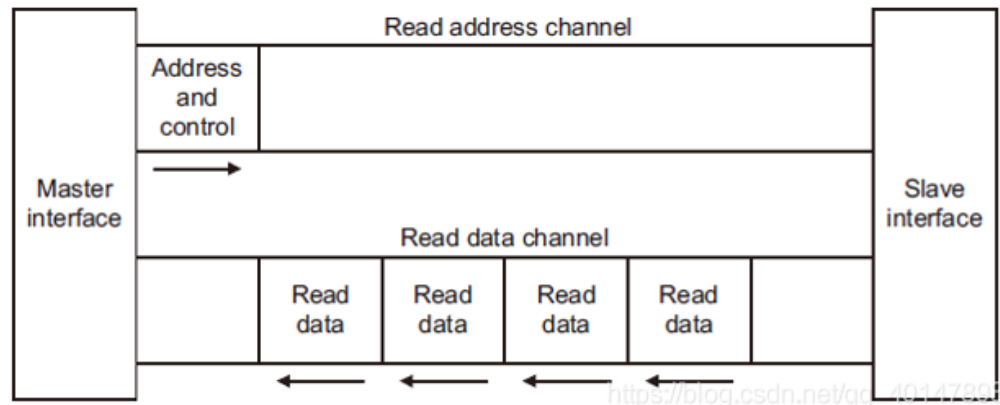
...

读取数据通道：

读取数据通道可同时传输读取数据和来自设备从设备到主设备的响应信息。

数据总线，可以是8、16、32、64、128、256、512或1024bit。

一个指示读取事务完成状态的读取响应

**写地址通道：**

每个写事务都有自己的地址通道。适当的地址通道携带通信所需的所有地址和控制信息。AXI协议支持以下机制：

可变的突发长度，每个突发长度从1-16个数据。

(连续传输的周期数就是突发长度。)

突发传送的大小为8-1024bit

...

写数据通道：

将写数据从主机传送到从机

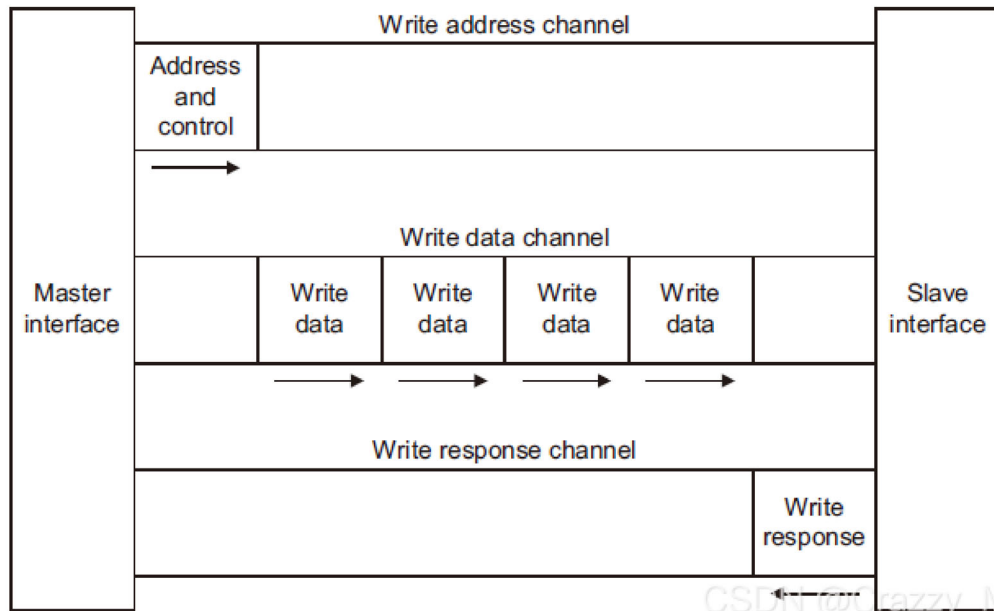
数据总线，可以是8、16、32、64、128、256、512或1024bit。

每八个数据位的一个字节通道选通脉冲，指示数据总线的哪个字节有效。

写响应通道：

写通道响应为从设备提供了一种响应事务的方法。所有写事务使用完成信令。

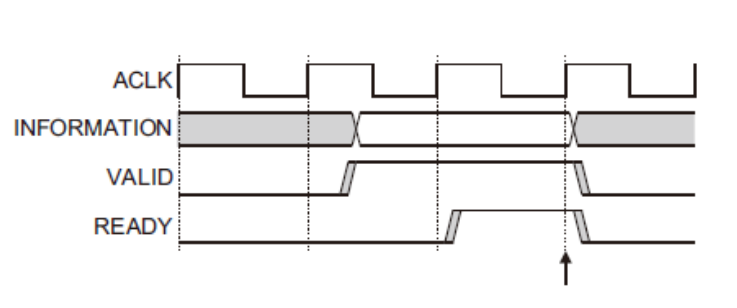
完成信号对于每个脉冲串仅发生一次，而不是针对其中的每个单独的数据传输发生一次突变。



数据传输使用写通道来实现master到slave的传输，slave使用写响应通道来完成一次写传输；读通道用来实现数据从slave到master的传输。

AXI使用基于VALID/READY的握手机制数据传输协议，传输源端使用VALID表明地址/控制信号、数据是有效的，目的端使用READY表明自己能够接受信息。

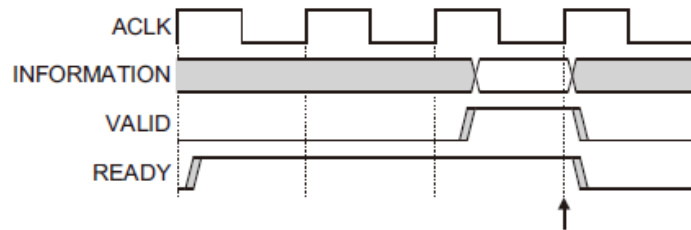
握手方式：

VALID在READY之前：**Figure 3-1 VALID before READY handshake**

主机放置数据或控制信号，并将VALID信号驱动为高电平。

来自主机的数据或控制信息保持稳定，直到目标驱动READY信号为高电平，表示它接受数据或控制信息，箭头显示何时开始传输。

在声明VALID之前，不允许等待READY声明。一旦VALID被断言它必须保持断言直到握手发生。

READY在VALID之前：**Figure 3-2 READY before VALID handshake**

目标驱动器在数据或控制信息被写入之前就驱动READY为高，这表明从机可以在一个接收器中接受数据或控制信息生效后立即循环。允许在声明相应的READY之前等待VALID声明。如果声明了READY，则允许在声明VALID之前取消声明READY。

VALID和READY同时：

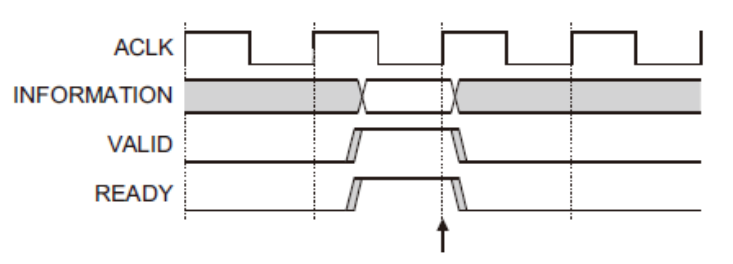


Figure 3-3 VALID with READY handshake

主机和从机恰好在同一周期内指示他们可以传输数据或控制信息。

二十四、RAM/SRAM/DRAM/SDRAM/DDR

RAM随机存储器

存储单元的内容可按需随意取出或存入，且存取的速度与存储单元位置无关的存储器。这种存储器在 **断电时将丢失其存储的内容**，故主要用于存储段时间使用的程序。

SRAM (Static Random-Access Memory)

静态随机存取存储器：只要供电它就会保持一个值。一般而言SRAM要比DRAM快，这是因为SRAM没有刷新周期。每个SRAM存储单元由6个晶体管组成，而DRAM存储单元由一个晶体管和一个电容器组成。相比而言，DRAM比SRAM每个存储单元的成本要高。照此推理，可以断定在给定的固定区域内DRAM的密度比SRAM 的密度要大。

DRAM (Dynamic Random Access Memory)

动态随机存取存储器：这是一种以电荷形式进行存储的半导体存储器。DRAM中的每个存储单元由一个晶体管和一个电容器组成。数据存储在电容器中。电容器会由于漏电而导致电荷丢失，因而DRAM器件是不稳定的。为了将数据保存在存储器中，DRAM器件必须有规律地进行刷新。

SRAM常常用于高速缓冲存储器，因为它有更高的速率；而DRAM常常用于PC中的主存储器，因为其拥有更高的密度。

SDRAM (synchronous dynamic random-access memory)

同步动态随机存储器：意思是指理论上其速度可达到与CPU同步。

DDR SDRAM(Dual Data Rate SDRAM)

简称DDR，也就是“双倍速率SDRAM”的意思。DDR可以说是SDRAM的升级版本，**DDR在时钟信号上升沿与下降沿各传输一次数据，这使得DDR的数据传输速度为传统SDRAM的两倍。**

二十五、ROM/PROM/EPROM/E2PROM/FLASH

ROM(Read-Only Memory)

只读存储器，这是一种线路最简单半导体电路，通过掩膜工艺，一次性制造，其中的代码与数据将永久保存（除非坏掉），不能修改。

PROM(Programmable Read-Only Memory)

可编程只读存储器。这样的产品只允许写入一次，所以也被称为“一次可编程只读存储器”(One Time Programming ROM, OTP-ROM)。PROM在出厂时，存储的内容全为1，用户可以根据需要将其中的某些单元写入数据0(部分的PROM在出厂时数据全为0，则用户可以将其中的部分单元写入，以实现对其“编程”的目的。

EPROM (Erasable Programmable Read-Only Memory)

它的特点是具有可擦除功能，擦除后即可进行再编程，但是缺点是擦除需要使用紫外线照射一定的时间。

EEPROM (Electrically Erasable Programmable Read-Only Memory)

它的最大优点是可直接用电信号擦除，也可用电信号写入。EEPROM不能取代RAM的原因是其工艺复杂，耗费的门电路过多，且重编程时间比较长，同时其有效重编程次数也比较低。

Flash memory

指的是“闪存”，所谓“闪存”，它也是一种非易失性的内存，属于EEPROM的改进产品。它的最大特点是必须按块(Block)擦除(每个区块的大小不定，不同厂家的产品有不同的规格)，而EEPROM则可以一次只擦除一个字节(Byte)。

二十六、硬核/软核/固核

1.硬核 (Hard IP Core)

硬核在EDA设计领域**指经过验证的设计版图**；具体在FPGA中指布局和工艺固定、经过前端和后端验证的设计，设计人员不能对其修改。不能修改的原因：

①：系统设计对各个模块的时序要求很严格，不允许打乱已有的物理版图。

②：保护只是产权的要求，不允许设计人员对其有任何改动。

IP 硬核的不许修改特点使其复用有一定的困难，因此只能用于某些特定应用，使用范围较窄。

2.软核 (Soft IP Core)

软核在EDA设计领域指的是**综合之前的寄存器传输级（RTL）模型**；具体在FPGA设计中指的是对电路的硬件语言描述，包括逻辑描述、网表和帮助文档等。软核只经过功能仿真，需要经过综合以及布局布线才能使用。其优点是灵活性高、可移植性强，允许用户自配置；缺点是对模块的预测性较低，在后续设计中存在发生错误的可能性，有一定的设计风险。软核是IP 核应用最广泛的形式。

3.固核 (Firm IP Core)

固核在EDA设计领域指的是**带有平面规划信息的网表**；具体在FPGA设计中可以看做带有布局布线规划的软核，通常以RTL代码和对应具体工艺网表的混合形式提供。将RTL 描述结合具体标准单元库进行综合优化设计，形成门级网表，再通过布局布线工具即可使用。和软核相比，固核的设计灵活性稍差，但在可靠性上有较大提高。目前，固核也是IP 核的主流形式之一。

例如当你使用一个IP核生成一个8×8的乘法器时，如果选择使用逻辑资源块来实现，那么此时的乘法器IP核就相当于一个**软核**；

如果你选择使用DSP资源来实现，那么此时的乘法器IP核就相当于一个**硬核**；

如果你要用DSP资源生成一个36×36的乘法器时，那么FPGA恐怕需要若干个DSP资源来实现，这时候，每个DSP核的布局布线都是固定的，但是到底选择哪几个DSP资源来实现是可以由编译器决定的，因此此时的乘法器IP核就相当于一个**固核**。

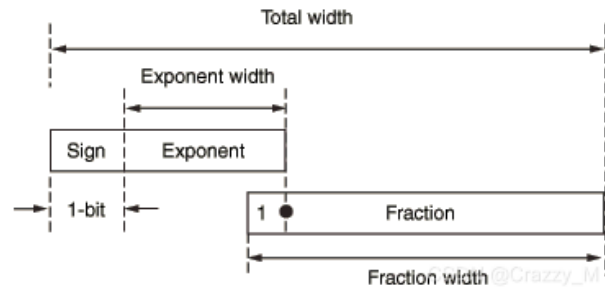
二十七、浮点数与定点数

1.32位浮点数

小数点非固定的数，可表示数据范围较广，整数，小数都可表示。包含float，double。

32位浮点数：

符号位	阶码	尾数
1位	8位	23位



真值=

$$V = (-1)^s \times M \times 2^E$$

s表示符号位，当s=0，V为正数；当s=1，V为负数。

E表示阶码。（阶码指明了小数点在数据中的位置）。

M表示尾数， $1 \leq M < 2$ 。（原理：所有的小数都能化成 $(1.xxx) \times 10^a$ ）。

通过具体实例来理解：

① 十进制小数转换为2进制

$$(100.25)_{10} = (1100100.01)_2$$

② 非规格数→规格化数（小数点在最高位后面）

$$(1100100.01)_2 = (1.10010001)_2 \times 2^6$$

③ 计算阶码（偏置值（127）+阶码真值（6））

$$1111111 + 110 = 1000101$$

④尾数（扩到23位）

10010001→100_1000_1000_0000_0000_0000

实际尾数是24位，默认最高位为1（观察上面的图），尾数M=

$$1 + (0.100_1000_1000_0000_0000_0000)_2$$

则32bit浮点数表示100.25为：

$$0|100_0010_1|100_1000_1000_0000_0000_0000$$

表示为十六进制为：42C88000

⑤ 反推 真值 = $\pm (1 + \text{尾数}) \times 2^{\text{阶码}-127}$

$$(1 + (0.100_1000_1000_0000_0000_0000)_2) \times 2^6 = (1 + 0.56640625)_{10} \times 64 = 100.25$$

再比如：

浮点数1.231，浮点数表示法为：

① 十进制小数转换为2进制

$$(1.231)_{10} = (1.00111011001000101)_2$$

② 非规格数→规格化数（小数点在最高位后面）

$$(1.00111011001000101)_2$$

③ 计算阶码（偏置值（127）+阶码真值（0））

$$1111111 + 000 = 1111111$$

④尾数（扩到23位）

版权：Crazyy_M

00111011001000101→001_1101_1001_0001_0100_0000

实际尾数是24位，默认最高位为1（观察上面的图），尾数M=

$$1 + (0.001_1101_1001_0001_0100_0000)_2$$

则32bit浮点数表示1.231为:

$$0|111_1111_1|001_1101_1001_0001_0100_0000$$

⑤ 反推验证 真值 = ± (1+尾数) × 2^ (阶码-127)

$$(1 + (0.001_1101_1001_0001_0100_0000)_2) \times 2^0 = (1 + 0.231)_{10} \times 1 = 1.231$$

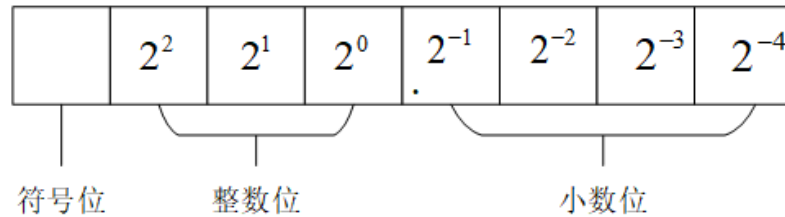
2.定点数

一种约定所有数值数据的小数点隐含在某一个固定的位置，称为定点表示法，简称定点数。

2^{-1}	0.5
2^{-2}	0.25
2^{-3}	0.125
2^{-4}	0.0625

以8bit为例，最高位符号位

若整数位占3位，小数位占4位，则其最大精度为0.0625，最大值为7.9375



...

若整数位占4位，小数位占3位，则其最大精度为0.125，最大值为15.875

若整数位占5位，小数位占2位，则其最大精度为0.250，最大值为31.750

若整数位占6位，小数位占1位，则其最大精度为0.500，最大值为63.500

若整数位占7位，小数位占0位，则其最大精度为1.000，最大值为127

3. 浮点数转换为定点数

举例来说，我们用8位定点数，1个符号位，4个整数位，3个小数位，则其可表示范围是-16.00~15.875，最大精度0.125。

有几个浮点数：0.145，1.231，2.364，7.512，每个却需要32bit表示。

如果我们可以将某些浮点数转换为定点数表示，在接受精度损失的前提下，每次就可以读取多个进行运行，可显著提高运算效率。

浮点数转换为定点数：

float 类型的1.231 转换为8位的定点数

我们用8位定点数，**1个符号位，4个整数位，3个小数位**。这个3称为量化系数。以下过程称为**量化**。

已知量为 float32(1.231)，转换为定点数int8(10)

$$int8(10) = float32(1.231) \times 2^3$$

比如：float 类型的1.231 转换为8位的定点数（我们已经规定好小数点在哪）。

$$1.231 \times 2^3 = 9.84(\text{取}10)_{10} = (0000_1010)_2$$

所以转换后的8bit为

$$(10)_{10} = (0000_1010)_2$$

0000_1010表示的定点数为（1个符号位，4个整数位，3个小数位）

$$(1.010)_2 = (1.25)_{10}$$

也就是说8bit的定点数，1个符号位，4个整数位，3个小数位1.25最能表示1.231，转换的时候精度出现了丢失。

float 类型的1.231 转换为8位的定点数

我们用8位定点数，**1个符号位，1个整数位，6个小数位**。这个6称为量化系数。以下过程称为**量化**。

已知量为 float32(1.231)，转换为定点数int8(10)

$$int8(10) = float32(1.231) \times 2^6$$

比如：float 类型的1.231 转换为8位的定点数（我们已经规定好小数点在哪）。

$$1.231 \times 2^6 = 78.784(\text{取}79)_{10} = (0100_1111)_2$$

所以转换后的8bit为

$$(79)_{10} = (0100_1111)_2$$

0100_1111表示的定点数为（1个符号位，1个整数位，6个小数位）

$$(1.001101)_2 = (1.234375)$$

也就是说8bit的定点数，1个符号位，1个整数位，6个小数位1.234375最能表示1.231，转换的时候精度也出现了丢失，但是越来越接近，通过两个例子自己领会。

3. 定点数转换为浮点数

定点数1.250 转换为浮点数

我们用8位定点数，**1个符号位，4个整数位，3个小数位**。这个3称为量化系数。以下过程称为**反量化**。

已知量为 $int8(10)$ ，转换为定点数 $float(1.250)$

$$float(1.250) = int8(10)/2^3$$

$$(10)_{10}/8 = (1.250)_{10}$$

达到CSDN字数限制，见博客 [FPGA/IC 秋招笔试/面试题总结（续）](#)

★★★如有错误，欢迎指导。

这这这也太卷了吧，那个天天拉我打王者的人既然进了大厂！我哭了！
这这这也太卷了吧，那个天天拉我打王者的人既然进了大厂！我哭了！这一次历时两个月，他拿到了一大堆的Offer...

m0_60720998的博客 84

[FPGA/IC 秋招笔试/面试题总结（续）](#)

Crazy_M的博客 101

[FPGA/IC 秋招笔试/面试题总结（续）](#) 来啦... 文章目录二十七、状态机1. Moore/Mealy状态机2.一、二、三段式状...

FPGA/IC 秋招笔试/面试题总结（续）



FPGA/IC秋招笔试面试题（续） 专栏收录该内容

持续更新，请根据目录浏览，觉得有用三连支持一下也是一种鼓励~。

原创不易，100页+，未经允许，请勿转载。

[FPGA/IC 秋招笔试/面试题总结（目录展示）](#)

[FPGA/IC 秋招笔试/面试题总结（续）](#) 来啦...

打印版，联系微信：[Crazyy_M](#)

文章目录

二十八、状态机

- 1. Moore/Mealy状态机
- 2. 一、二、三段式状态机（序列检测器）

二十九、序列检测器另一种实现方法（移位寄存器）

三十、FPGA和CPLD的区别

三十一、关键路径

三十二、单脉冲跨时钟域传输（快到慢）（慢到快）Verilog代码实现

- 1. 单脉冲，慢到快
- 2. 单脉冲，快到慢

三十三、多bit数据跨时钟域（握手机制）

- 1. 单字节跨时钟域
- 2. 多字节跨时钟域

三十四、Verilog实现glitch free（无毛刺）时钟切换电路

- 1. 两个相关时钟源（频率成倍数关系）切换
- 2. 异步时钟切换

三十五、Verilog实现8bit环形计数器

三十六、Verilog初始化一个ROM（用于仿真）

三十七、数值（有无符号）运算、比较

- 1. 原码、反码、补码
- 2. 有符号数比较器
- 3. 有符号数求绝对值
- 4. 有符号数加法器

- 1.CRC原理介绍
- 2.CRC简介
- 3.通过实例来了解CRC
- 4.CRC 计算器的使用
- 5.CRC电路解析
- 6.CRC Verilog实现 (并行)
- 7.CRC Verilog实现 (串行)

三十九、按键消抖

- 1.只对按键按下消抖
- 2.按键按下、松开都进行消抖

四十、OC门和OD门

- 1.OC门
- 2.OD门 (漏极开路输出)

四十一、数字IC设计流程、数字IC后端设计流程

- 1.数字IC设计流程
- 2.数字IC 后端设计流程

四十二、芯片功耗问题

- 1.动态功耗
- 2.静态功耗

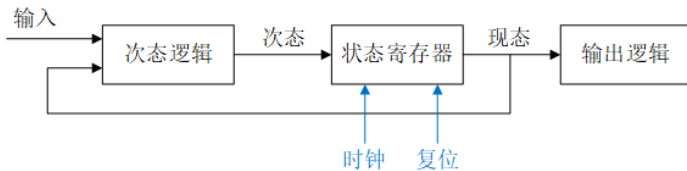
四十三、计算时钟频偏

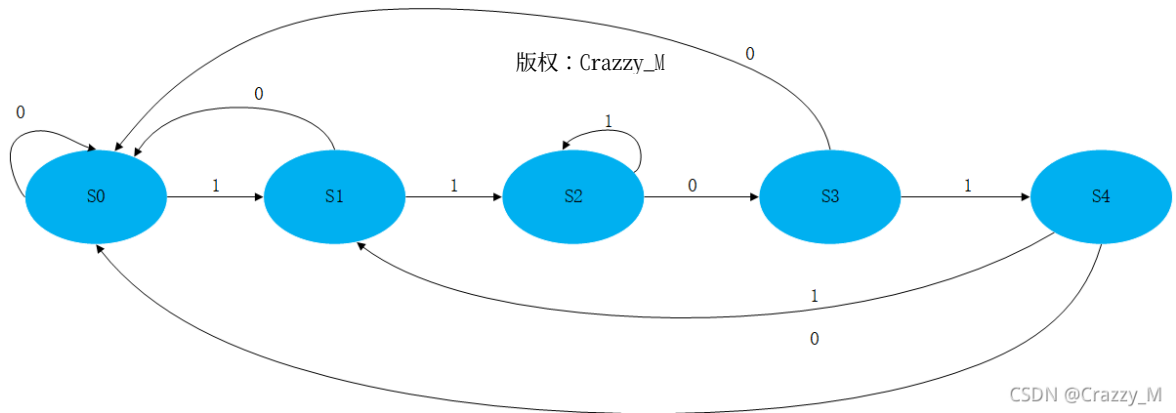
二十八、状态机

1. Moore/Mealy状态机

(1) **Moore**状态机:

输出只由当前状态决定，即次态= f (现态。输入) , 输出= f (现态)





Moore型状态机输出函数只由状态变量决定，想要输出 $Z=1$ ，必须 $S4$ 状态形成，即寄存器中的 1101 都打进去后才可以。输出 $Z=1$ 会在下一个有效沿到来的时候被赋值。

状态机输出部分：

```

1  always @(*)
2  begin
3      if(current_state == s4) dout = 1;
4      else dout = 0;
5  end

```

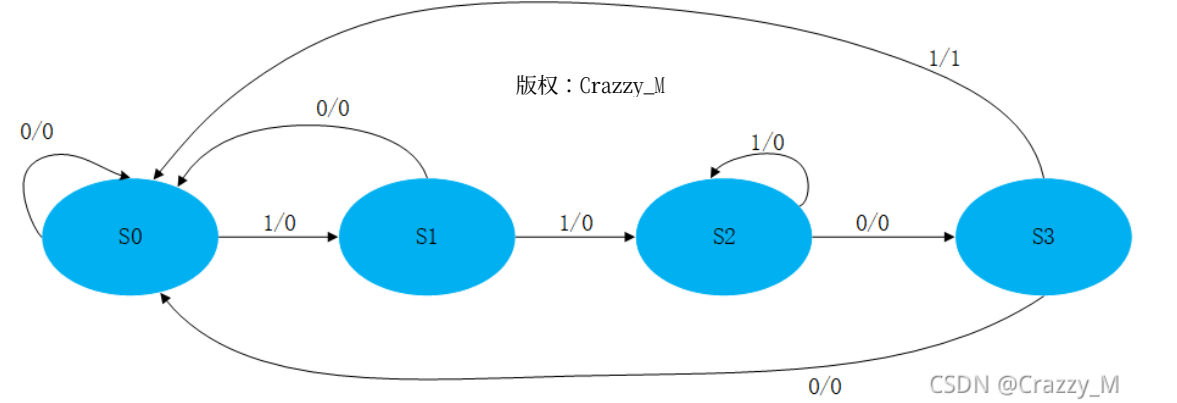
(2) Mealy状态机：

输出不但与当前状态有关，还与当前输入值有关，即次态= f （现态，输入），输出= f （现态，输入）



Mealy型状态机则是以现时的输入信号结合即将变成次态的现态，编码成输出信号。

Mealy型状态机检测序列 1101 如下



Mealy型状态机输出函数是由输入和状态变量共同决定的。状态在**S3**的时候如果输入为**1**，直接以组合电路输出**Z=1**，不需要等到下一个有效沿到来，从而不需要**S4**状态。

状态机输出部分：

```
1 always @ (*)
2 begin
3     if(reset) dout = 1'b0;
4     else if( (current_state == s3)&&(din == 1'b1) ) dout = 1'b1;
5     else dout = 1'b0;
6
7 end
```

Moore型**Mealy**型比较：

Moore型状态机**使用更安全**。

输出在时钟沿变化。（**Moore**型比**Mealy**型输出延迟一个节拍）

在**Mealy**机器中，输入更改可能会在逻辑完成后立即导致输出更改，当两台机器互连时出现大问题 - 如果

不小心，可能会发生异步反馈。

Mealy型状态机**对输入的反应更快**

在相同周期内反应-不需要等待时钟。

在**Moore**机器中，可能需要更多逻辑来将状态解码为输出 - 在时钟边沿之后更多的门延迟。

并非所有时序电路都可以使用**Mealy**模型实现。一些时序电路只能作为摩尔机器实现。

2.一、二、三段式状态机（序列检测器）

一段式状态机	所有的逻辑（输入、输出、状态）都在一个 always 中解决。
两段式状态机	时序逻辑和组合逻辑划分开（两个 always ） 时序逻辑中进行当前状态和下一状态的切换 组合逻辑中实现各个输入、输出、以及状态的判断

	(三个always)
三段式状态机	always 1 时序逻辑实现当前状态和下一状态的切换 always 2 组合逻辑，实现状态的判断 always 3 时序逻辑，实现状态下的输入输出（也可以组合电路输出）

一段式状态机不利于维护

两段式状态机相对容易维护，不过组合逻辑电路输出较易出现毛刺等常见的问题

三段式状态机代码易于维护，时序逻辑的输出解决了两段式写法中组合逻辑的毛刺问题。

通过编写“1101”序列检测器，输入序列为“1010_1101_0101_1101”，进行对一、二、三状态机做以解释：

1.一段式状态机

所有的逻辑（输入、输出、状态）都在一个always中解决。

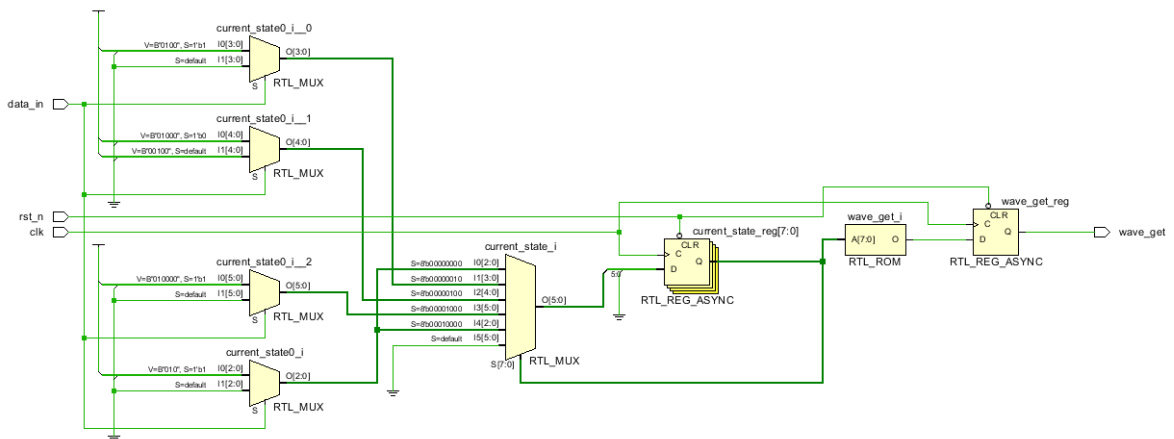
```
1 //一段式状态机
2 //1101序列检测器
3 module top
4 (
5     input      clk,
6     input      rst_n,
7
8     input      data_in,
9     output reg wave_get
10
11 );
12
13 parameter  S0 = 8'b0000_0000,
14            S1 = 8'b0000_0010,
15            S2 = 8'b0000_0100,
16            S3 = 8'b0000_1000,
17            S4 = 8'b0001_0000;
18
19 reg[7:0]   current_state; //当前状态
20
21 always@(posedge clk or negedge rst_n)
22 begin
23     if(!rst_n)
24     begin
25         current_state <= S0;
26         wave_get  <= 1'b0;
27     end
28     else
29     begin
30         case(current_state)
31             S0:
32             begin
33                 current_state <= (data_in == 1'b1)?S1:S0;
34                 wave_get  <= 1'b0;
35             end
36             S1:
37             begin
38                 current_state <= (data_in == 1'b1)?S2:S0;
```

```

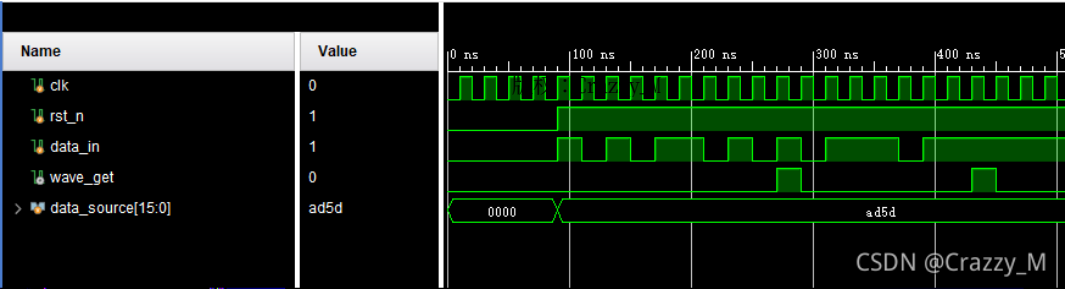
39     wave_get <= 1'b0;
40 end
41 S2:
42 begin
43     current_state <= (data_in == 1'b0)?S3:S2;
44     wave_get <= 1'b0;
45 end
46 S3:
47 begin
48     current_state <= (data_in == 1'b1)?S4:S0;
49     wave_get <= 1'b0;
50 end
51 S4:
52 begin
53     current_state <= (data_in == 1'b1)?S1:S0;
54     wave_get <= 1'b1;
55 end
56 default:
57 begin
58     current_state <= S0;
59 end
60 endcase
61 end
62
63 end
64
65 endmodule
66

```

RTL视图:



CSDN @Crazy_M



2.二段式状态机

时序逻辑和组合逻辑划分开（两个always），
时序逻辑中进行当前状态和下一个状态的切换，
组合逻辑中实现输入、输出、以及状态的判断。

```
1 //二段式状态机
2 //1101序列检测器
3 module top
4 (
5     input      clk,
6     input      rst_n,
7
8     input      data_in,
9     output reg wave_get
10
11 );
12
13 parameter  S0 = 8'b0000_0000,
14            S1 = 8'b0000_0010,
15            S2 = 8'b0000_0100,
16            S3 = 8'b0000_1000,
17            S4 = 8'b0001_0000;
18
19
20 reg[7:0]    current_state; //当前状态
21 reg[7:0]    next_state; //当前状态
22
23 //时序逻辑实现当前状态和下一个状态的切换
24 always@(posedge clk or negedge rst_n)
25 begin
26     if(!rst_n)
27         current_state <= S0;
28     else
29         current_state <= next_state;
30 end
31
32 //组合逻辑中实现输入、输出、以及状态的判断
33 always@(*)
34 begin
35     case(current_state)
36         S0:
```

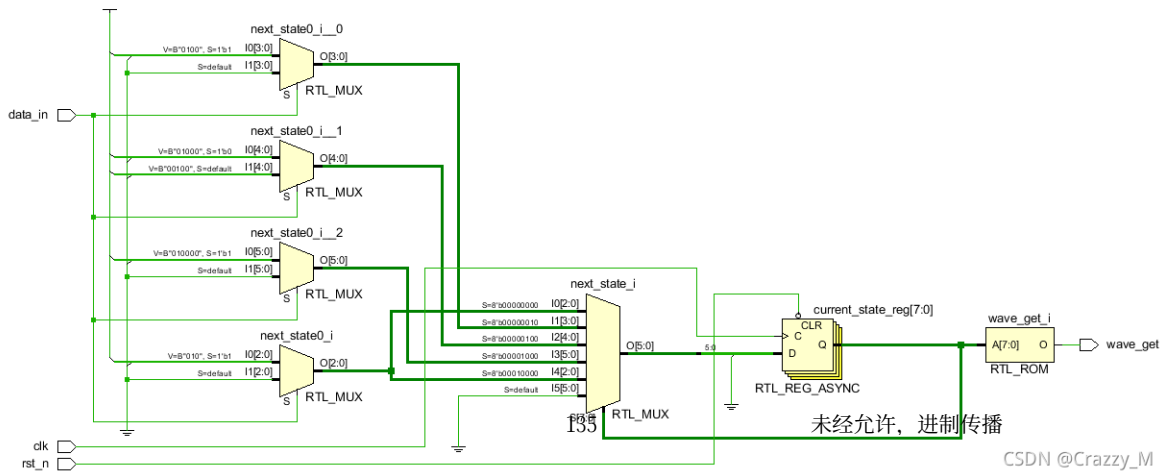
```

37     begin
38         next_state = (data_in == 1'b1)?S1:S0;
39         wave_get    = 1'b0;
40     end
41     S1:
42     begin
43         next_state = (data_in == 1'b1)?S2:S0;
44         wave_get    = 1'b0;
45     end
46     S2:
47     begin
48         next_state = (data_in == 1'b0)?S3:S2;
49         wave_get    = 1'b0;
50     end
51     S3:
52     begin
53         next_state = (data_in == 1'b1)?S4:S0;
54         wave_get    = 1'b0;
55     end
56     S4:
57     begin
58         next_state = (data_in == 1'b1)?S1:S0;
59         wave_get    = 1'b1;
60     end
61     default:
62     begin
63         next_state = S0;
64     end
65 endcase
66 end
67
68 endmodule

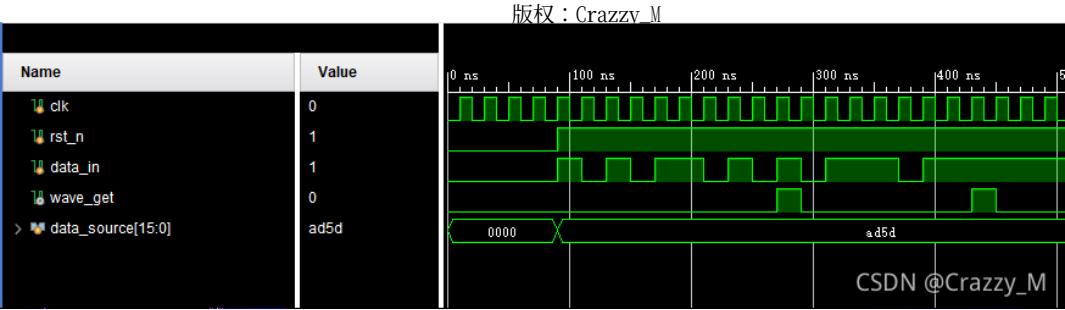
```

版权：Crazy_M

RTL视图：



观察波形， wave_get信号是在 1101后一个周期输出的， 因为走了S4这个状态（观察状态转移图）



3.三段式状态机

3个always

always1 时序逻辑，实现当前状态和下一个状态的切换，

always2 组合逻辑，实现状态的判断。

always3 时序逻辑，实现状态下的输入输出。

```
1 //三段式状态机
2 module top
3 (
4     input      clk,
5     input      rst_n,
6
7     input      data_in,
8     output     reg wave_get
9
10 );
11
12
13 parameter  S0 = 8'b0000_0000,
14            S1 = 8'b0000_0010,
15            S2 = 8'b0000_0100,
16            S3 = 8'b0000_1000,
17            S4 = 8'b0001_0000;
18
19 reg[7:0]   current_state; //当前状态
20 reg[7:0]   next_state; //当前状态
21
22 //时序逻辑实现当前状态和下一个状态的切换
23 always@(posedge clk or negedge rst_n)
24 begin
25     if(!rst_n)
26         current_state <= S0;
27     else
28         current_state <= next_state;
29 end
30
31 //组合逻辑中实现输入、输出、以及状态的判断
32 always@(*)
```

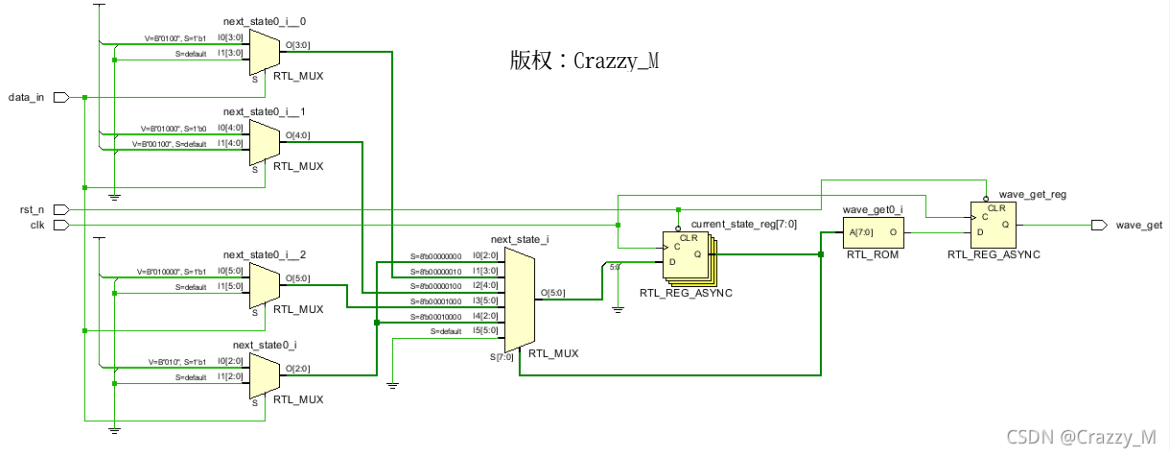
```

33 begin
34     case(current_state)
35         S0:
36             begin
37                 next_state = (data_in == 1'b1)?S1:S0;
38             end
39         S1:
40             begin
41                 next_state = (data_in == 1'b1)?S2:S0;
42             end
43         S2:
44             begin
45                 next_state = (data_in == 1'b0)?S3:S2;
46             end
47         S3:
48             begin
49                 next_state = (data_in == 1'b1)?S4:S0;
50             end
51         S4:
52             begin
53                 next_state = (data_in == 1'b1)?S1:S0;
54             end
55         default:
56             begin
57                 next_state = S0;
58             end
59     endcase
60 end
61
62 //时序逻辑实现状态下的输出
63 always@(posedge clk or negedge rst_n)
64 begin
65     if(!rst_n)
66     begin
67         wave_get    <= 1'b0;
68     end
69     else
70     begin
71         wave_get    <= (current_state == S4)?1'b1:1'b0;
72     end
73 end
74
75 endmodule

```

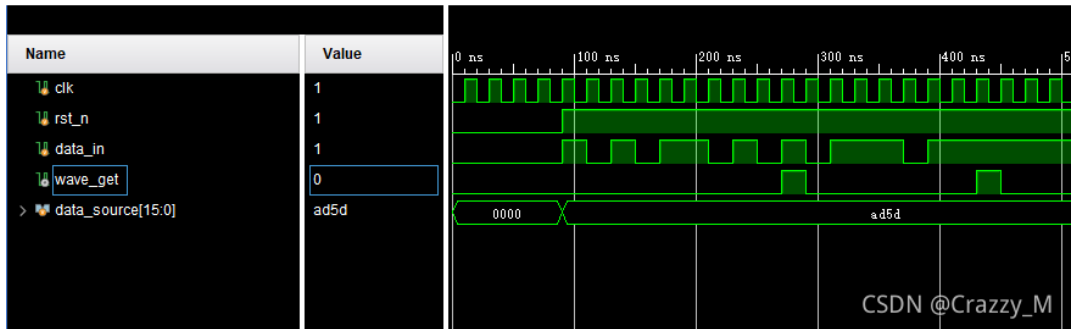
版权：Crazy_M

RTL视图:



CSDN @Crazy_M

观察波形，wave_get信号是在1101后一个周期输出的，因为走了S4这个状态（观察状态转移图）



CSDN @Crazy_M

testbench:

```
1 `timescale 1ns / 1ps
2 //
3 // Company:
4 // Engineer:
5 //
6 // Create Date: 2021/08/26 21:19:53
7 // Design Name:
8 // Module Name: top_test
9 // Project Name:
10 // Target Devices:
11 // Tool Versions:
12 // Description:
13 //
14 // Dependencies:
15 //
16 // Revision:
```

```

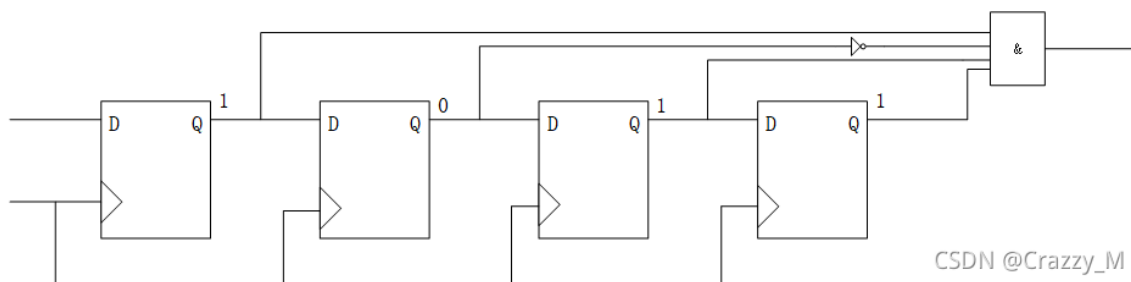
17 // Revision 0.01 - File Created
18 // Additional Comments:
19 //
20 //
21
22
23 module top_test;
24
25     reg        clk;
26     reg        rst_n;
27
28     reg        data_in;
29     wire       wave_get;
30
31     reg[15:0]  data_source;
32
33     top u1
34     (
35         .clk(clk),
36         .rst_n(rst_n),
37
38         .data_in(data_in),
39         .wave_get(wave_get)
40     );
41
42     initial
43     begin
44         clk          = 0;
45         rst_n        = 0;
46         data_in       = 0;
47         data_source = 16'b0000_0000_0000_0000;
48
49         repeat(5)@(posedge clk);
50         rst_n    = 1;
51         data_source = 16'b1010_1101_0101_1101;
52
53         wave_generate();
54
55     end
56
57     task wave_generate;
58     integer i;
59     begin
60         for(i=0;i<16;i=i+1)
61         begin
62             data_in <= data_source[15-i];
63             @(posedge clk);
64         end
65     end
66
67 endtask
68
69 always # 10 clk = ~clk;
70
71 endmodule

```

版权：Crazy_M

二十九、序列检测器另一种实现方法（移位寄存器）

例如检测序列1101可以用以下移位寄存器的方式：



CSDN @Crazy_M

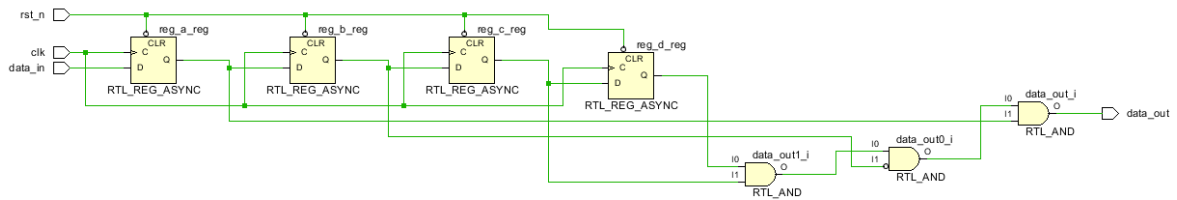
```

1 //1101序列检测器
2 module top
3 (
4     input  clk,
5     input  rst_n,
6     input  data_in,
7
8     output data_out
9 );
10
11
12 reg reg_a;
13 reg reg_b;
14 reg reg_c;
15 reg reg_d;
16
17 assign data_out = reg_d&reg_c&(~reg_b)&reg_a;
18
19 always@(posedge clk or negedge rst_n)
20 begin
21     if(!rst_n)
22     begin
23         reg_a <= 1'b0;
24         reg_b <= 1'b0;
25         reg_c <= 1'b0;
26         reg_d <= 1'b0;
27     end
28     else
29     begin
30         reg_d <= reg_c;
31         reg_c <= reg_b;
32         reg_b <= reg_a;
33         reg_a <= data_in;
34     end
35 end
36 endmodule

```

RTL视图:

版权: Crazy_M



CSDN @Crazy_M

testbench

```

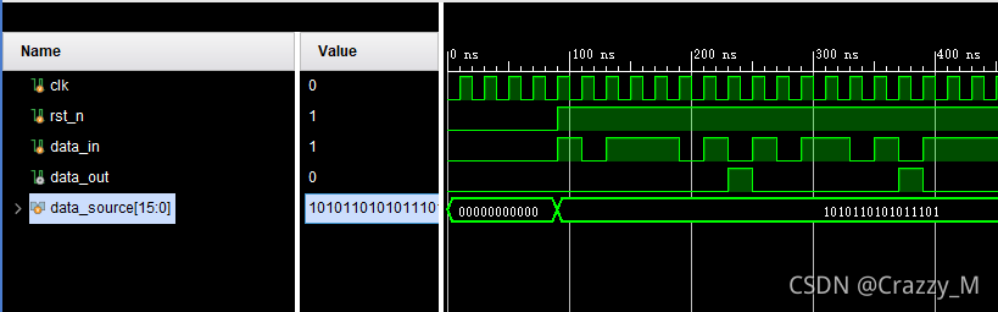
1 module top_test;
2
3 reg      clk;
4 reg      rst_n;
5 reg      data_in;
6 wire     data_out;
7
8 reg[15:0] data_source;
9
10 top u1
11 (
12     .clk(clk),
13     .rst_n(rst_n),
14
15     .data_in(data_in),
16     .data_out(data_out)
17 );
18
19 initial
20 begin
21     clk      = 0;
22     rst_n    = 0;
23     data_source = 16'b0000_0000_0000_0000;
24     data_in   = 0;
25
26     repeat(5)@(posedge clk);
27     rst_n = 1;
28     data_source = 16'b1010_1101_0101_1101;
29
30     data_gen();
31 end
32
33 task data_gen;

```

```
34 integer i;
35 begin
36     for(i=0;i<16;i=i+1)
37         begin
38             data_in <= data_source[i];
39             @(posedge clk);
40         end
41     end
42 endtask
43
44 always #10 clk = ~clk;
45
46 endmodule
```

版权：Crazy_M

波形：



三十、FPGA和CPLD的区别

	CPLD	FPGA
内部结构	Product term （基于乘机项）	Look up Table （基于查找表）
程序存储	内部EEPROM/FLASH	SRAM， 外挂EEPROM
资源类型	组合逻辑资源丰富	时序逻辑资源丰富
集成度	低	高
使用场合	完成控制逻辑	能完成较复杂的算法
速度	慢	快？
其他资源	-	PLL、RAM、乘法器
保密性	可加密	一般不能保密

三十一、关键路径

关键路径就是输入到输出延时最大的路径，找到了关键路径便能求得最大时钟频率。

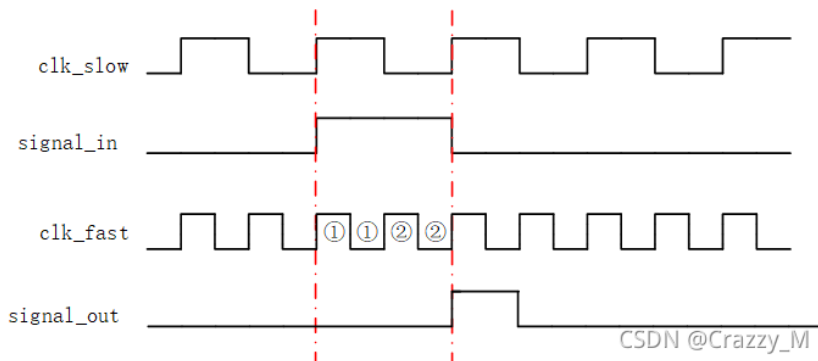
未经允许，进制传播

三十二、单脉冲跨时钟域传输（快到慢）（慢到快）Verilog代码实现

1.单脉冲，慢到快

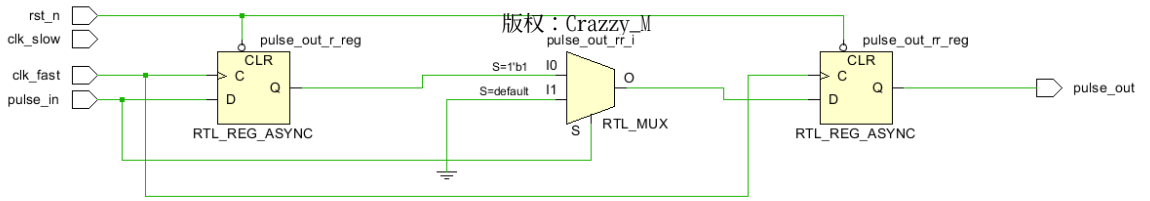
单脉冲从慢到快，可以打两拍输出

版权：Crazyy_M

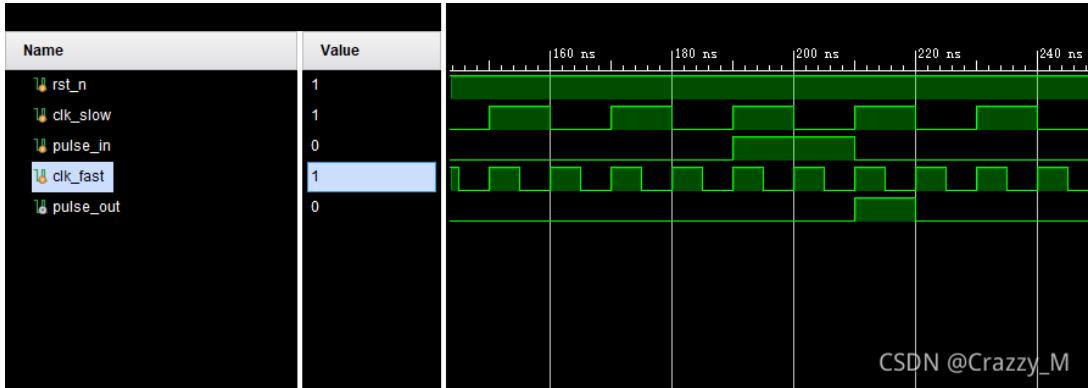


Verilog代码

```
1 //单脉冲 慢到快
2 module top
3 (
4     input  rst_n,
5     input  clk_fast,
6     input  clk_slow,
7     input  pulse_in,
8
9     output pulse_out
10 );
11
12 reg pulse_out_r;           //一拍
13 reg pulse_out_rr;         //两拍
14
15 assign pulse_out = pulse_out_rr;
16 always@(posedge clk_fast or negedge rst_n) //快时钟采样
17 begin
18     if(!rst_n)
19         {pulse_out_rr,pulse_out_r} <= {2{1'b0}};
20     else if(pulse_in == 1'b1)
21         {pulse_out_rr,pulse_out_r} <= {pulse_out_r,pulse_in};
22     else
23         {pulse_out_rr,pulse_out_r} <= {2{1'b0}};
24 end
25 end
26
27 endmodule
28
```



CSDN @Crazy_M



testbench:

```

1 module top_test;
2
3   reg    rst_n;
4   reg    clk_fast;
5   reg    clk_slow;
6   reg    pulse_in;
7   wire   pulse_out;
8
9
10  top u1
11  (
12    .rst_n (rst_n ),
13    .clk_fast (clk_fast ),
14    .clk_slow (clk_slow ),
15    .pulse_in (pulse_in ),
16
17    .pulse_out(pulse_out)
18  );
19
20
21
22  initial
23  begin

```

```

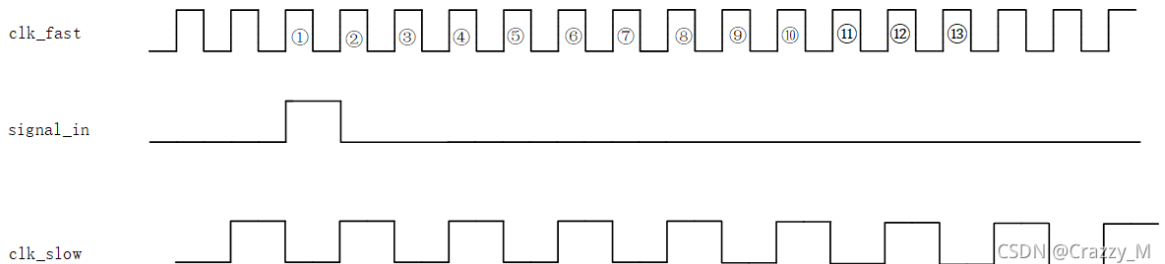
24     clk_fast      = 1;
25     clk_slow      = 0;
26     rst_n         = 0;
27     pulse_in      = 0;
28
29     repeat(5)@(posedge clk_slow);
30     rst_n         = 1;
31     repeat(5)@(posedge clk_slow);
32     pulse_gen();
33
34 end
35
36
37 task pulse_gen;
38 begin
39     pulse_in <= 1;
40     @(posedge clk_slow);
41     pulse_in <= 0;
42 end
43 endtask
44
45 always #10 clk_slow = ~clk_slow;
46 always #5  clk_fast  = ~clk_fast;
47
48 endmodule

```

版权：Crazy_M

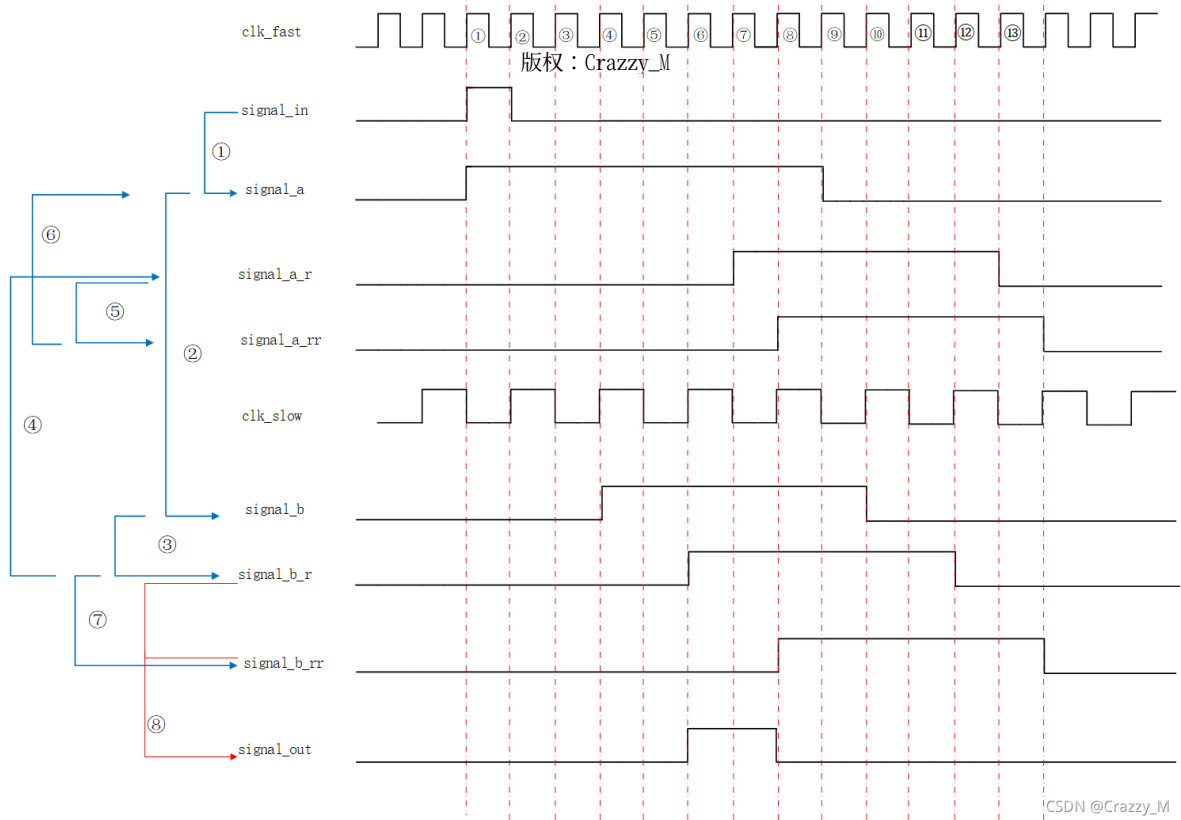
2.单脉冲，快到慢

单脉冲，从快时钟域到慢时钟域，如下图，clk_slow采集signal，通过两级寄存器同步是没用的。



CSDN@Crazy_M

快时钟域，到慢时钟域同步，采用脉冲展宽的方法。如下图所示（重点看下图的信号）



信号：

signal_in：

clk_fast时钟域产生的单脉冲。

signal_a：

clk_fast时钟域展宽的信号。

signal_a_r：

展宽数据signal_b在clk_slow同步到clk_fast时钟一拍数据。

signal_a_rr：

展宽数据signal_b在clk_slow同步到clk_fast时钟二拍数据(两拍才稳定)

(该数据为快时钟域采集到的 稳定的慢时钟域已经采集到的展宽信号的反馈信号)

此时钟信号可以作为拉低signal_a的判断。

signal_b：

展宽数据signal_a在clk_fast同步到clk_slow时钟一拍数据。

signal_b_r：

展宽数据signal_a在clk_fast同步到clk_slow时钟二拍数据(两拍才稳定)。

signal_b_rr：

展宽数据signal_a在clk_fast同步到clk_slow时钟三拍数据。

signal_b_r：

该信号为慢时钟域采集到稳定的展宽信号。

signal_b_rr:

该信号为慢时钟域采集到稳定的展宽信号的延迟一拍

这两个信号用于产生输出单脉冲 $\text{pulse_out} = (\sim\text{signal_b_rr}) \& \text{signal_b_r}$

版权：Crazy_M

(重点看下图的路径①~⑧，和上面图是一样的)

思路（按照上图的序号路径走~）：

路径①：

输入信号在clk_fast时钟域将signal_in展宽为signal_a（在快时钟域检测到signal_in==1，拉高signal_a，慢时钟域采集到展宽的信号后拉低signal_a）。

路径②：

clk_fast时钟域展宽的信号signal_a同步到clk_slow时钟域生成信号signal_b。

路径③：

clk_slow时钟域signal_b打一拍生成signal_b_r（相当于打两拍得到了慢时钟域采样的signal_a稳定数据，降低了亚稳态）。

路径④：

clk_slow采样的signal_b_r数据同步到clk_fast时钟域生成signal_a_r信号。

路径⑤：

clk_fast时钟域的signal_a_r打一拍生成signal_a_rr信号，在clk_fast时钟域的该信号指示了clk_slow时钟域已经采集到了展宽的脉冲，捕捉到此时就可以将展宽脉冲拉低了。

路径⑥：

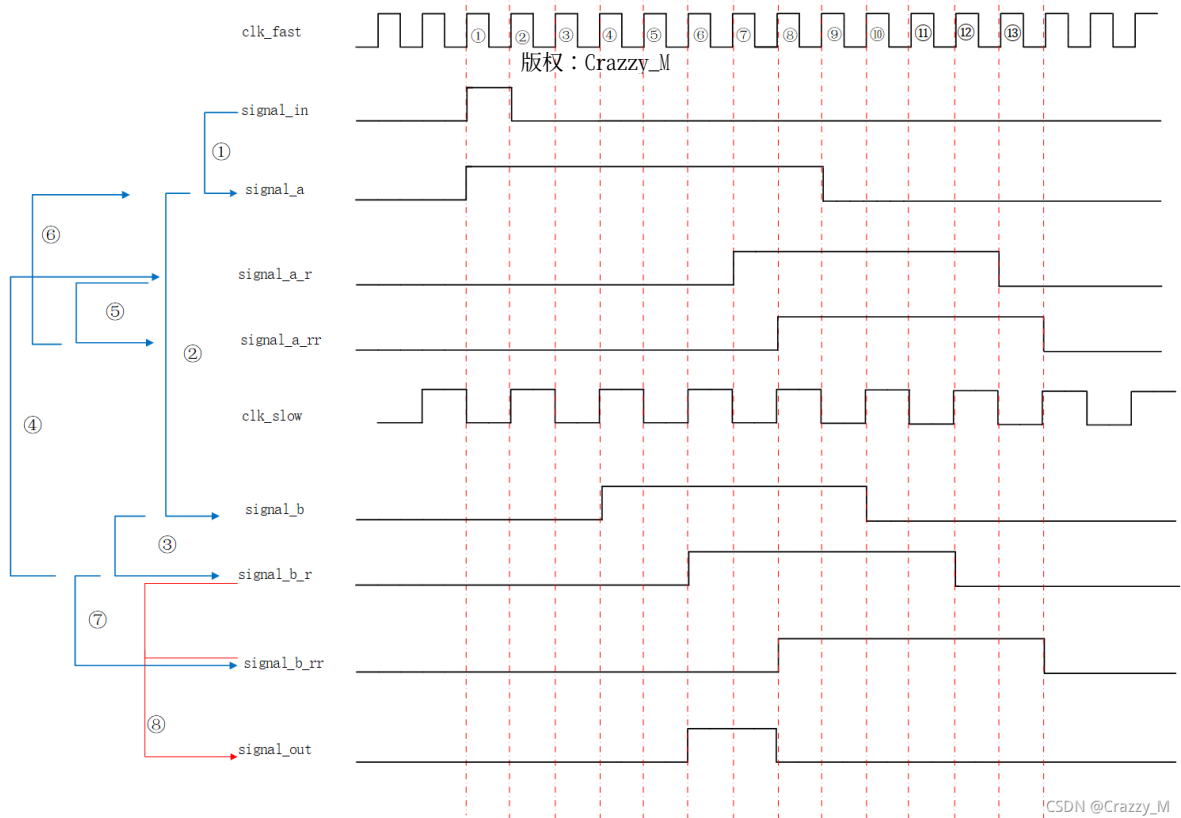
signal_a_rr为1时就可以把展宽脉冲signal_a拉低。

路径⑦：

clk_slow时钟域signal_b_r打一拍生成signal_b_rr。

路径⑧：

signal_b打一拍后为signal_b_r，signal_b_r打一拍后为signal_b_rr，也就是我们利用clk_slow时钟域展宽信号的第二拍和第三拍数据进行取反相与（ $\text{pulse_out} = (\sim\text{signal_b_rr}) \& \text{signal_b_r}$ ）就得到了clk_slow时钟域的的脉冲（看到有人用第一拍和第二拍，个人感觉应该不太妥）。



verilog代码：

```

1 //单脉冲 慢到快
2 module top
3 (
4     input  rst_n,
5     input  clk_fast,
6     input  clk_slow,
7     input  pulse_in,
8
9     output pulse_out
10 );
11
12 reg    signal_a;    //clk_fast时钟域展宽的脉冲
13 reg    signal_a_r;  //展宽数据signal_b在clk_slow同步到clk_fast时钟一拍数据
14 reg    signal_a_rr; //展宽数据signal_b在clk_slow同步到clk_fast时钟二拍数据(两拍才稳定)
15                                //(该数据为快时钟域采集到的 稳定的慢时钟域已经采集到的展宽信号的反馈信号)
16                                //此时钟信号可以作为拉低signal_a的判断
17
18 reg    signal_b;    //展宽数据signal_a在clk_fast同步到clk_slow时钟一拍数据 未经允许，进制传播
19 reg    signal_b_r;  //展宽数据signal_a在clk_fast同步到clk_slow时钟二拍数据(两拍才稳定)
20 reg    signal_b_rr; //展宽数据signal_a在clk_fast同步到clk_slow时钟三拍数据

```

```

21 //signal_b_r:该信号为慢时钟域采集到稳定的展览信号的延迟一拍
22 //signal_b_rr:该信号为慢时钟域采集到稳定的展览信号的延迟一拍
23 //这两个信号用于产生输出单脉冲 pulse_out = (~signal_b_rr) & signal_b_r;
24 版权：Crazzy_M
25
26
27 //在快时钟域将脉冲展宽
28 always@(posedge clk_fast or negedge rst_n) //快时钟
29 begin
30     if(!rst_n)
31         signal_a <= 1'b0;
32     else if(pulse_in == 1'b1)
33         signal_a <= 1'b1;
34     else if(signal_a_rr == 1'b1) //检测到慢时钟的反馈后展览信号拉低
35         signal_a <= 1'b0;
36     else
37         signal_a <= signal_a;
38 end
39
40 //将展宽的脉冲同步到慢时钟域(打两拍)
41 always@(posedge clk_slow or negedge rst_n) //慢时钟
42 begin
43     if(!rst_n)
44         {signal_b_rr,signal_b_r,signal_b} <= {3{1'b0}};
45     else
46         {signal_b_rr,signal_b_r,signal_b} <= {signal_b_r,signal_b,signal_a};
47 end
48
49 assign pulse_out = (~signal_b_rr) & signal_b_r;
50
51 //在快时钟域，采集signal_b_r(慢时钟已经采集到的展览的脉冲)后作为快时钟域展览脉冲拉低信号
52 always@(posedge clk_fast or negedge rst_n) //快时钟
53 begin
54     if(!rst_n)
55         {signal_a_rr,signal_a_r} <= {2{1'b0}};
56     else
57         {signal_a_rr,signal_a_r} <= {signal_a_r,signal_b_r};
58
59 end
60
61 endmodule
62

```

testbench:

```

1 module top_test;
2
3 reg    rst_n;
4 reg    clk_fast;
5 reg    clk_slow;
6 reg    pulse_in;
7 wire    pulse_out;
8
9 top u1
10 (

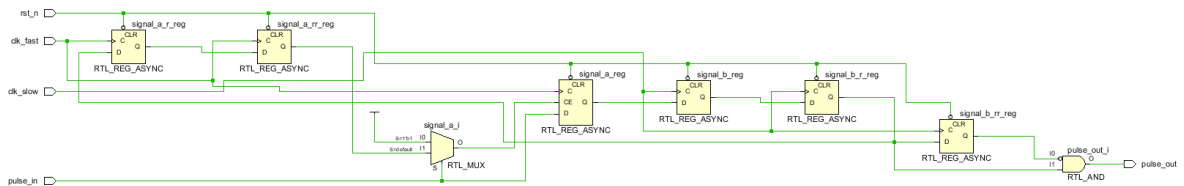
```

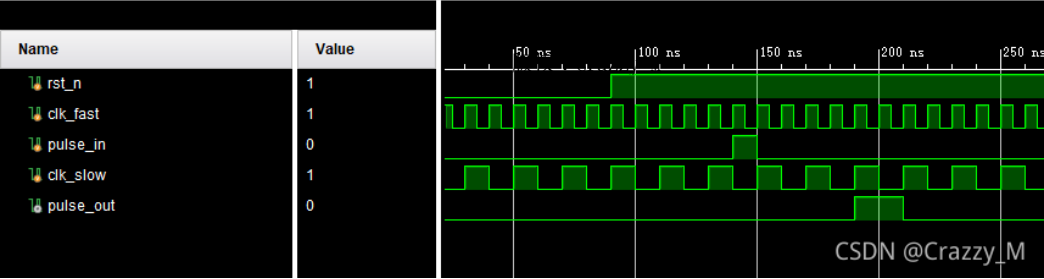
```

11     .rst_n    (rst_n    ),
12     .clk_fast (clk_fast ),
13     .clk_slow (clk_slow ),
14     .pulse_in (pulse_in ),
15
16     .pulse_out(pulse_out)
17
18 );
19
20 initial
21 begin
22     clk_fast    = 1;
23     clk_slow    = 0;
24     rst_n       = 0;
25     pulse_in    = 0;
26
27     repeat(5)@(posedge clk_slow);
28     rst_n       = 1;
29     repeat(5)@(posedge clk_fast);
30     pulse_gen();
31
32 end
33
34 task pulse_gen;
35 begin
36     pulse_in <= 1;
37     @(posedge clk_fast);
38     pulse_in <= 0;
39 end
40 endtask
41
42 always #10 clk_slow = ~clk_slow;
43 always #5  clk_fast = ~clk_fast;
44
45 endmodule

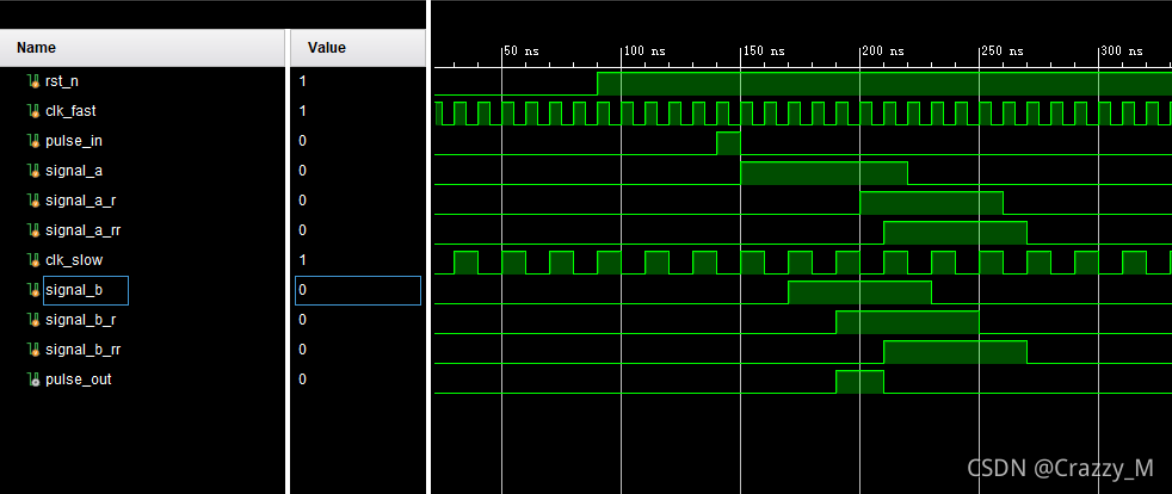
```

RTL视图:

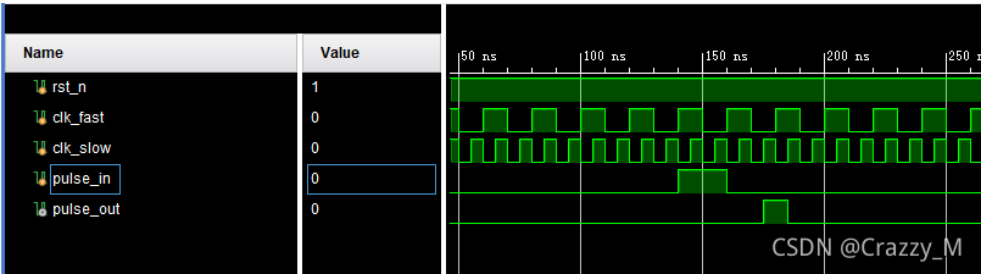




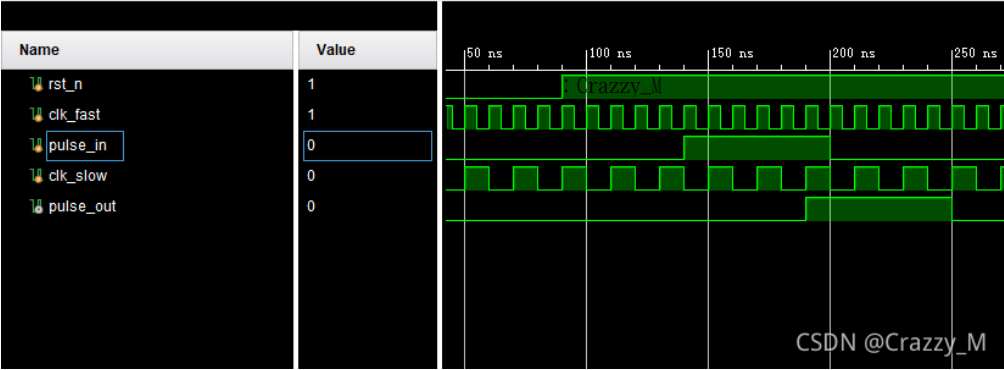
详细波形：



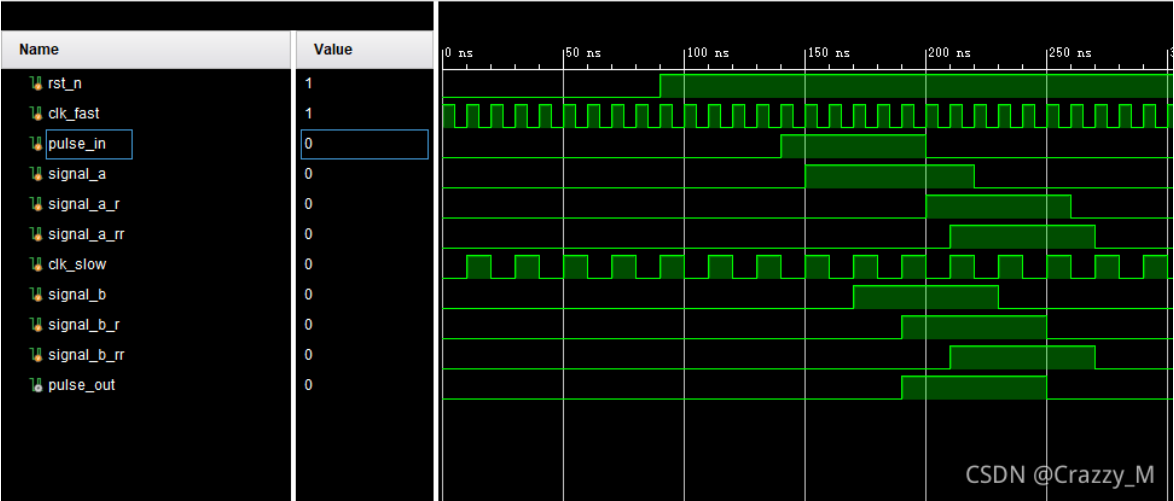
其实本代码也适用于慢时钟域到快时钟域的单脉冲数据传输：



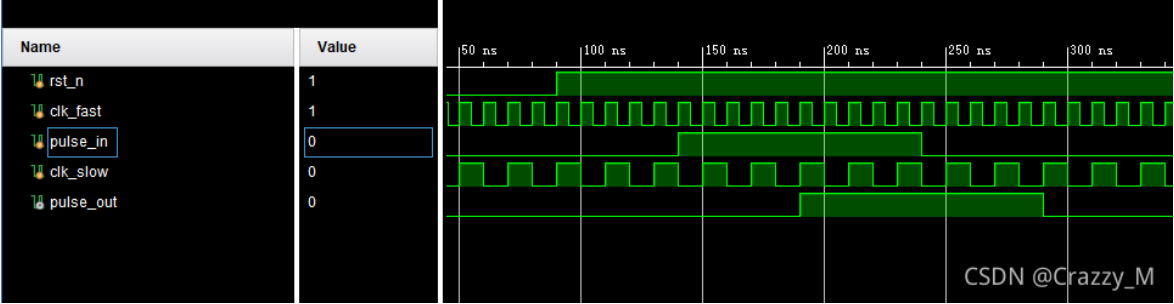
在时钟为2倍关系，输入脉冲宽度大于4个周期时，还可以传输相同长度的信号（信号长度为6个快时钟周期）：



详细波形：



信号长度为10个快时钟周期)：



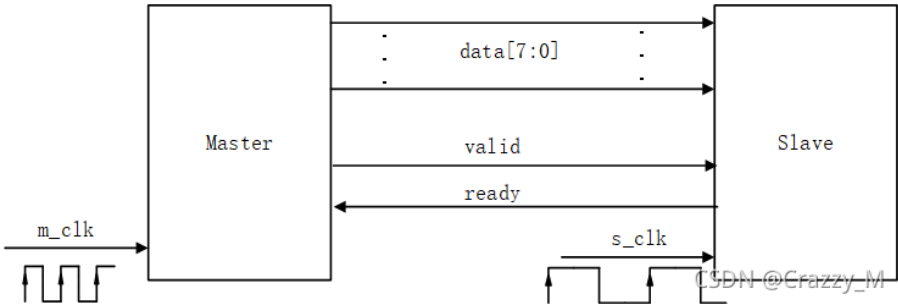
```

1 module top_test;
2
3 reg    rst_n;
4 reg    clk_fast;
5 reg    clk_slow;
6 reg    pulse_in;
7 wire   pulse_out;
8
9
10 top u1
11 (
12     .rst_n    (rst_n    ),
13     .clk_fast (clk_fast ),
14     .clk_slow (clk_slow ),
15     .pulse_in (pulse_in ),
16
17     .pulse_out(pulse_out)
18 );
19
20
21 initial
22 begin
23     clk_fast    = 1;
24     clk_slow    = 0;
25     rst_n       = 0;
26     pulse_in    = 0;
27
28     repeat(5)@(posedge clk_slow);
29     rst_n       = 1;
30     repeat(5)@(posedge clk_fast);
31     pulse_gen();
32
33 end
34
35
36 task pulse_gen;
37 integer i;
38 begin
39     for(i=0;i<10;i=i+1)
40     begin
41         pulse_in <= 1;
42         @(posedge clk_fast);
43     end
44     pulse_in <= 0;
45 end
46 endtask
47
48 always #10 clk_slow = ~clk_slow;
49 always #5  clk_fast = ~clk_fast;
50
51 endmodule

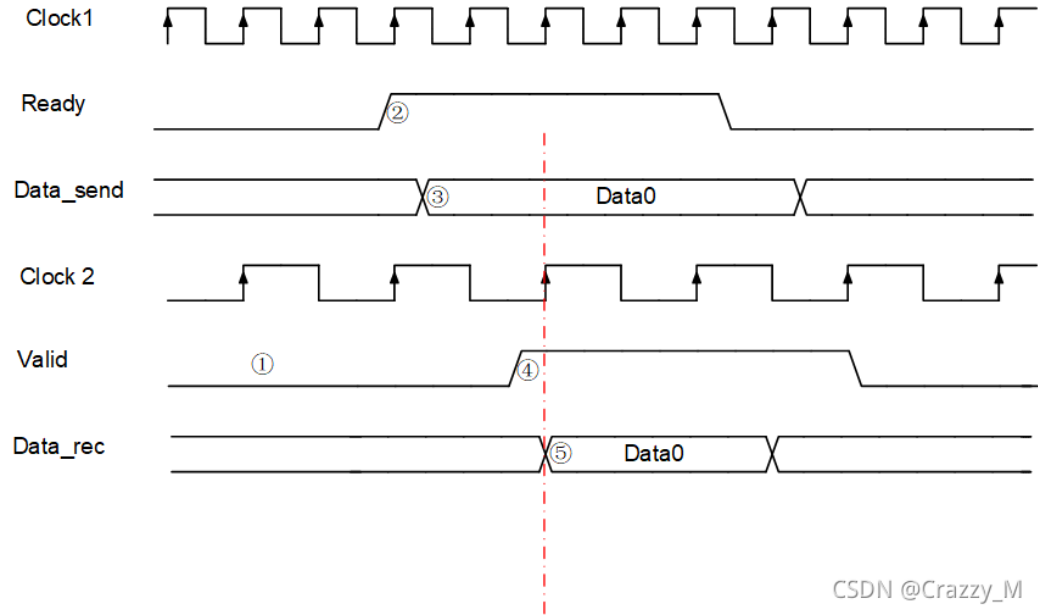
```

1.单字节跨时钟域

跨时钟域多bit时钟传输, 首先我们有两个时钟域的模块, 我们可以认为是一个Master和一个Slave, 模型如下所示 (数据位宽8bit) :
版权: Crazy_M



信号说明
data[7:0]: 数据线。
Ready: Slave准备就绪, 向Master反馈。
Valid: 指示数据线上数据有效, 有效时, 从机可以在数据线上采集数据。



信号	Master	Slave
data[7:0]	154 输出	未经允许, 进制传播 输入
Ready	输入	输出

握手时序如上所示，跟着序号分析思路：

版权：Crazy_M

- ①：clk2时钟下，Slave检测来自Master的Valid信号为低时，输出Ready置1。
- ②：clk1时钟下，Master检测来自Slave的Ready。
- ③：clk1时钟下，Master检测来自Slave的Ready为1时将数据Data0放置数据线。并且将Valid信号置1。
- ④：clk2时钟下，Slave检测来自Master的Valid信号。
- ⑤：clk2时钟下，Slave检测来自Master的Valid信号为1时，在数据线上采集数据。

从上面红线可以看到，我们在clk2时钟上升沿采集数据是在数据Data_send的稳定时刻。

握手信号在时钟域的传输中又涉及到了单bit信号跨时钟域的问题：

m_ready_rr：是s_clk时钟下的ready经过m_clk打两拍得到的。

s_valid_rr：是m_clk时钟下的valid经过s_clk打两拍得到的。

因为valid和ready是相互制约的，也就是接收到valid，ready才会变化，接收到ready，valid才会变化，所以单bit的ready、valid信号是长脉冲信号，直接经过两级触发器打拍可以降低亚稳态。

上代码：

```
1
2 //通过握手主机发送1个字节的数据至从机
3 module top
4 (
5
6     input    m_rst_n,          //主机复位
7     input    m_clk,           //主机时钟
8     input    m_ready,         //表示从机接收数据已经准备就绪
9     output   m_valid,         //主机发送数据有效
10    output[7:0] m_data,
11
12    input    s_rst_n,          //从机复位
13    input    s_clk,           //从机时钟
14    input    s_valid,         //接收到有效的数据
15    output   s_ready,         //准备好接收数据
16    input[7:0] s_data,
17    output[7:0] data
18
19 );
20 //Master,产生数据,产生valid信号
21 reg[7:0] m_data_r;
22 reg m_valid_r;              //Master输出valid
23 reg m_ready_r;              //来自Slave的ready信号打一拍
24 reg m_ready_rr;             //来自Slave的ready信号打两拍
25
26 assign m_data = m_data_r;
27 assign m_valid = m_valid_r;
28 always@(posedge m_clk or negedge m_rst_n)
29 begin
30     if(!m_rst_n)
31     begin
32         m_data_r <= 8'h00;
33         m_valid_r <= 1'b0;
34         m_ready_r <= 1'b0;
```

```

35     m_ready_rr  <= 1'b0;
36 end
37 else
38     begin//接收到从机准备就绪信号，开始数据发送    版权：Crazyy_M
39         {m_ready_rr,m_ready_r} <= {m_ready_r,m_ready};           //异步过来的信号打2拍降低亚稳态
40         m_data_r  <= (m_ready_rr == 1'b1)? 8'h55:8'h00;           //从机就绪时，数据放置总线
41         m_valid_r  <= (m_ready_rr == 1'b1)?1'b1:1'b0;           //从机就绪时，指示主机放置总线数据有效
42     end
43 end
44
45 //Slave,接收数据，产生Ready信号
46 reg      s_ready_r;           //Slave输出ready
47 reg      s_valid_r;           //来自Master的valid信号打一拍
48 reg      s_valid_rr;          //来自Master的valid信号打两拍
49 reg[7:0] data_r;
50
51 assign    s_ready   = s_ready_r;
52 assign    data      = data_r;
53 always@(posedge s_clk or negedge s_rst_n)
54 begin
55     if(!s_rst_n)
56     begin
57         s_ready_r  <= 1'b0;
58         s_valid_r   <= 1'b0;
59         s_valid_rr  <= 1'b0;
60         data_r      <= 8'h00;
61     end
62     end
63     else
64     begin//在检测到主机数据无效的时候，准备就绪接收数据
65         {s_valid_rr,s_valid_r} <= {s_valid_r,s_valid};//异步过来的信号打2拍降低亚稳态
66         s_ready_r  <= (s_valid_rr == 1'b0)?1'b1:1'b0;
67         data_r      <= (s_valid_rr == 1'b1)?s_data:8'h00;
68     end
69 end
70
71 endmodule

```

testbench:

```

1  module top_test;
2
3  reg      m_rst_n;
4  reg      s_rst_n;
5  reg      m_clk;
6  wire     s_ready;
7  wire[7:0] m_data;
8
9  reg      s_clk ;
10 wire     m_valid;
11 wire[7:0] data;
12
13 top u1
14 (
15     .m_rst_n(m_rst_n),

```

```

16 .m_clk (m_clk ),
17 .m_ready(s_ready),
18 .m_valid(m_valid),
19 .m_data(m_data),
20
21 .s_rst_n(s_rst_n),
22 .s_clk (s_clk) ,
23 .s_valid(m_valid),
24 .s_ready(s_ready),
25 .s_data(m_data),
26
27 .data (data)
28 );
29
30 initial
31 begin
32     m_clk  = 0;
33     s_clk  = 0;
34     m_rst_n = 0;
35     s_rst_n = 0;
36
37     #20;
38     m_rst_n = 1;
39     s_rst_n = 1;
40 end
41
42 always #5    m_clk = ~m_clk;
43 always #10   s_clk = ~s_clk;
44
45 endmodule
46

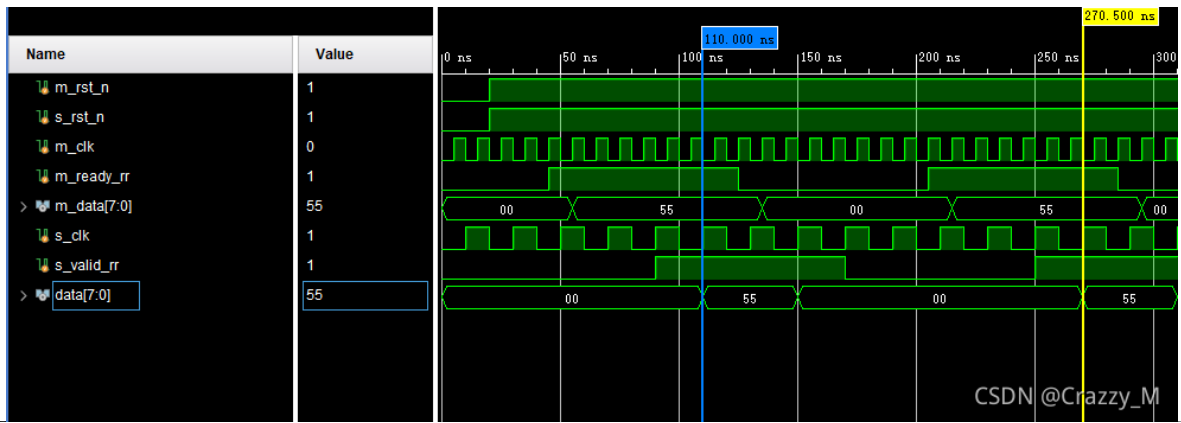
```

波形：

m_ready_rr：是s_clk时钟下的ready经过m_clk打两拍得到的。

s_valid_rr：是m_clk时钟下的valid经过s_clk打两拍得到的。

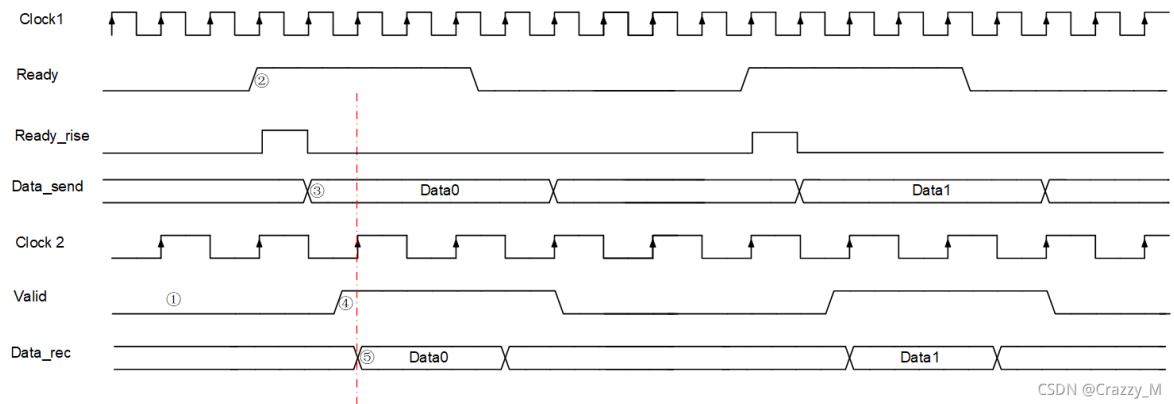
因为valid和ready是相互制约的，也就是接收到valid，ready才会变化，接收到ready，valid才会变化，所以单bit的ready、valid信号是长脉冲信号，直接经过两级触发器打拍降低亚稳态。



观察发现，s_clk时钟采集数据是在m_data的数据稳定位置，数据传输正常。
版权：Crazyy_M

2.多字节跨时钟域

在Ready上升沿的时候切换传输数据，其他和上面一样，其实握手可以很灵活设计，这只是一种方法。



CSDN @Crazyy_M

代码：

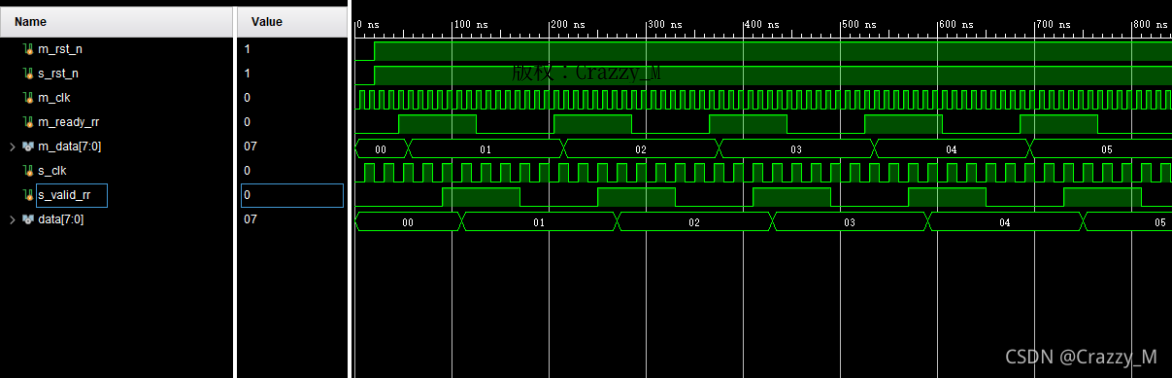
```
1 //通过握手主机发送多个字节的数据至从机
2 module top
3 (
4     input    m_rst_n,           //主机复位
5     input    m_clk,            //主机时钟
6     input    m_ready,          //表示从机接收数据已经准备就绪
7     output   m_valid,          //主机发送数据有效
8     output[7:0] m_data,
9
10
11     input    s_rst_n,          //从机复位
12     input    s_clk,            //从机时钟
13     input    s_valid,          //接收到有效的数据
14     output   s_ready,          //准备好接收数据
15     input[7:0] s_data,
16     output[7:0] data
17
18 );
19
20 reg[7:0] m_data_r;
21 reg m_valid_r;
22 reg m_ready_r;
23 reg m_ready_rr;
24 reg m_ready_rrr;
25 wire m_ready_rise;
26
27 //Master,产生数据,产生valid信号,检测m_ready的上升沿,进行数据的切换
28 assign m_data = m_data_r;
```

```

29 assign      m_valid = m_valid_r;
30
31 assign      m_ready_rise = (~m_ready_rrr)&m_ready_rr;
32                                     版权：Crazy_M
33 always@(posedge m_clk or negedge m_rst_n)
34 begin
35     if(!m_rst_n)
36     begin
37         m_data_r    <= 8'h00;
38         m_valid_r    <= 1'b0;
39         m_ready_r    <= 1'b0;
40         m_ready_rr   <= 1'b0;
41         m_ready_rrr  <= 1'b0;
42     end
43     else
44     begin//接收到从机准备就绪信号，开始数据发送
45         {m_ready_rrr,m_ready_rr,m_ready_r} <= {m_ready_rr,m_ready_r,m_ready}; //异步过来的信号打2拍降低亚稳态，第三拍
46 用来检测上升沿
47         m_data_r    <= (m_ready_rise == 1'b1)? m_data_r+1:m_data_r; //从机就绪时，数据放置总线
48         m_valid_r    <= (m_ready_rr == 1'b1)?1'b1:1'b0; //从机就绪时，指示主机放置总线数据有效
49     end
50 end
51
52
53 reg         s_ready_r;
54 reg         s_valid_r;
55 reg         s_valid_rr;
56 reg[7:0]    data_r;
57 //Slave,接收数据，产生Ready信号
58 assign      s_ready   = s_ready_r;
59 assign      data      = data_r;
60 always@(posedge s_clk or negedge s_rst_n)
61 begin
62     if(!s_rst_n)
63     begin
64         s_ready_r    <= 1'b0;
65         s_valid_r    <= 1'b0;
66         s_valid_rr   <= 1'b0;
67         data_r       <= 8'h00;
68     end
69     else
70     begin//在检测到主机数据无效的时候，准备就绪接收数据
71         {s_valid_rr,s_valid_r} <= {s_valid_rr,s_valid}; //异步过来的信号打2拍降低亚稳态
72         s_ready_r    <= (s_valid_rr == 1'b0)?1'b1:1'b0;
73         data_r       <= (s_valid_rr == 1'b1)?s_data:data_r;
74     end
75 end
76 end
77
78 endmodule

```

```
2
3 reg      m_rst_n;
4 reg      s_rst_n;
5 reg      m_clk;
6 wire      s_ready;
7 wire[7:0] m_data;
8
9 reg      s_clk ;
10 wire     m_valid;
11 wire[7:0] data;
12
13 top u1
14 (
15     .m_rst_n(m_rst_n),
16     .m_clk  (m_clk  ),
17     .m_ready(s_ready),
18     .m_valid(m_valid),
19     .m_data(m_data),
20
21     .s_rst_n(s_rst_n),
22     .s_clk  (s_clk) ,
23     .s_valid(m_valid),
24     .s_ready(s_ready),
25     .s_data(m_data),
26
27     .data  (data)
28 );
29
30 initial
31 begin
32     m_clk  = 0;
33     s_clk  = 0;
34     m_rst_n = 0;
35     s_rst_n = 0;
36
37     #20;
38     m_rst_n = 1;
39     s_rst_n = 1;
40 end
41
42 always #5    m_clk = ~m_clk;
43 always #10   s_clk = ~s_clk;
44
45 endmodule
46
```



三十四、Verilog实现glitch free（无毛刺）时钟切换电路

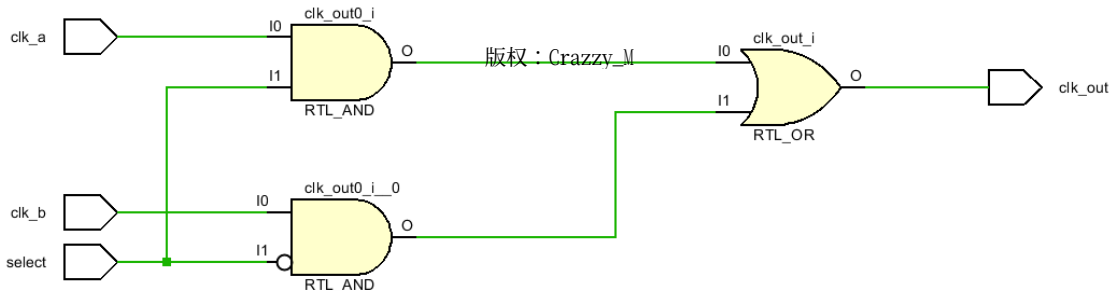
时钟切换逻辑避免产生glitch，应当先关闭当前时钟，再打开目标时钟。不管是在关闭还是使能，都必须保证当前时钟或目标的使能信号的跳变都分别发生在时钟为低电平期间进行的，防止时钟产生glitch。

输入两个时钟clk_a、clk_b。
select: 1:输出clk_a; 0:输出clk_b。



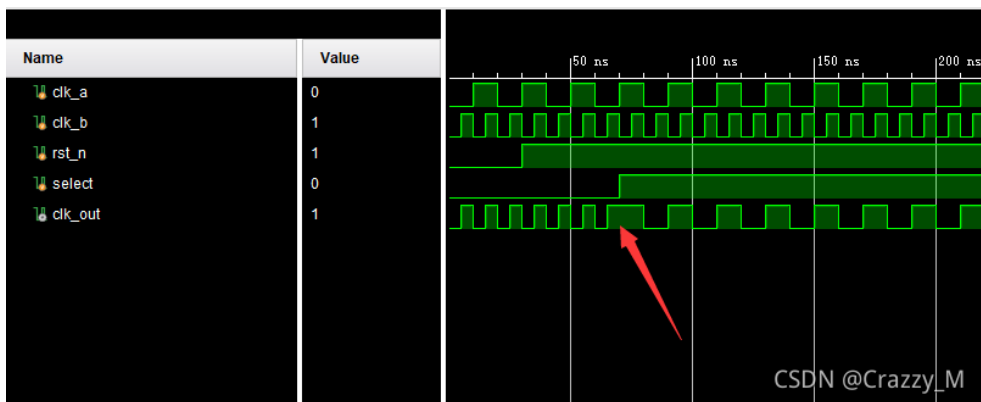
首先最先想到的就是通过组合逻辑直接输出clk_out。

```
1 | assign clk_out = (clk_a&select)|(clk_b&(~select));
```



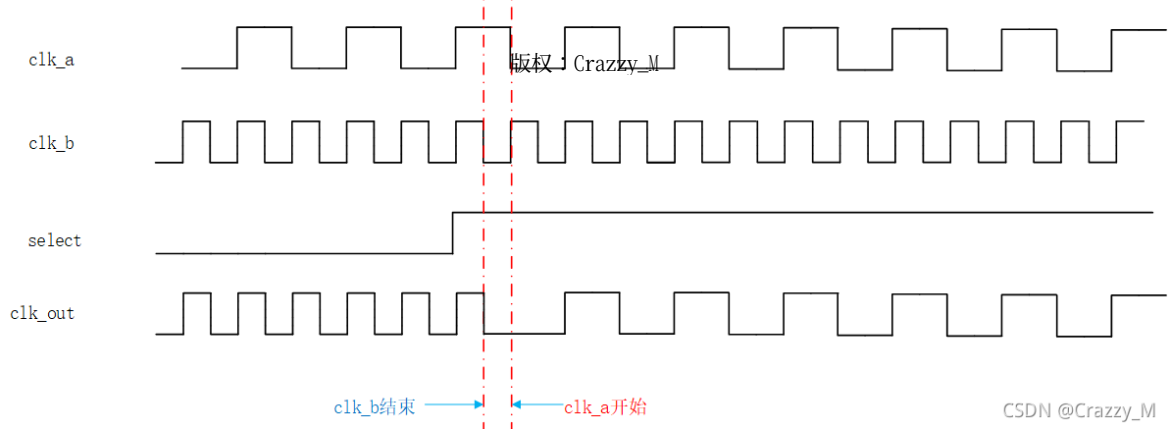
CSDN @Crazy_M

select在时钟的高电平器件进行了时钟的切换，这样在时钟切换处会出现与clk_a、clk_b不相关的脉冲，认为出现了毛刺（出现了斩波），这样时钟切换是不安全的，后续电路是不能直接使用的。

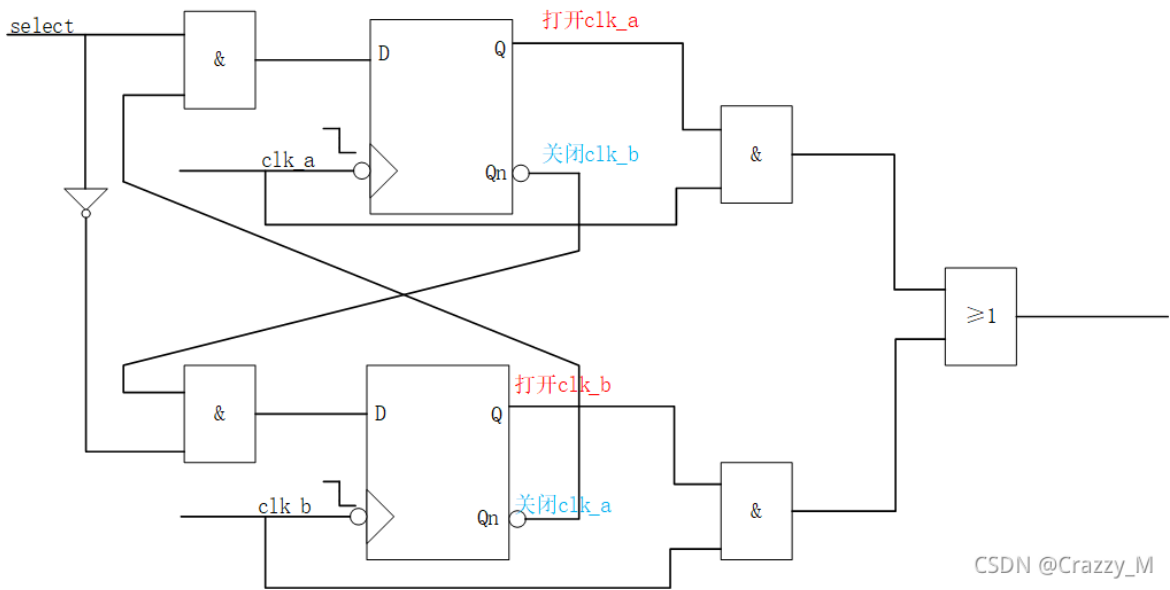


1.两个相关时钟源（频率成倍数关系）切换

为了防止毛刺的产生，控制信号只能在时钟低电平时来切换时钟。



所以我们插入下降沿有效的D触发器来实现时钟低电平时完成时钟切换。（在关闭一个时钟之后再打开另一个时钟）



```

1 module top
2 (
3     input  clk_a,
4     input  clk_b,
5     input  rst_n,
6     input  select,
7
8     output clk_out
9
10 );

```

```

11
12 reg out_a;//用于打开clk_a输出
13 reg out_b;//用于打开clk_b输出
14                                     版权：Crazy_M
15 assign clk_out = (out_a & clk_a) | (out_b & clk_b);
16 //out_a、out_b初始值均为0，select初始值为0,out_b置1,输出clk_b。
17 //select置1时，out_b置0,out_a置1
18
19 always@(negedge clk_a or negedge rst_n)
20 begin
21     if(!rst_n)
22         out_a <= 1'b0;
23     else
24         out_a <= (~out_b) & select;
25 end
26
27
28 always@(negedge clk_b or negedge rst_n)
29 begin
30     if(!rst_n)
31         out_b <= 1'b0;
32     else
33         out_b <= (~out_a) & (~select);
34 end
35
36
37 endmodule

```

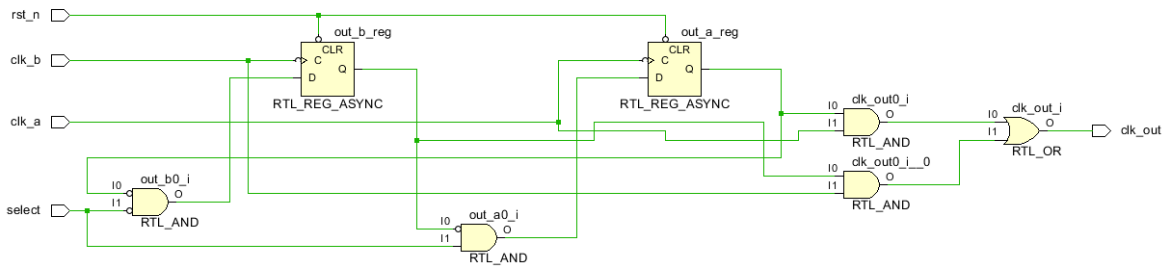
testbench:

```

1 module top_test;
2
3 reg    clk_a;
4 reg    clk_b;
5 reg    rst_n;
6 reg    select;
7 wire    clk_out;
8
9 top u1
10 (
11     .clk_a(clk_a),
12     .clk_b(clk_b),
13     .rst_n(rst_n),
14     .select(select),
15
16     .clk_out(clk_out)
17
18 );
19
20 initial
21 begin
22     rst_n = 0;
23     select = 0;
24     clk_a = 0;
25     clk_b = 1;

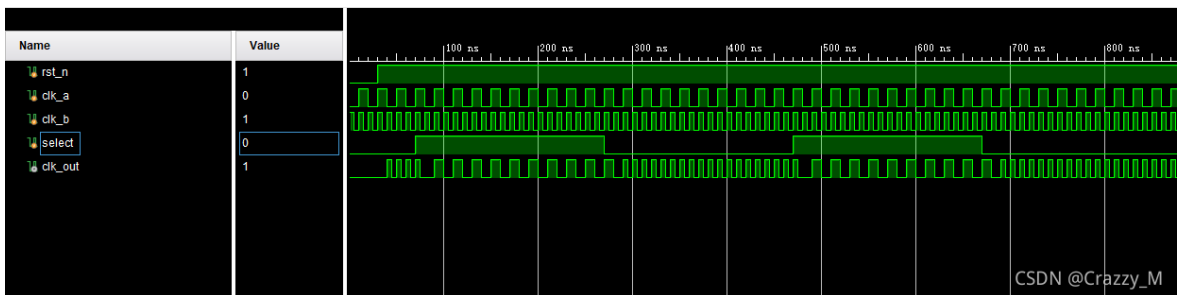
```

RTL:

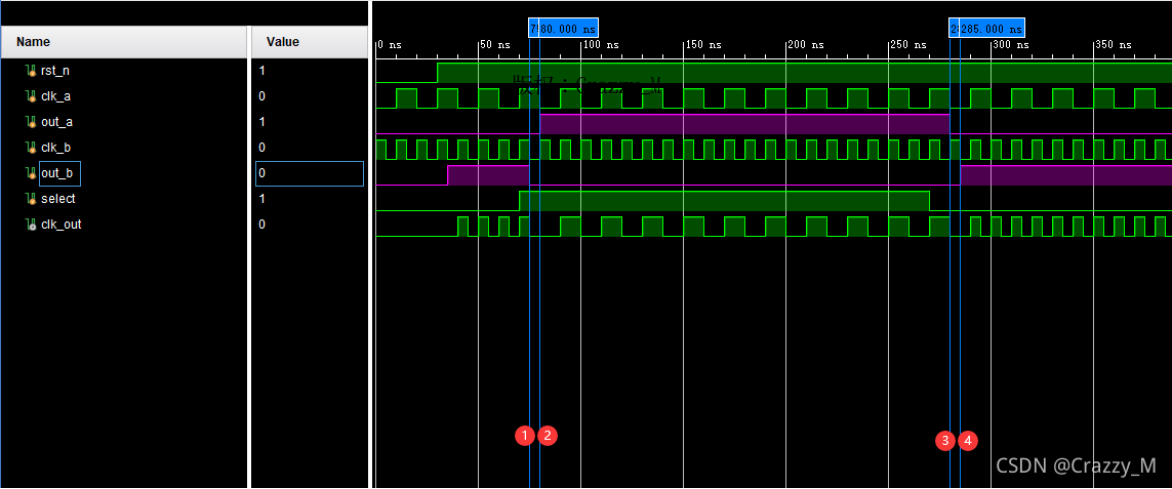


CSDN @Crazy_M

波形:



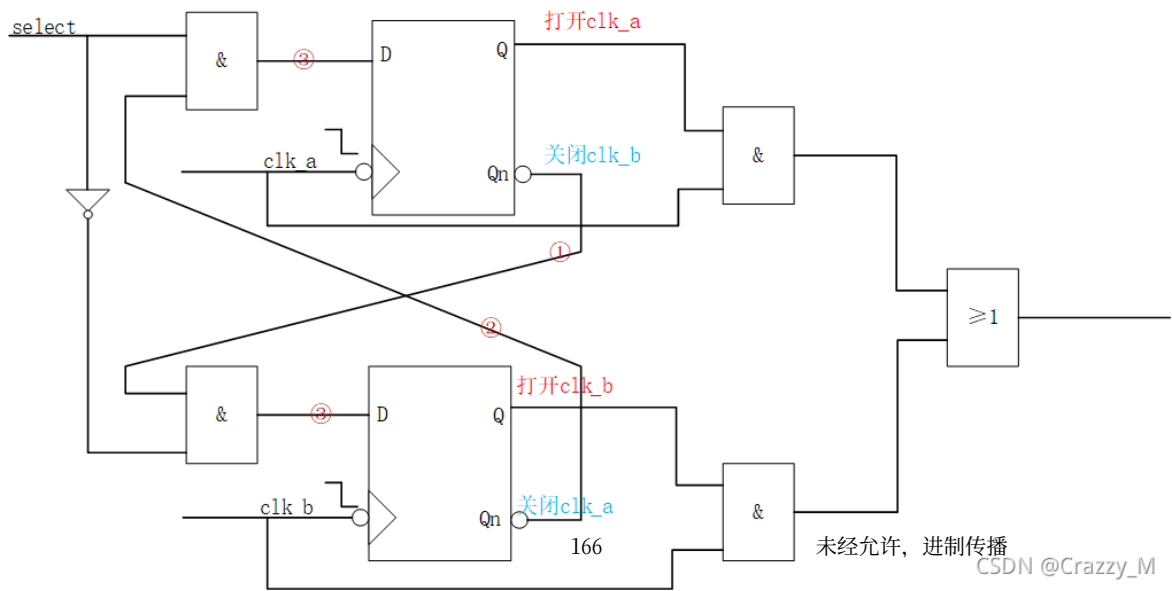
详细波形:



其实就是通过时钟的下降沿和select产生了新的控制信号out_a、out_b，该控制信号是在时钟的低电平有效，这样防止产生毛刺（波形斩断）。

- 在光标①处：clk_b的下降沿结束clk_b时钟的输出。
- 在光标②处：clk_a的下降沿开始clk_b时钟的输出。
- 在光标③处：clk_a的下降沿结束clk_a时钟的输出。
- 在光标④处：clk_b的下降沿开始clk_b时钟的输出。

在下图的设计中，路径①②③上的数据如果在时钟有效沿捕捉时发生变化就可能产生亚稳态。因为clk_a、clk_b是相关的时钟，我们通过约束就可以降低亚稳态的产生。

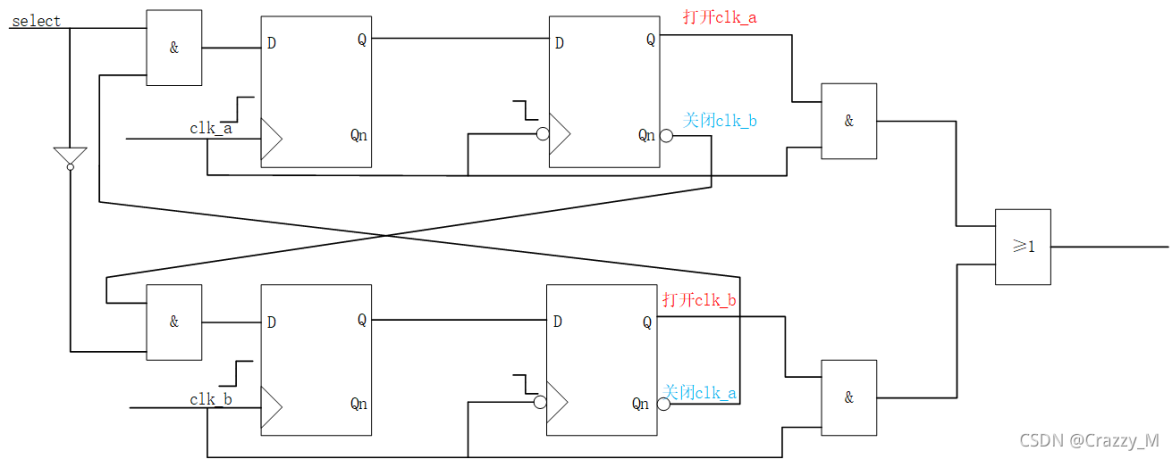


但如果是异步时钟呢？

2.异步时钟切换

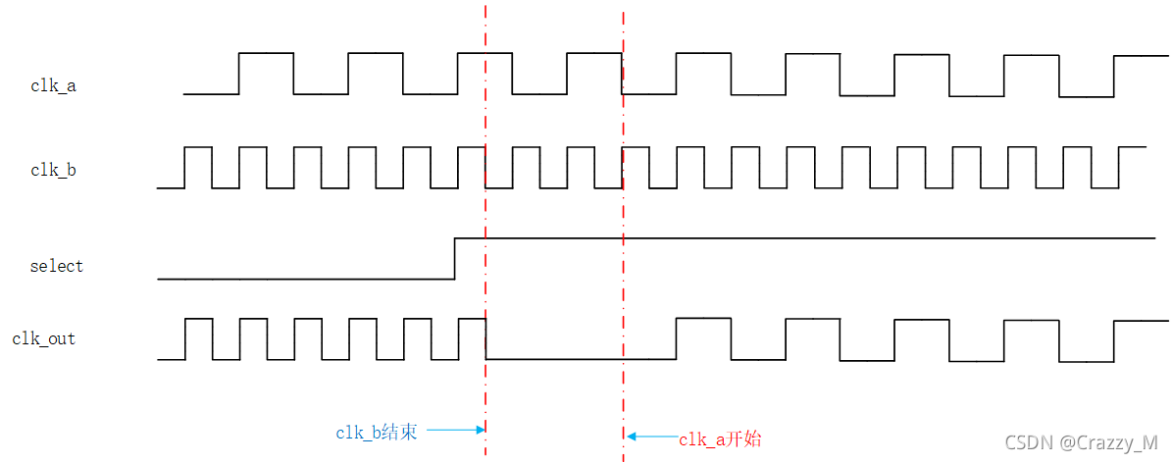
版权：Crazy_M

在异步时钟域，上图中①②③上的数据路径上很容易出现亚稳态的现象，所以在两个D触发器前面加一级上升沿触发器，这样就降低了来自异步时钟反馈信号的亚稳态现象，如下所示：



CSDN @Crazy_M

也就是，根据select和clk_a、clk_b的关系，产生新的控制信号（out_a_r、out_b_r），该控制信号在时钟的低电平作用。



CSDN @Crazy_M

代码：

```
1 //异步时钟
2 module top
3 (
4     input  clk_a,
```

```

5     input   clk_b,
6     input   rst_n,
7     input   select,
8
9     output  clk_out
10
11 );
12 reg out_a;//二级寄存器输出
13 reg out_b;//一级寄存器输出
14 reg out_a_r;//二级寄存器输出, 用于打开clk_a输出
15 reg out_b_r;//二级寄存器输出, 用于打开clk_b输出
16
17
18 assign  clk_out = (out_a_r & clk_a) | (out_b_r & clk_b);
19
20
21 always@(posedge clk_a or negedge rst_n)
22 begin
23     if(!rst_n)
24         out_a <= 1'b0;
25     else
26         out_a <= (~out_b_r) & select;
27 end
28 always@(negedge clk_a or negedge rst_n)
29 begin
30     if(!rst_n)
31         out_a_r <= 1'b0;
32     else
33         out_a_r <= out_a;
34 end
35
36 always@(posedge clk_b or negedge rst_n)
37 begin
38     if(!rst_n)
39         out_b <= 1'b0;
40     else
41         out_b <= (~out_a_r) & (~select);
42 end
43 always@(negedge clk_b or negedge rst_n)
44 begin
45     if(!rst_n)
46         out_b_r <= 1'b0;
47     else
48         out_b_r <= out_b;
49 end
50
51 endmodule

```

testbench:

```

1 module top_test;
2
3 reg    clk_a;
4 reg    clk_b;
5 reg    rst_n;

```

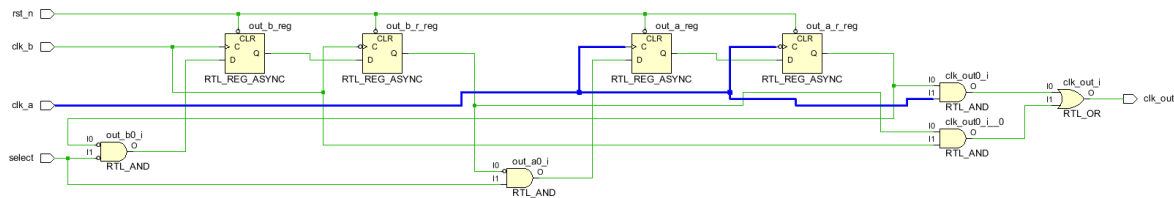
```

6 reg select;
7 wire clk_out;
8
9
10
11 top u1
12 (
13     .clk_a(clk_a),
14     .clk_b(clk_b),
15     .rst_n(rst_n),
16     .select(select),
17
18     .clk_out(clk_out)
19 );
20
21 initial
22 begin
23     rst_n = 0;
24     select = 0;
25     clk_a = 0;
26     clk_b = 1;
27     repeat(2)@(posedge clk_a);
28     rst_n = 1;
29
30     repeat(2)@(posedge clk_a);
31     select = 1;
32     repeat(10)@(posedge clk_a);
33     select = 0;
34     repeat(10)@(posedge clk_a);
35     select = 1;
36     repeat(10)@(posedge clk_a);
37     select = 0;
38 end
39
40 always #10 clk_a = ~clk_a;
41 always #5 clk_b = ~clk_b;
42
43 endmodule

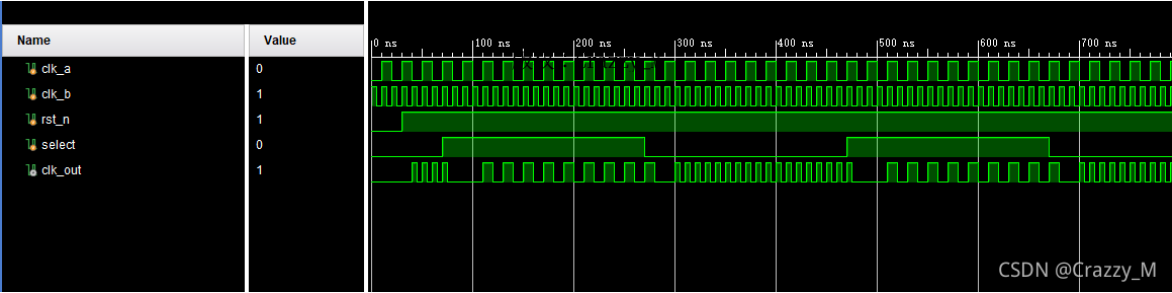
```

版权：Crazyzy_M

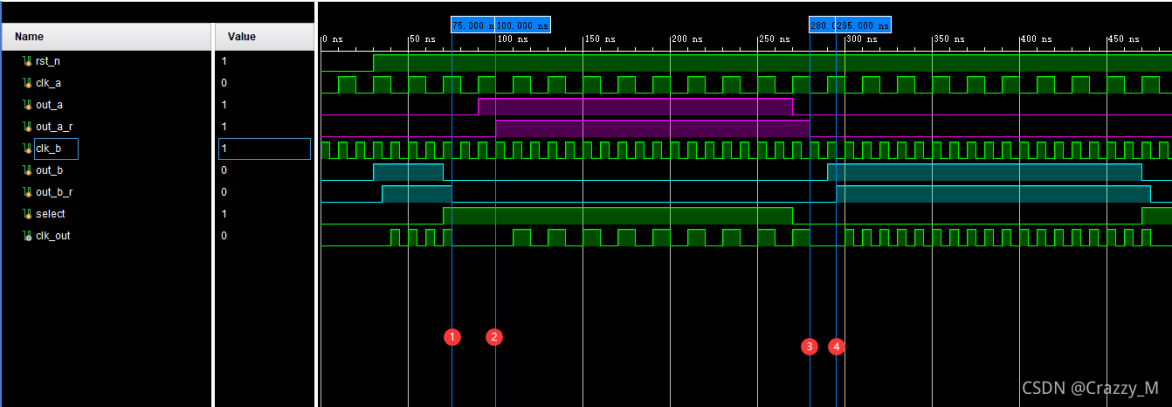
RTL:



波形:



详细波形：



在光标①处：clk_b的下降沿结束clk_b时钟的输出。
在光标②处：clk_a的下降沿开始clk_b时钟的输出。
在光标③处：clk_a的下降沿结束clk_a时钟的输出。
在光标④处：clk_b的下降沿开始clk_b时钟的输出。

三十五、Verilog实现8bit环形计数器

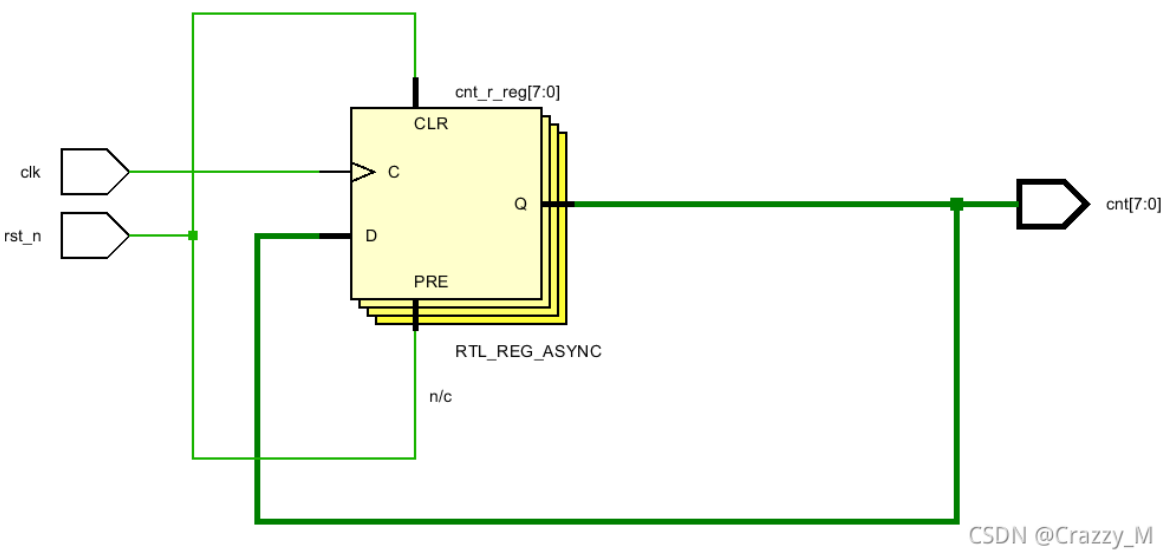
复位有效时输出0000_0001，释放后依次输出0000_0010，0000_0100，0000_1000。。。

```
1 module top
2 (
3     input      clk,
4     input      rst_n,
5
6     output[7:0] cnt
7
8 );
9 reg[7:0] cnt_r;
10
11 assign cnt = cnt_r;
12 always@(posedge clk or negedge rst_n)
```

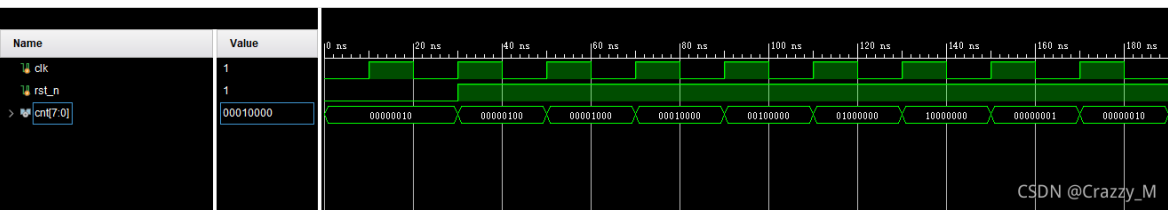
```
14 begin
15     if(!rst_n)
16         cnt_r <= 8'b0010;
17     else
18         cnt_r <= {cnt_r[7:0],cnt_r[7]};
19 end
20
21 endmodule
```

版权：Crazy_M

RTL:



波形:



三十六、Verilog初始化一个ROM（用于仿真）

本地中新建一个txt文件，通过 \$readmemb 或 \$readmemh 读取文件存放至mem存储单元。

1.二进制：

文件内容：

0001
0010
0011
0100
0101
0110
0111
1000

版权：Crazy_M

```

1 module top_test;
2
3 reg[3:0] memory[7:0]; //申请八个四位的存储单元
4 reg[4:0] i;
5
6 initial
7 begin
8     $readmemb("E:/FPGA_xilinx/Xilinx_Project/UltraScale/rom/rom.srcs/sim_1/new/file.txt",memory); //读取file.tx
9     t中的数字到memory
10    for(i=0;i<=7;i=i+1)
11        $display("%b",memory[i]);
12 end
13
endmodule

```

显示窗口：

```

# } else {
#     send_msg_id Add_Wave-1 WARNING "No top level sigr
# }
# }
# run 1000ns
0001
0010
0011
0100
0101
0110
0111
1000

```

CSDN @Crazy_M

2.十六进制

文件内容：

0002
0040
0600
8000
000a
00c0
0e00
febc

版权：Crazy_M

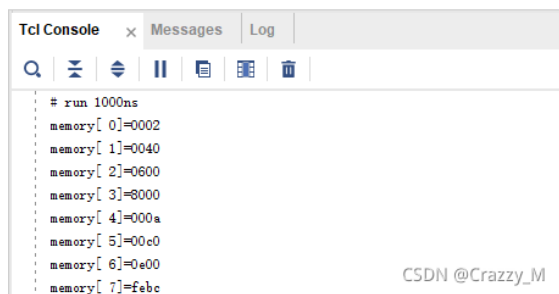
代码：

```

1 module top_test;
2
3 reg[15:0] memory[7:0]; //申请八个四位的存储单元
4 reg[4:0] i;
5
6 initial
7 begin
8     $readmemh("E:/FPGA_xilinx/Xilinx_Project/UltraScale/rom/rom.srcs/sim_1/new/file.txt",memory); //读取file.tx
9     t中的数字到memory
10    for(i=0;i<=7;i=i+1)
11        $display("memory[%d]=%h",i,memory[i]);
12 end
13
14 endmodule

```

显示窗口：



```

# run 1000ns
memory[ 0]=0002
memory[ 1]=0040
memory[ 2]=0600
memory[ 3]=8000
memory[ 4]=000a
memory[ 5]=00c0
memory[ 6]=0e00
memory[ 7]=febc

```

CSDN @Crazy_M

三十七、数值（有无符号）运算、比较

1.原码、反码、补码

1.原码：

有符号数最高为表示符号位（0正，1负），其余位表示真值，8bit表示如下

未经允许，进制传播

±	D6	D5	D4	D3 版权：Crazzy_M	D2	D1	D0
---	----	----	----	-------------------	----	----	----

8bit无符号数表示范围[0~255]。

8bit有符号数表示范围[-128~127]。

+3原码：

0000_0011

-3原码：

1000_0011

2.反码：
正数的反码是其本身

+3原码：

0000_0011

+3反码：

0000_0011

负数的反码是在其原码的基础上符号位不变，其余为取反。

-3原码：

1000_0011

-3反码：

1111_1100

3.补码：（计算机中二进制数都数都是按补码存储的）
正数的补码是其本身。

+3原码：

0000_0011

+3补码：

负数的补码是在原码的基础上，符号位不变，其余取反再+1（其实就是反码+1）。

-3原码：

1000_0011

-3反码：

1111_1101

计算机中二进制数为什么要用补码来表示呢？

在计算机中只有加法器，没有减法器，因此减法是通过加法器来计算的，比如3-3是通过3+(-3)这样计算的。

```
1 //3的原码
2 0000_0011
3 //3的补码（即-3在计算机中的表示形式）
4 1111_1101
5
6 //则3+（-3）
7 0000_0011
8 +1111_1101
9 -----
10 10000_0000
11 //因为是八位数，所以为0000_0000就为0
12
```

负数在计算机中是补码表示！补码表示！补码表示！

-3表示为1111_1101（补码），不是1000_0011（原码）

```
1 data_a = 3;
2 data_b = -3;
```

Name	Value	0 ps	1 ps	2 ps	3 ps	4 ps
> data_a[7:0]	00000011				00000011	
> data_b[7:0]	11111101				11111101	

先判断是否异号？

异号

则符号位为0（正数）的数 > 符号位为1的数（负数）。 版权：Crazy_M

同号

则判断符号位为1（负数）还是0（正数）

符号位为1（负数）：转换为正数比较，哪个转换完的正数小，则原值大。

符号位为0（正数）：直接比较。

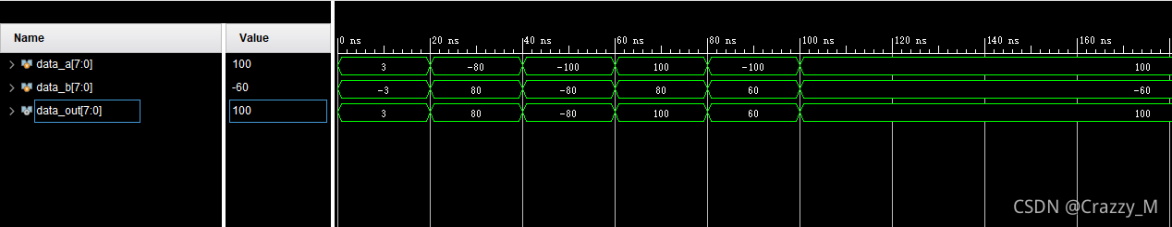
```
1 //有符号数比较
2 module top
3 #
4 (
5     parameter    DATA_WIDTH = 8
6 )
7 (
8
9     input    [DATA_WIDTH-1:0]    data_a,
10    input    [DATA_WIDTH-1:0]    data_b,
11
12    output    [DATA_WIDTH-1:0]    data_out
13 );
14
15 assign data_out = max_data(data_a,data_b);
16
17 function [DATA_WIDTH-1:0]max_data;
18 input    [DATA_WIDTH-1:0]    compare_a;
19 input    [DATA_WIDTH-1:0]    compare_b;
20 reg                                compare_flag;        //1:max_data=compare_a,    0:max_data=compare_b
21 begin
22 //比较符号位
23     if(compare_a[DATA_WIDTH-1] != compare_b[DATA_WIDTH-1])//异号
24     begin
25         compare_flag = (compare_a[DATA_WIDTH-1] > compare_b[DATA_WIDTH-1])?1'b0:1'b1;
26     end
27     else                                //同号(分正负判断)
28     begin
29         compare_flag = (compare_a[DATA_WIDTH-1])?
30             ((-compare_a[DATA_WIDTH-1:0]) < (-compare_b[DATA_WIDTH-1:0]))://负数,化为负数比较
31             (compare_a[DATA_WIDTH-1:0] > compare_b[DATA_WIDTH-1:0]);        //正数
32     end
33
34     max_data = compare_flag?compare_a:compare_b;
35 end
36
37 endfunction
38
39 endmodule
```

或者直接用 \$signed

```
1 module top
2 #
3 (
```

```
4     parameter    DATA_WIDTH = 8
5 )
6 (
7     // 版权：Crazyzy_M
8     input    [DATA_WIDTH-1:0]    data_a,
9     input    [DATA_WIDTH-1:0]    data_b,
10
11     output   [DATA_WIDTH-1:0]    data_out
12 );
13 assign data_out = ($signed(data_a) >= $signed(data_b))?data_a:data_b;
14
15
16 endmodule
```

结果展示：



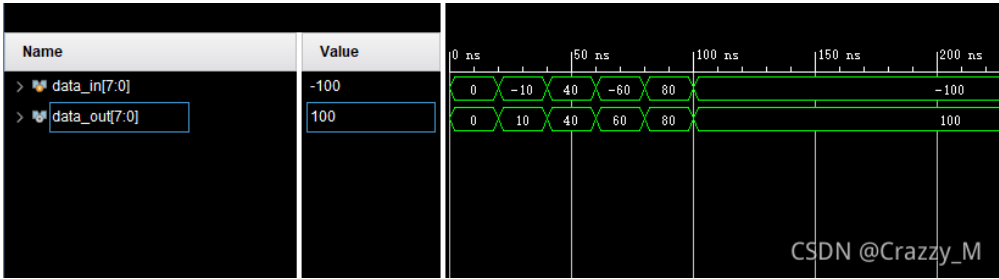
3.有符号数求绝对值

- 思路：
- 先判断符号
 - 0：原值。
 - 1：变负为正。

```
1 module top
2 #
3 (
4     parameter    DATA_WIDTH = 8
5 )
6 (
7
8     input    [DATA_WIDTH-1:0]    data_in,
9     output   [DATA_WIDTH-1:0]    data_out
10 );
11
12 assign data_out = data_abs(data_in);
13
14 function [DATA_WIDTH-1:0]data_abs;
15 input    [DATA_WIDTH-1:0]Signed_data_in;
16
17 data_abs = (Signed_data_in[DATA_WIDTH-1])?(-Signed_data_in):(Signed_data_in);
18
19 endfunction
20
```

结果展示:

版权：Crazyy_M

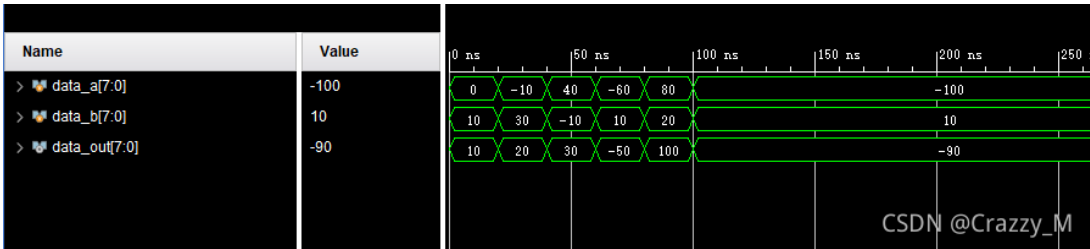


4.有符号数加法器

符号为进行扩展

```
1 module top
2 #
3 (
4     parameter    DATA_WIDTH = 8
5 )
6 (
7
8     input    [DATA_WIDTH-1:0]    data_a,
9     input    [DATA_WIDTH-1:0]    data_b,
10
11     output   [DATA_WIDTH-1:0]    data_out
12 );
13
14 assign data_out = {data_a[DATA_WIDTH-1],data_a}+{data_b[DATA_WIDTH-1],data_b};
15
16 endmodule
17
```

结果展示:



  &数据产生固定位数校验码的一种编码技术。先简单介绍下原理：要发送的帧后面附加一个数，要使所生成的新帧能与发送端和接收端共同选定的整数整除，也就是余数为0（采用的模2除法）。

1.CRC原理介绍

版权：Crazyy_M

CRC用在数据传输的时候，比如我们发送的数据为DATA，发送端为TX，接收端为RX。

如果我们对发送的数据不做任何的处理，TX发送出去的数据就为DATA。

TX：



但是因为传输信道、线路等影响下出错总是存在的，我们怎么知道RX接收端接收的数据就一定是DATA呢。

RX：



CRC（循环冗余校验）就是用来检测或者校验传输时是否发生错误，实质就是在数据的末尾加n bit的CRC校验位，如以下格式，别着急，继续往下看。



接下来看看原理（用二进制简单的例子先理解CRC干了个啥，重点看原理）：
我们先要知道模2除法
模2除法每一位除的结果都不影响其他位，既不向上借位。例如：

1011

1001

1001000

1011

0001000

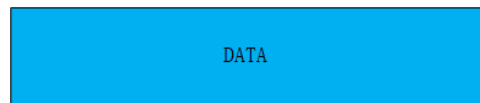
1011

0011

SDN @ Crazyy_M

计算机中都是二进制传输比如我发送的数据为1010，在接收端我们怎么检测到接收到的数据就是1010呢？
CRC的原理就是在发送方和接收方规定一个规则，比如发送方的数据（加上CRC后的数据）能被一个数整数（模2除法），传输到接收方数据（包含CRC的数据）同样可以被那个数整除，如果整除不了，这说明数据发生了错误。

TX: 发送的数据为 DATA = 1010。



DATA	CRC
------	-----

所以第一步我们先将数据宽展为{DATA,n(0)}, 这个n怎么确定呢?



可以观察下面的模2除法:

被除数为{DATA,CRC_初始值} = {1010,000} = 1010_000。

$$\begin{array}{r}
 \\
 \\
 \hline
 10111 \overline{) 1 } \\
 \underline{1 } } \\
 \\
 \\
 \hline
 \\
 \\
 \hline

 \end{array}$$

所以CRC就为011。

我们传输的时候的数据就为 1010_011。看这个时候的数据去除以我们的规则1011，是不是余数就为0。同样接收端接收到数据同样除以规则1011检查余数是否为0，如果不为0则说明数据出现了错误。

$$\begin{array}{r}
 1011 \overline{) 1010011} \\
 \underline{1011} \\
 0001011 \\
 \underline{1011} \\
 0001011 \\
 \underline{1011} \\
 000000
 \end{array}$$

什么??? 让FPGA去做除法???

模2除法其实就是数据的异或

我们的规则每次跟异或结果的最左边的1对齐进行异或，直到异或的结果位数小于我们定义规则数的位数，这样就求得了余数，自己动手练练。

$$\begin{array}{r}
 1010000 \\
 \underline{1011} \\
 0001000 \\
 \underline{1011} \\
 0011
 \end{array}$$

以上是为了了解CRC的原理，上面表示的名词非专业名词，具体看下面。

2.CRC简介

上面我们所说的规则，专业名词叫做生成多项式，CRC的生成多项式有多种，如下所示（只看CRC算法名称、多项式公式、多项式这三栏，（宽度这一栏指的时候CRC的位宽，和多项式的最高次幂相同））。

常见CRC参数模型如下:

CRC算法名称	多项式公式	宽度	多项式	初始值	结果异或值	输入反转	输出反转
CRC-4/ITU	$x^4 + x + 1$	4	03	00	00	true	true
CRC-5/EPC	$x^5 + x^3 + 1$	5	09	09	00	false	false
CRC-5/ITU	$x^5 + x^4 + x^2 + 1$	5	15	00	00	true	true
CRC-5/USB	$x^5 + x^2 + 1$	5	05	1F	1F	true	true
CRC-6/ITU	$x^6 + x + 1$	6	03	00	00	true	true
CRC-7/MMC	$x^7 + x^3 + 1$	7	09	00	00	false	false
CRC-8	$x^8 + x^2 + x + 1$	8	07	00	00	false	false
CRC-8/ITU	$x^8 + x^2 + x + 1$	8	07	00	55	false	false
CRC-8/ROHC	$x^8 + x^2 + x + 1$	8	07	FF	00	true	true
CRC-8/MAXIM	$x^8 + x^5 + x^4 + 1$	8	31	00	00	true	true
CRC-16/IBM	$x^{16} + x^{15} + x^2 + 1$	16	8005	0000	0000	true	true
CRC-16/MAXIM	$x^{16} + x^{15} + x^2 + 1$	16	8005	0000	FFFF	true	true
CRC-16/USB	$x^{16} + x^{15} + x^2 + 1$	16	8005	FFFF	FFFF	true	true
CRC-16/MODBUS	$x^{16} + x^{15} + x^2 + 1$	16	8005	FFFF	0000	true	true
CRC-16/CCITT	$x^{16} + x^{12} + x^5 + 1$	16	1021	0000	0000	true	true
CRC-16/CCITT-FALSE	$x^{16} + x^{12} + x^5 + 1$	16	1021	FFFF	0000	false	false
CRC-16/X25	$x^{16} + x^{12} + x^5 + 1$	16	1021	FFFF	FFFF	true	true
CRC-16/XMODEM	$x^{16} + x^{12} + x^5 + 1$	16	1021	0000	0000	false	false
CRC-16/DNP	$x^{16} + x^{13} + x^{12} + x^{11} + x^{10} + x^8 + x^6 + x^5 + x^2 + 1$	16	3D65	0000	FFFF	true	true
CRC-32	$x^{32} + x^{26} + x^{23} + x^{22} + x^{16} + x^{12} + x^{11} + x^{10} + x^8 + x^7 + x^5 + x^4 + x^2 + x + 1$	32	04C11DB7	FFFFFFFF	FFFFFFFF	true	true
CRC-32/MPEG-2	$x^{32} + x^{26} + x^{23} + x^{22} + x^{16} + x^{12} + x^{11} + x^{10} + x^8 + x^7 + x^5 + x^4 + x^2 + x + 1$	32	04C11DB7	FFFFFFFF	00000000	false	false

CSDN @Crazy_M

为什么又多了个多项式公式呢?
比如二进制的8bit数据

$$1010_0101$$

这个数是不是可以表示

$$2^7 + 2^5 + 2^2 + 2^0$$

原理是一样的。。。

例如生成多项式公式为：

$$x^4 + x + 1$$

版权：Crazy_M

对应的生成多项式为**10011**。

再观察上表中的多项式那一行，

$$x^4 + x + 1$$

对应的多项式为**03**，也就是二进制的**011**，为什么不是我们上面提到的**10011**呢？

因为数据跟多项式的的最高位异或的结果必然为**0**（因为我们是数据与多项式的最左位的**1**对齐异或的），所以在表示中它忽略了第一位。

上例中我们的生成多项式为**1011**，对应的生成多项式公式为：

$$x^3 + x + 1$$

3.通过实例来了解CRC

例：假设在CRC的校验中使用的生成多项式为

$$G(x) = x^3 + x + 1$$

4位原始报文为**1010**，求编码后的报文？

生成多项式公式对应的生成多项式为

$$1011$$

则CRC的位宽为3bit。

原始数据为

1010

将数据展宽为（CRC位先用0补齐）

1010

000

然后将展宽的数据和生成多项式进行异或，直到结果（余数）小于生成多项式的位宽（4）。

版权：Crazy_M

编码后的报文为：



上例中我们的生成多项式为1011，对应的生成多项式公式为：

$$x^3 + x + 1$$

打开CRC计算器

CRC (循环冗余校验) 在线计算

版权：Crazy_M

☒Hex ☐Ascii

需要校验的数据：

a 数据 1010

输入的数据为16进制，例如：31 32 33 34

参数模型 NAME：

自定义

宽度 WIDTH：

3

校验位的位宽=多项式位宽-1

多项式 POLY (Hex)：

3

例如：3D65

011，因为它忽略了最高位。所以对应的生成多项式为1011

初始值 INIT (Hex)：

0

例如：FFFF

结果异或值 XOROUT (Hex)：

0

例如：0000

☐输入数据反转 (REFIN) ☐输出数据反转 (REFOUT)

计算

清空

校验计算结果 (Hex)：

7

高位在左低位在右，使用时请注意高低位顺序！！

复制

校验计算结果 (Bin)：

111

结果

复制

CSDN @Crazy_M

在CRC计算器中包含了多个标准的CRC多项式：

CRC (循环冗余校验) 在线计算

版权: Crazy_M

☒ Hex ☐ Ascii

需要校验的数据:

a

输入的数据为16进制, 例如: 31 32 33 34

参数模型 NAME:

自定义

宽度 WIDTH:

自定义

多项式 POLY (Hex):

CRC-4/ITU

x^4+x+1

初始值 INIT (Hex):

CRC-5/EPC

x^5+x^3+1

异或值 XOROUT (Hex):

CRC-5/ITU

$x^5+x^4+x^2+1$

校验计算结果 (Hex):

CRC-5/USB

x^5+x^2+1

校验计算结果 (Bin):

CRC-6/ITU

x^6+x+1

CRC-7/MMC

x^7+x^3+1

CRC-8

x^8+x^2+x+1

CRC-8/ITU

x^8+x^2+x+1

CRC-8/ROHC

x^8+x^2+x+1

CRC-8/MAXIM

$x^8+x^5+x^4+1$

CRC-16/IBM

$x^{16}+x^{15}+x^2+1$

CRC-16/MAXIM

$x^{16}+x^{15}+x^2+1$

CRC-16/USB

$x^{16}+x^{15}+x^2+1$

CRC-16/MODBUS

$x^{16}+x^{15}+x^2+1$

CRC-16/CCITT

$x^{16}+x^{12}+x^5+1$

CRC-16/CCITT-FALSE

$x^{16}+x^{12}+x^5+1$

CRC-16/X25

$x^{16}+x^{12}+x^5+1$

CRC-16/XMODEM

$x^{16}+x^{12}+x^5+1$

CRC-16/DNP

$x^{16}+x^{13}+x^{12}+x^{11}+x^{10}+x^8+x^6+x^5+x^2+1$

CSDN @Crazy_M

可以灵活运用:

●Hex

○Ascii

需要校验的数据:

a

版权：Crazyy_M

输入的数据为16进制，例如：31 32 33 34

参数模型 NAME:

CRC-4/ITU

x4+x+1

宽度 WIDTH:

4

多项式 POLY (Hex) :

03

例如：3D65

初始值 INIT (Hex) :

00

例如：FFFF

结果异或值 XOROUT (Hex) :

00

例如：0000

☒输入数据反转 (REFIN)

☒输出数据反转 (REFOUT)

计算

清空

校验计算结果 (Hex) :

4

复制

高位在左低位在右，使用时请注意高低位顺序!!!

校验计算结果 (Bin) :

0100

复制

CSDN @Crazyy_M

5.CRC电路解析

我们通过硬件实现CRC校验，就要了解CRC硬件电路产生的原理。

首先我们经过CRC编码的数据是按以下数据格式传输的。原始数据我们是知道的{D3,D2,D1,D0}，但是{C2,C1,C0}我们是不知道的。我们需要搭建可以产生CRC的电路。

D3	D2	D1	D0	C2	C1	C0
----	----	----	----	----	----	----

所以我们要明白，我们搭建的电路，输入是原始数据，输出为CRC校验位。

我们搭建电路的规则就是根据多项式，在寄存器之间添加异或门。（原理下面会解释）

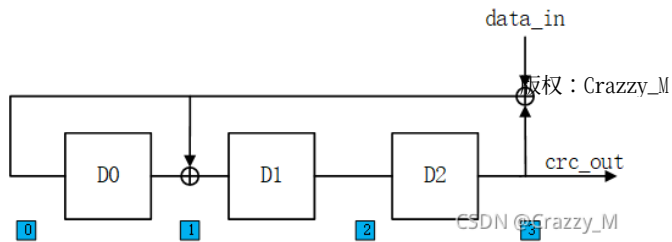
还是拿我们的例子，原始数据为1010，生成多项式为1011。生成多项式公式为

$$x^3 + x + 1$$

电路对应如下（蓝色框是辅助加异或门我自己填的，方便理解）：

187

未经允许，进制传播



观察上图添加的异或门的位置，再观察生成多项式公式。在0处没有添加异或门的原因是，添加了异或门也是与0异或，何必要浪费资源。

再看看为什么，这样设计会算出CRC呢？

下图中

方框代表寄存器

寄存器前面的数据表示现在的状态，

后面的数字表示寄存器的上一个状态

数据1010依次从高到低位输入电路

一路走下去，看看会发生什么

$$\begin{array}{r}
 1 \ 0 \ 1 \ 0 \ 0 \ 0 \ 0 \\
 1 \ 0 \ 1 \ 1 \\
 \hline
 0 \ 0 \ 0 \ 1 \ 0 \ 0 \ 0 \\
 \\
 1 \ 0 \ 1 \ 1 \\
 \hline
 0 \ 0 \ 1 \ 1
 \end{array}$$

版权：Crazy_M

6.CRC Verilog实现（并行）

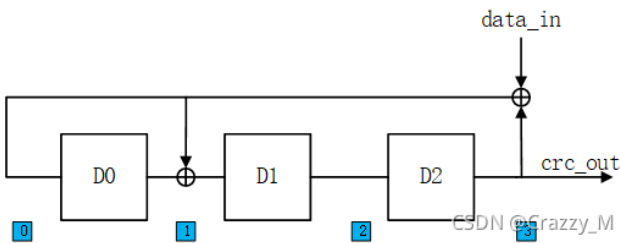
并行输出：

还是针对上例，输入数据1010，多项式公式为

$$x^3 + x + 1$$

多项式为：1011

对应电路为：（按照我们这个电路的原理，我们也可以用组合逻辑实现，不使用触发器，往下看）



我们先试试用组合逻辑来搭一下这个电路：

```

1 module top
2 #
3 (
4     parameter    DATA_WIDTH = 4,
5     parameter    CRC_WIDTH   = 3
6 )
7 (
8     input    [DATA_WIDTH-1:0]    data_in, //并行输入
9     output   reg[CRC_WIDTH-1:0]   crc_out
10 );
11
12 reg                data_temp;
13 always@(*)
14 begin
15     crc_out = { CRC_WIDTH{1'b0}};
16     for(i=(DATA_WIDTH-1);i>=0;i=i-1)
17         begin
18
19
20
21
22
23
24
25
26
27
28
29
30
31
32
33
34
35
36
37
38
39
40
41
42
43
44
45
46
47
48
49
50
51
52
53
54
55
56
57
58
59
60
61
62
63
64
65
66
67
68
69
70
71
72
73
74
75
76
77
78
79
80
81
82
83
84
85
86
87
88
89
90
91
92
93
94
95
96
97
98
99
100

```

```

17
18     data_temp  = data_in[i]^crc_out[2];
19     crc_out[2] = crc_out[1];
20     crc_out[1] = data_temp^crc_out[0];    版权：Crazy_M
21     crc_out[0] = data_temp;
22     end
23 end
24
25 endmodule

```

testbench:

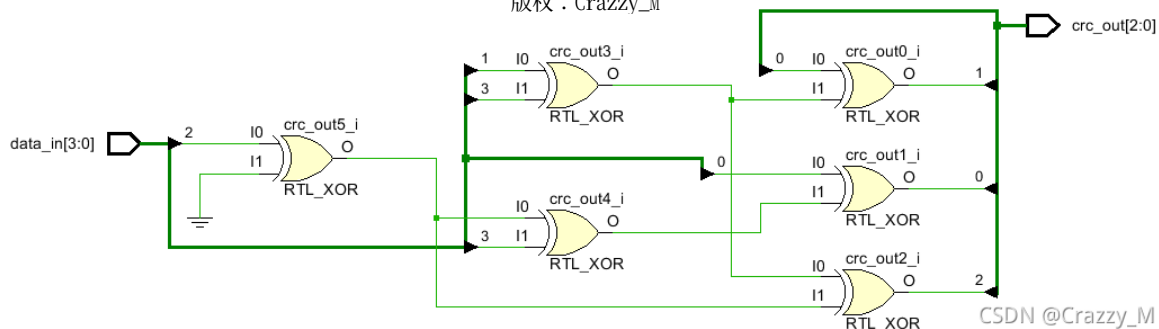
```

1 module top_test;
2
3 parameter  DATA_WIDTH = 4;
4 parameter  CRC_WIDTH  = 3;
5
6 reg [DATA_WIDTH-1:0]  data_in;
7 wire[CRC_WIDTH-1:0] crc_out;
8
9 top u1
10 (
11     .data_in (data_in),
12     .crc_out (crc_out)
13 );
14
15 initial
16 begin
17     data_in = 4'b0000;
18     #20;
19     rst_n = 1;
20
21     data_in = 4'b1010;
22     #20;
23     data_in = 4'b1111;
24     #20;
25     data_in = 4'b1110;
26     #20;
27     data_in = 4'b0111;
28
29 end
30 endmodule

```

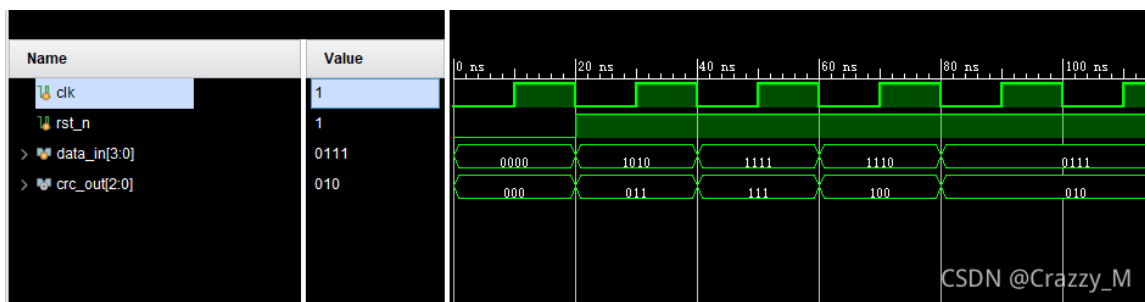
RTL:

可以看出全是由异或门搭建出来的



CSDN @Crazyy_M

看看结果，复位结束，data产生变化时，CRC数据计算完成。



CSDN @Crazyy_M

结果

数据	CRC
1010	011
1111	111
1110	100
0111	010

结合计算机对比一下：

数据1010，多项式1011：

☒Hex ☐Ascii

需要校验的数据:

a

版权: Crazy_M

输入的数据为16进制, 例如: 31 32 33 34

参数模型 NAME:

自定义

宽度 WIDTH:

3

多项式 POLY (Hex):

03

例如: 3D65

初始值 INIT (Hex):

00

例如: FFFF

结果异或值 XOROUT (Hex):

0

例如: 0000

☐输入数据反转 (REFIN)

☐输出数据反转 (REFOUT)

计算

清空

校验计算结果 (Hex):

3

复制

高位在左低位在右, 使用时请注意高低位顺序!!!

校验计算结果 (Bin):

011

复制

CSDN @Crazy_M

数据1111, 多项式1011:

☒Hex ☐Ascii

需要校验的数据:

f

输入的数据为16进制, 例如: 31 32 33 34

参数模型 NAME:

自定义

宽度 WIDTH:

3

多项式 POLY (Hex):

03

例如: 3D65

初始值 INIT (Hex):

00

例如: FFFF

结果异或值 XOROUT (Hex):

0

例如: 0000

☐输入数据反转 (REFIN)

☐输出数据反转 (REFOUT)

计算

清空

校验计算结果 (Hex):

7

复制

高位在左低位在右, 使用时请注意高低位顺序!!!

校验计算结果 (Bin):

111

复制

CSDN @Crazy_M

Hex☒ Ascii☐

版权：Crazy_M

需要校验的数据：

e

输入的数据为16进制，例如：31 32 33 34

参数模型 NAME：

自定义

宽度 WIDTH：

3

多项式 POLY (Hex)：

03

例如：3D65

初始值 INIT (Hex)：

00

例如：FFFF

结果异或值 XOROUT (Hex)：

0

例如：0000

☐ 输入数据反转 (REFIN)

☐ 输出数据反转 (REFOUT)

计算

清空

校验计算结果 (Hex)：

4

复制

校验计算结果 (Bin)：

100

复制

高位在左低位在右，使用时请注意高低位顺序！！

CSDN @Crazy_M

☒ Hex ☐ Ascii

需要校验的数据：

7

版权：Crazyy_M

输入的数据为16进制，例如：31 32 33 34

参数模型 NAME：

自定义

宽度 WIDTH：

3

多项式 POLY (Hex)：

03

例如：3D65

初始值 INIT (Hex)：

00

例如：FFFF

结果异或值 XOROUT (Hex)：

0

例如：0000

☐ 输入数据反转 (REFIN)

☐ 输出数据反转 (REFOUT)

计算

清空

校验计算结果 (Hex)：

2

复制

高位在左低位在右，使用时请注意高低位顺序！！

校验计算结果 (Bin)：

010

复制

CSDN @Crazyy_M

对比无误。

在这介绍一下用网站生成CRC：

<https://www.easics.com/crctool/>

Polynomial

Specify the generator polynomial by clicking on the polynomial terms, or select a predefined value from the list below.

1

x

x²

x³

x⁴

x⁵

x⁶

x⁷

x⁸

x⁹

x¹⁰

x¹¹

x¹²

x¹³

x¹⁴

x¹⁵

x¹⁶

x¹⁷

x¹⁸

x¹⁹

x²⁰

x²¹

x²²

x²³

x²⁴

x²⁵

x²⁶

x²⁷

x²⁸

x²⁹

x³⁰

x³¹

x³²

x³³

x³⁴

x³⁵

x³⁶

x³⁷

x³⁸

x³⁹

x⁴⁰

x⁴¹

x⁴²

x⁴³

x⁴⁴

x⁴⁵

x⁴⁶

x⁴⁷

x⁴⁸

x⁴⁹

x⁵⁰

x⁵¹

x⁵²

x⁵³

x⁵⁴

x⁵⁵

x⁵⁶

x⁵⁷

x⁵⁸

x⁵⁹

x⁶⁰

x⁶¹

x⁶²

x⁶³

x⁶⁴

x⁶⁵

x⁶⁶

x⁶⁷

x³ + x¹ + 1

自由选择

Select a predefined value:

Data Width

Select the number of data bits to be processed in one step:

数据宽度

4

Output Language

☐ VHDL

☒ Verilog

Email for Download

Email

Download

CSDN @Crazy_M

在这个网站上通过你的选择就会产生相应的verilog代码：

比如我的选择还是我们的例子
输入数据1010，多项式公式为

$$x^3 + x + 1$$

多项式为：1011

```
1 module top
2 #
3 (
4     parameter    DATA_WIDTH = 4,
5     parameter    CRC_WIDTH = 3,
6
7
8 )
9 (
10
11     input    [DATA_WIDTH-1:0]    data_in, //并行输入
12     output   [CRC_WIDTH -1:0]    data_crc
13 );
14
15
16 assign data_crc =nextCRC3_D4(data_in,3'b000);
17
```

196

未经允许，进制传播

```

18 function [2:0] nextCRC3_D4;
19
20 input [3:0] Data;
21 input [2:0] crc;
22 reg [3:0] d;
23 reg [2:0] c;
24 reg [2:0] newcrc;
25 begin
26     d = Data;
27     c = crc;
28
29     newcrc[0] = d[3] ^ d[2] ^ d[0] ^ c[1] ^ c[2];
30     newcrc[1] = d[2] ^ d[1] ^ d[0] ^ c[0] ^ c[1];
31     newcrc[2] = d[3] ^ d[2] ^ d[1] ^ c[0] ^ c[1] ^ c[2];
32     nextCRC3_D4 = newcrc;
33 end
34 endfunction
35 endmodule

```

版权：Crazy_M

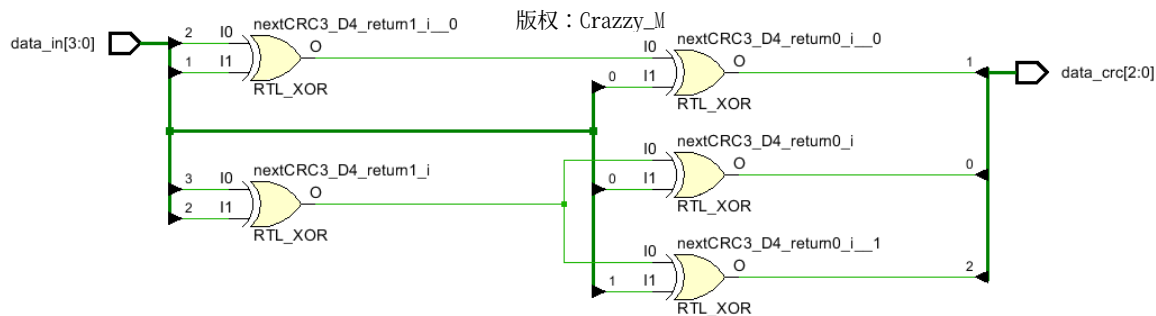
testbench:

```

1 module top_test;
2
3 reg [3:0] data_in;
4 wire[2:0] data_crc;
5
6 top u1
7 (
8     .data_in (data_in),
9     .data_crc (data_crc)
10 );
11
12 initial
13 begin
14
15     data_in = 4'b0000;
16     #20;
17     data_in = 4'b1010;
18     #20;
19     data_in = 4'b1111;
20     #20;
21     data_in = 4'b1110;
22     #20;
23     data_in = 4'b0111;
24
25 end
26 endmodule

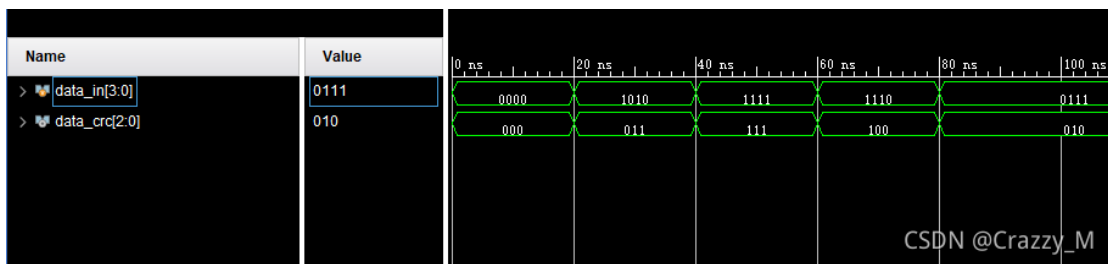
```

RTL: 也是通过异或门实现的



CSDN @Crazyzy_M

结果（和上面结果一样）：

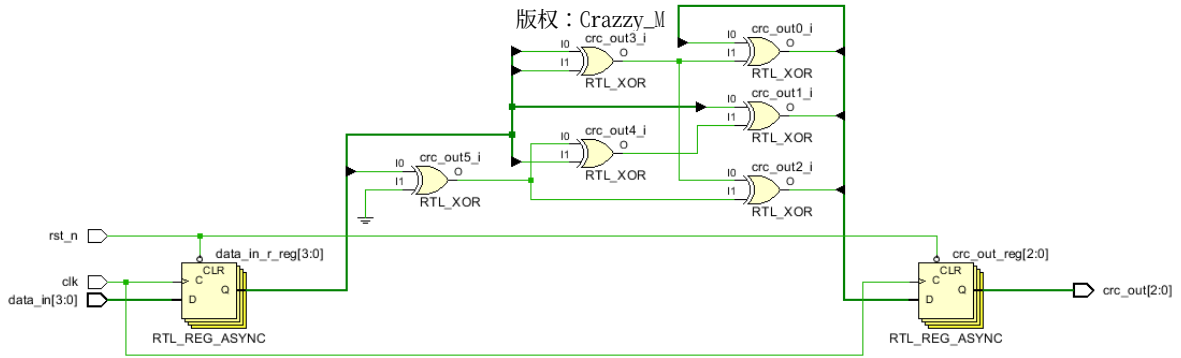


通过上述组合逻辑实现CRC校验的产生，但是组合逻辑在使用中是存在风险的，比如信号因走线的长度使到达门电路的有时差就可能出现竞争冒险的风险。

我的理解，相对安全一点，我们在数据的输入输出可以通过D触发器进行时钟的同步。



对应的RTL应该是这样：



对应代码：

```

1 module top
2 #
3 (
4     parameter    DATA_WIDTH = 4,
5     parameter    CRC_WIDTH  = 3
6 )
7 )
8 (
9     input        clk,
10    input        rst_n,
11    input  [DATA_WIDTH-1:0]  data_in,
12    output reg[CRC_WIDTH-1:0]  crc_out
13 );
14
15 reg            data_temp;
16 reg [DATA_WIDTH-1:0]  data_in_r;
17
18 integer        i;
19
20 reg[CRC_WIDTH-1:0]  crc_out_r;
21
22
23
24 always@(posedge clk or negedge rst_n)
25 begin
26     if(!rst_n)
27         crc_out <= { CRC_WIDTH{1'b0}};
28     else
29         crc_out <= crc_out_r;
30 end
31 always@(posedge clk or negedge rst_n)
32 begin
33     if(!rst_n)

```

```

34     data_in_r <= { DATA_WIDTH{1'b0}};
35     else
36         data_in_r <= data_in;
37 end
38
39
40 always@(*)
41 begin
42     crc_out_r = { CRC_WIDTH{1'b0}};
43     for(i=(DATA_WIDTH-1);i>=0;i=i-1)
44     begin
45         data_temp = data_in_r[i]^crc_out_r[2];
46         crc_out_r[2] = crc_out_r[1];
47         crc_out_r[1] = data_temp^crc_out_r[0];
48         crc_out_r[0] = data_temp;
49     end
50
51 end
52
53 endmodule

```

版权：Crazy_M

testbench:

```

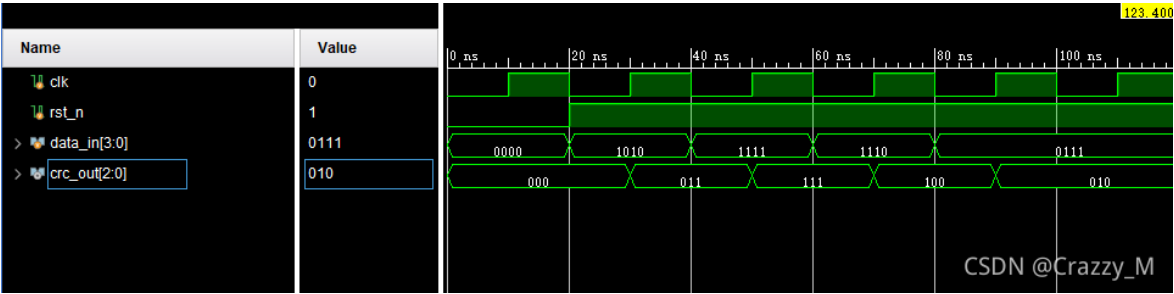
1 module top_test;
2
3 parameter DATA_WIDTH = 4;
4 parameter CRC_WIDTH = 3;
5
6 reg clk;
7 reg rst_n;
8 reg [DATA_WIDTH-1:0] data_in;
9 wire [CRC_WIDTH-1:0] crc_out;
10
11 top u1
12 (
13     .clk (clk),
14     .rst_n (rst_n),
15     .data_in (data_in),
16     .crc_out (crc_out)
17 );
18
19
20 initial
21 begin
22     clk = 0;
23     rst_n = 0;
24     data_in = 4'b0000;
25     #20;
26     rst_n = 1;
27
28     data_in = 4'b1010;
29     #20;
30     data_in = 4'b1111;
31     #20;
32     data_in = 4'b1110;

```

```
33     #20;
34     data_in = 4'b0111;
35 end
36
37 always #10 clk = ~clk ;
38
39 endmodule
40
```

版权：Crazy_M

结果中存在延迟

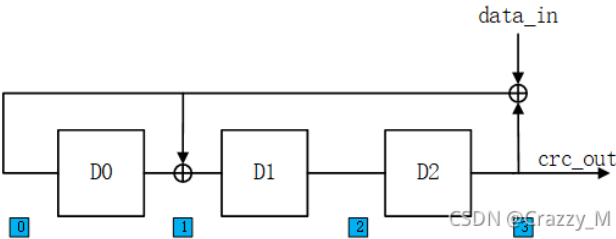


但是时钟还是存在过多的组合逻辑，尽量减少组合逻辑我们看看串行怎么实现？

7.CRC Verilog实现（串行）

CRC串行输出：

下图中的方框我们可以看做是触发器，通过移位寄存器的原理，移出CRC的结果。



我们数据串行输入，看以下代码：

```
1 module top
2 #
3 (
4     parameter DATA_WIDTH = 4,
5     parameter CRC_WIDTH = 3
6 )
7
8 (
9     input clk,
10    input rst_n,
```

201

未经允许，进制传播

```

11     input    [0:0]      data_in,           //串行输入
12     output  [CRC_WIDTH-1:0]  crc_out
13
14                                     版权：Crazy_M
15 );
16
17 wire          data_temp;
18 reg[CRC_WIDTH-1:0]  crc_out_r;
19
20 assign  crc_out      = crc_out_r ;
21 assign  data_temp    = data_in^crc_out_r[2];
22
23 always@(posedge clk or negedge rst_n)
24 begin
25     if(!rst_n)
26     begin
27         crc_out_r      <={CRC_WIDTH{1'b0}};
28     end
29     else
30     begin
31         crc_out_r[2]    <= crc_out_r[1];
32         crc_out_r[1]    <= data_temp^crc_out_r[0];
33         crc_out_r[0]    <= data_temp;
34     end
35 end
36
37 endmodule

```

testbench:

```

1  module top_test;
2
3  parameter  CRC_WIDTH  = 3;
4
5  reg          clk;
6  reg          rst_n;
7  reg [0:0]    data_in;
8  wire[CRC_WIDTH-1:0]  crc_out;
9
10 top u1
11 (
12     .clk      (clk),
13     .rst_n    (rst_n),
14     .data_in  (data_in),
15     .crc_out  (crc_out)
16 );
17
18
19 initial
20 begin
21     clk = 0;
22     rst_n = 0;
23     data_in = 0;
24     #20;
25     rst_n = 1;

```

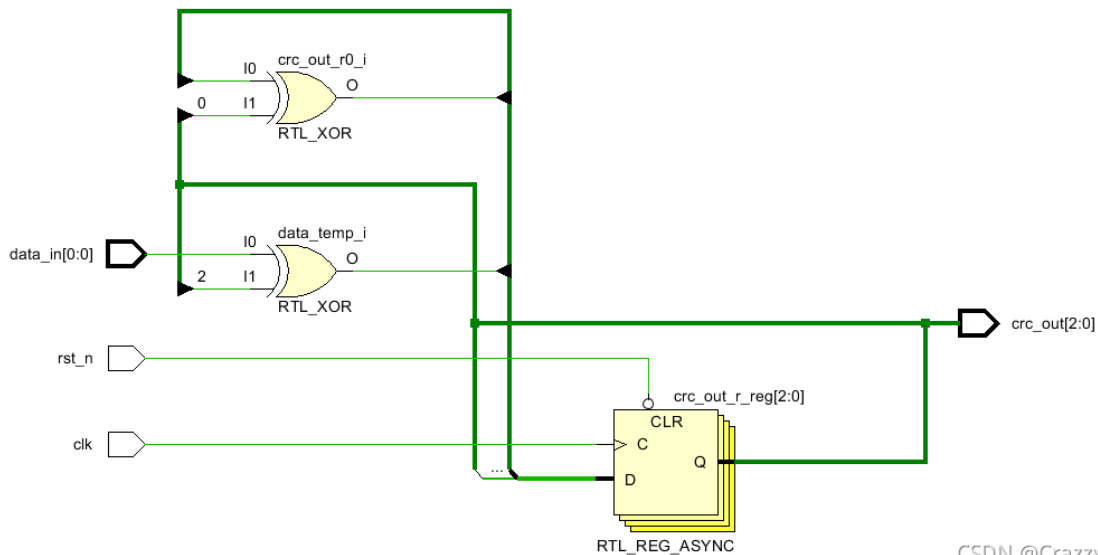
```

26
27   @(negedge clk);
28   data_in = 1'b1;
29   @(negedge clk);
30   data_in = 1'b0;
31   @(negedge clk);
32   data_in = 1'b1;
33   @(negedge clk);
34   data_in = 1'b0;
35   @(negedge clk);
36
37
38
39 end
40
41 always #10 clk = ~clk ;
42
43 endmodule

```

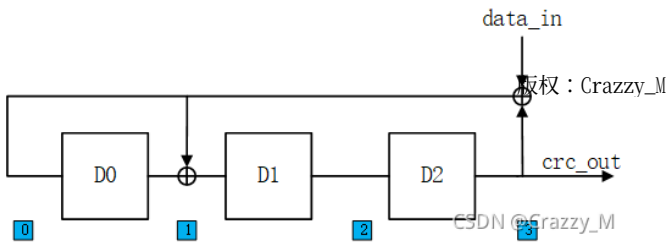
版权：Crazy_M

RTL:



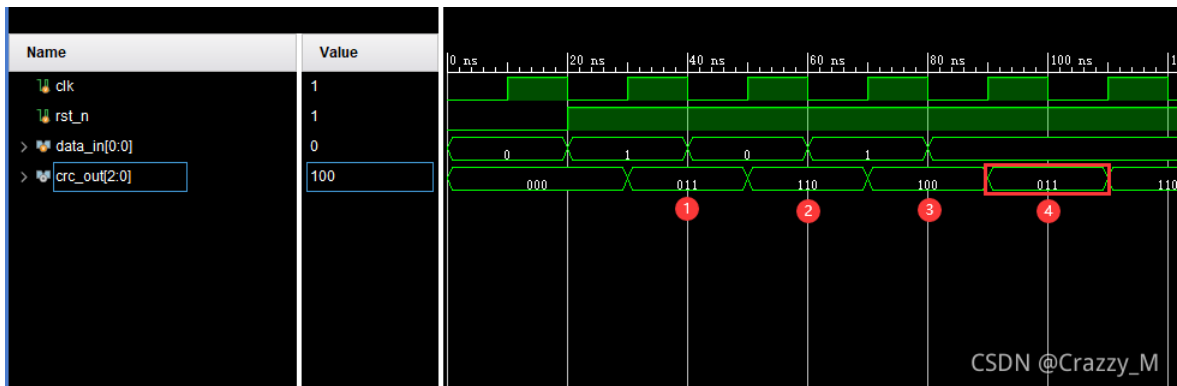
CSDN @Crazy_M

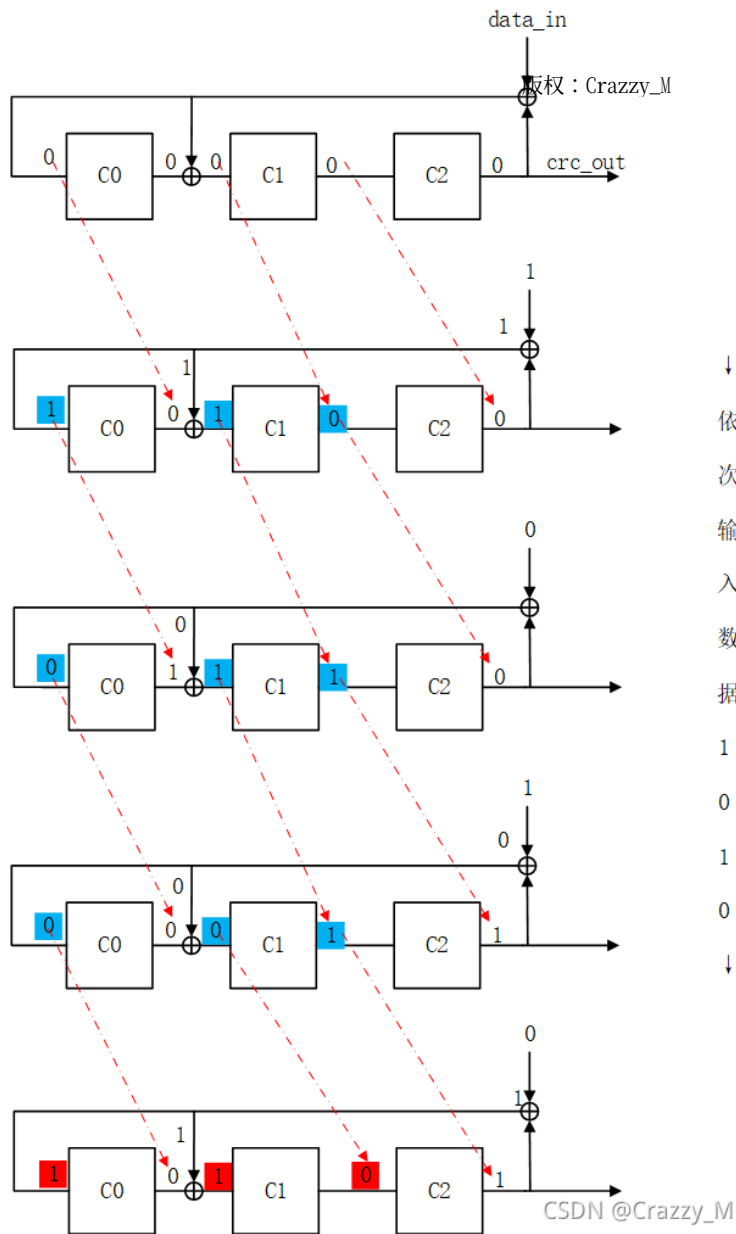
是不是一样的嘞~



因为数据是串行进去的，我们结果是不等4bit的数据完全进入以上网络CRC的结果才可以出来呢。

看看结果：是不是，是不是，是不是？是吧~

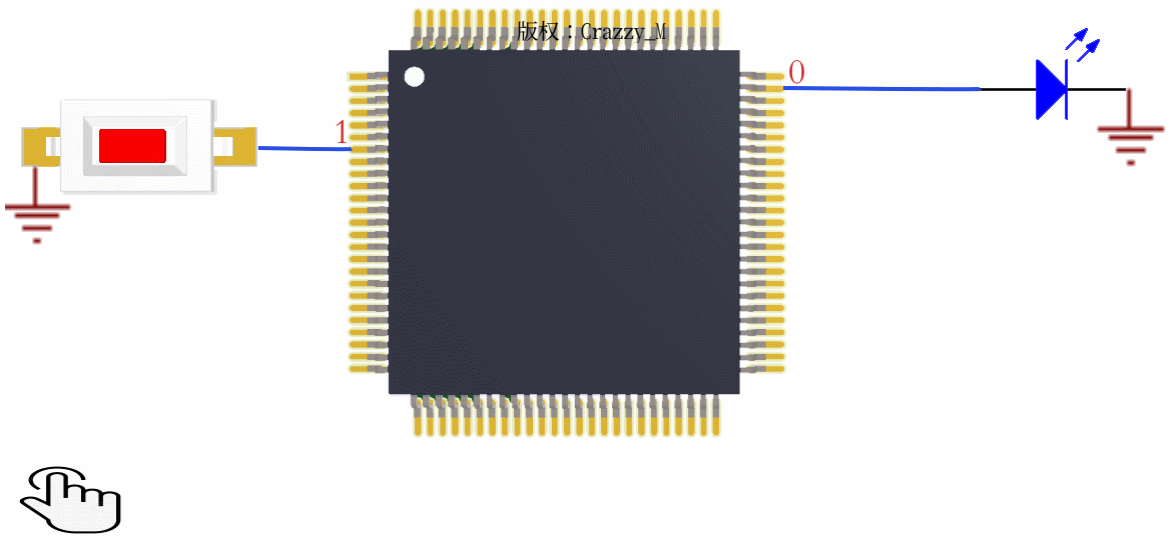




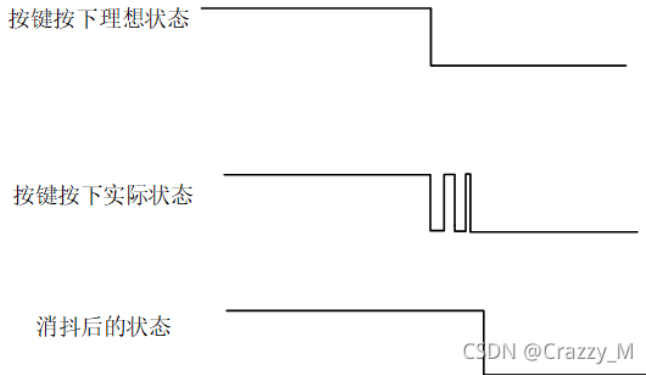
nice~

至此，CRC分析完毕。

三十九、按键消抖



按键作为基本的人机输入接口，很多电子设备都会用到，由于机械特性，在按键按下或者松开的时候，按键的输入值不是理想变化。而是存在抖动，我们在使用按键接入的信号时应该跳过这段抖动的时间。如下图所示，按键按下为低电平，按键松开为高电平



1.只对按键按下消抖

例：输入时钟为10Mhz，抖动小于15ms，用verilog实现。

通过计数来实现时钟的消抖，输入时钟为10Mhz，时钟周期为100ns，100ns时钟计数150_000为15ms。

代码：

```
1 module top
2 (
```

```

3   input  clk,
4   input  rst_n,
5
6   input  key_in,          //按键按下输入0 版权：Crazy_M
7   output key_out          //按键按下经过消抖输出0
8
9   );
10
11  parameter cnt_max = 32'd149_999;
12  reg [31:0] cnt;
13  wire      cnt_start;    //按键按下开启计数
14
15  reg      key_in_r;
16
17  assign cnt_start = key_in_r & ~key_in; //判断按键状态
18  assign key_out = ((cnt == cnt_max) && (key_in_r == 1'b0)) ? 1'b0 : 1'b1;
19  always@(posedge clk or negedge rst_n)
20  begin
21      if(!rst_n)
22      begin
23          cnt          <= 32'd0;
24          key_in_r     <= 1'b1;
25      end
26      else
27      begin
28          cnt          <= (cnt == cnt_max) ? ((cnt_start) ? 32'd0 : cnt) : cnt+1;
29          key_in_r     <= key_in;
30      end
31  end
32
33  endmodule

```

testbench:

```

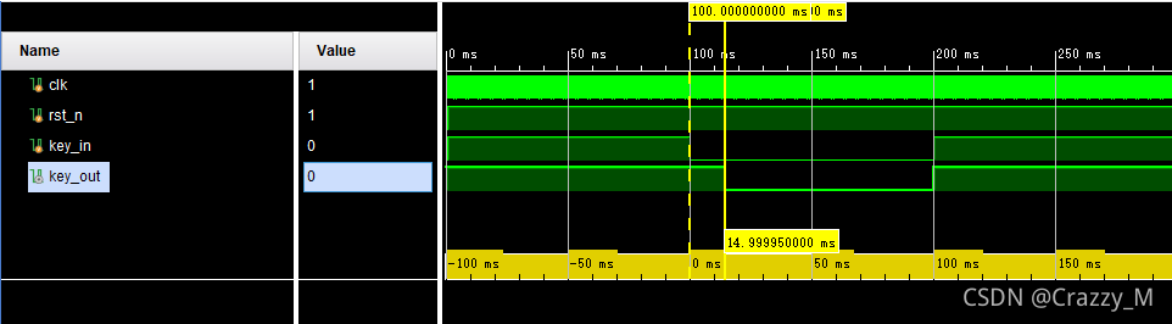
1  module top_test;
2
3  reg    clk;
4  reg    rst_n;
5  reg    key_in;
6  wire   key_out;
7
8  top u1
9  (
10     .clk    (clk),
11     .rst_n  (rst_n),
12     .key_in (key_in),
13     .key_out(key_out)
14 );
15
16 initial
17 begin
18     clk    = 0;
19     rst_n  = 0;
20     key_in = 0;
21     #20;

```

```
22     rst_n    = 1;
23
24     key_gen;
25
26 end
27
28 task key_gen;
29 begin
30     key_in = 1'b1;
31     repeat(1_000_000)@(negedge clk);
32     key_in = 1'b0;
33     repeat(1_000_000)@(negedge clk);
34     key_in = 1'b1;
35 end
36 endtask
37
38 always # 50 clk = ~clk;
39
40 endmodule
```

版权：Crazy_M

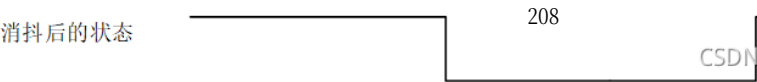
波形展示：



CSDN @Crazy_M

2.按键按下、松开都进行消抖

上面只是实现了按键按下的消抖，但是在按键松开的时候同样存在抖动的问题。



按键按下、松开都进行消抖：

代码：

版权：Crazy_M

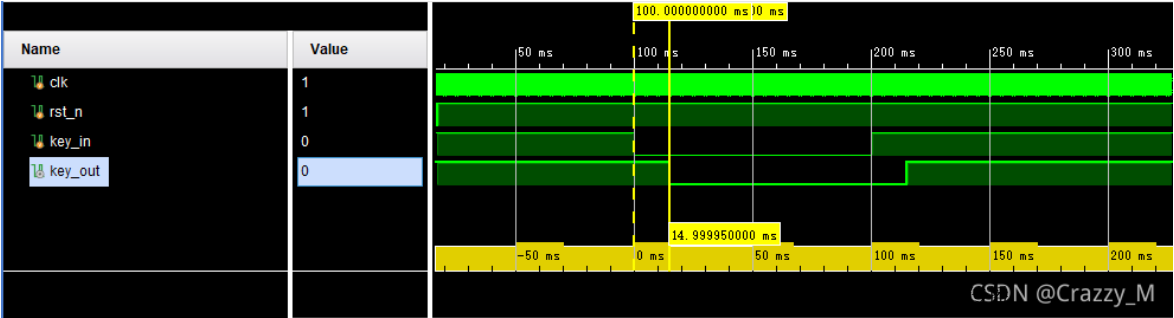
```
1 //输入时钟为10Mhz,时钟周期为100ns,消抖时间为15ms,100ns时钟计数150_000为15ms
2 //按键为低电平有效
3 module top
4 (
5     input  clk,
6     input  rst_n,
7
8     input  key_in,          //按键按下输入0
9     output key_out          //按键按下经过消抖输出0
10 );
11
12
13 parameter cnt_max = 32'd149_999;
14 reg [31:0] cnt_down;
15 reg [31:0] cnt_up;
16 wire      cnt_start;      //按键按下开启计数
17
18 reg      key_in_r;
19
20 assign    cnt_dstart = (key_in_r&~key_in);    //按键按下
21 assign    cnt_ustart = (~key_in_r&key_in);     //按键弹起
22
23 assign    key_out = ((cnt_down == cnt_max) && (key_in_r == 1'b0))?1'b0:(((cnt_up == cnt_max) && (key_in_r == 1'b1
24 ))?1'b1:key_out);
25 always@(posedge clk or negedge rst_n)
26 begin
27     if(!rst_n)
28     begin
29         cnt_down    <= 32'd0;
30         cnt_up      <= 32'd0;
31         key_in_r    <= 1'b1;
32     end
33     else
34     begin
35         cnt_down    <= (cnt_down == cnt_max)?((cnt_dstart)?32'd0:cnt_down):cnt_down+1;
36         cnt_up      <= (cnt_up == cnt_max)?((cnt_ustart)?32'd0:cnt_up):cnt_up+1;
37         key_in_r    <= key_in;
38     end
39 end
40
41 endmodule
```

testbench:

```
1 module top_test;
2
3 reg    clk;
4 reg    rst_n;
5 reg    key_in;
```

```
6 wire    key_out;
7
8
9 top u1                                     版权：Crazyy_M
10 (
11     .clk      (clk),
12     .rst_n    (rst_n),
13     .key_in   (key_in),
14     .key_out  (key_out)
15
16 );
17
18 initial
19 begin
20     clk      = 0;
21     rst_n    = 0;
22     key_in   = 0;
23     #20;
24     rst_n    = 1;
25
26     key_gen;
27
28 end
29
30 task key_gen;
31 begin
32     key_in = 1'b1;
33     repeat(1_000_000)@(negedge clk);
34     key_in = 1'b0;
35     repeat(1_000_000)@(negedge clk);
36     key_in = 1'b1;
37 end
38 endtask
39
40 always # 50 clk = ~clk;
41
42 endmodule
43
```

波形展示：



四十、OC门和OD门

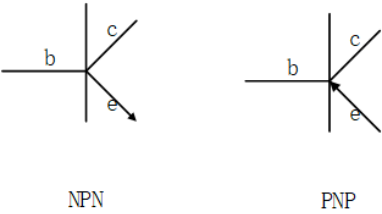
1.OC门

版权：Crazyy_M

集电极开路门（Open Collector），是对三极管而言的

先复习一下三极管：

- b：基极
- c：集电极
- e：发射级



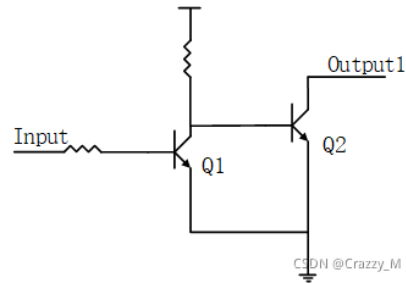
三极管主要两个用途，放大、开关。
放大：主要是对交流的放大。

$I_b = \beta I_c$

$I_e = I_b + I_c$

开关：PN结导通电压为0.7。

OC门：集电极开路



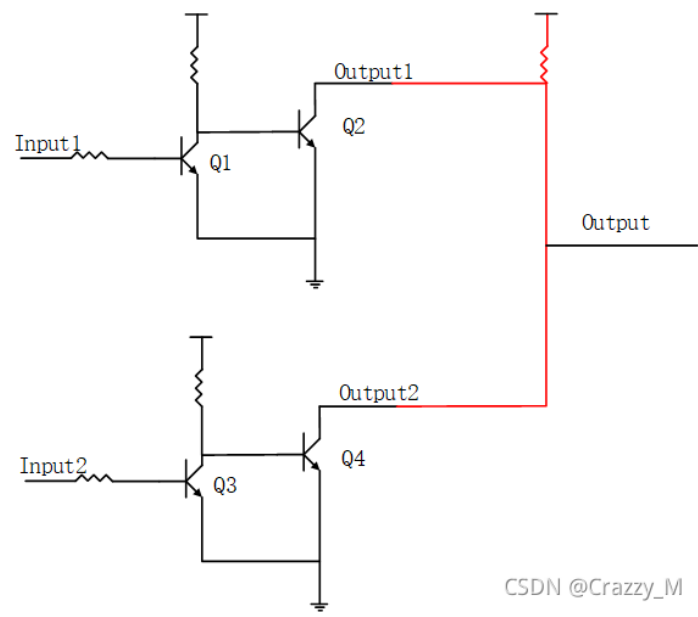
如上图所示，在输入端输入

Input	Q1	Q2	Output1
0	不导通	导通	211 ⁰ 未经允许，进制传播
1	导通	不导通	高阻态（既不是0也不是1，由后级电路决定）

OC门实现线与（线上连接的信号最终的结果是相与的关系，如下图红线中的Output1和Output2）

由于集电极开路，没法输出高电平，所以在实现线与的逻辑时，在集电极加上拉电阻。

版权：Crazy_M

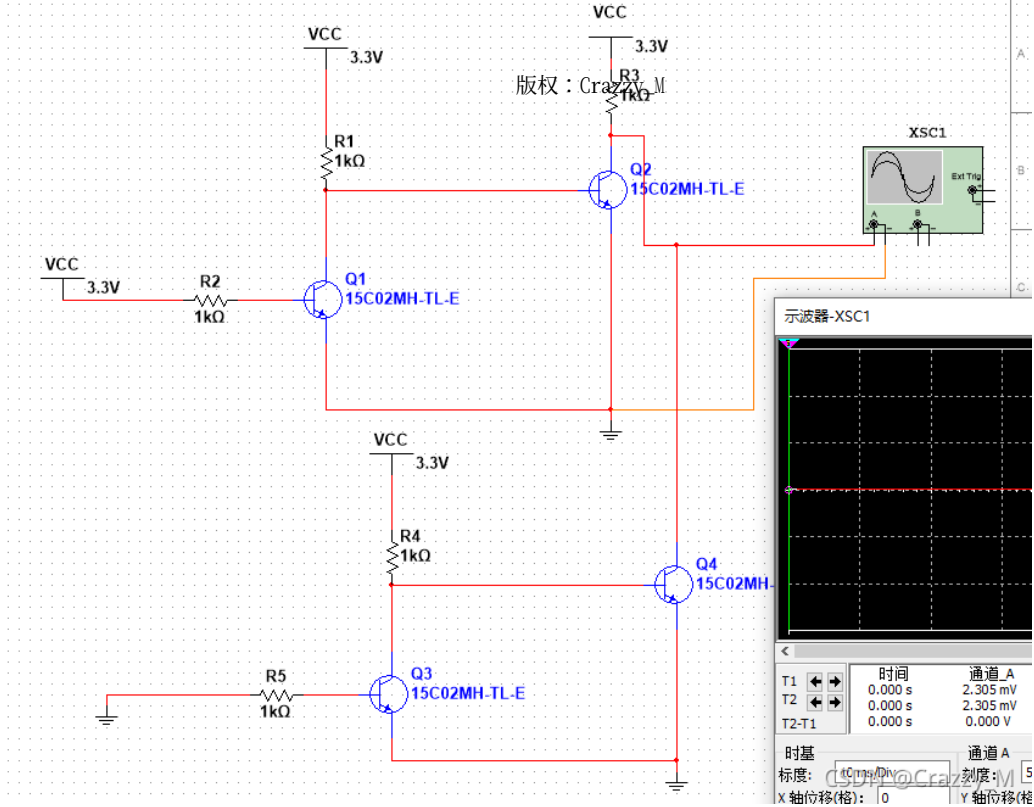


CSDN @Crazy_M

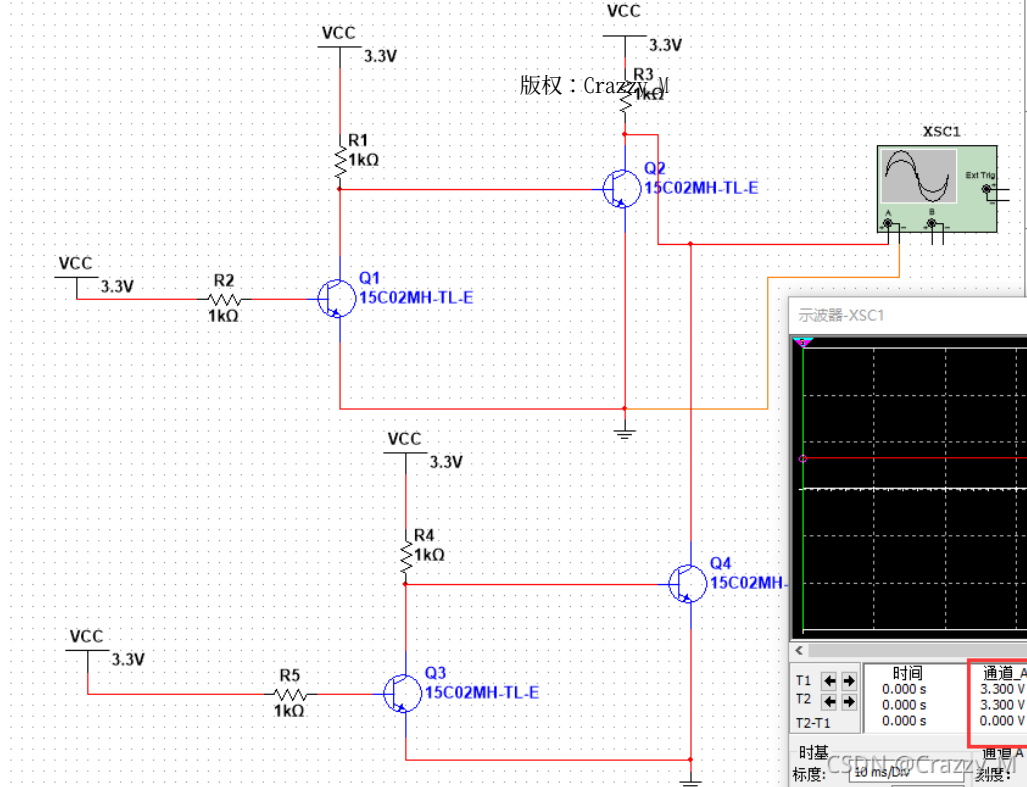
Input1	Q1	Q2	Input2	Q3	Q4	Output1	Output2	Output
0	截止	导通	0	截止	导通	0	0	0
0	截止	导通	1	导通	截止	0	1	0
1	导通	截止	0	截止	导通	1	0	0
1	导通	截止	1	导通	截止	1	1	1

实现了线与。

有0则为0

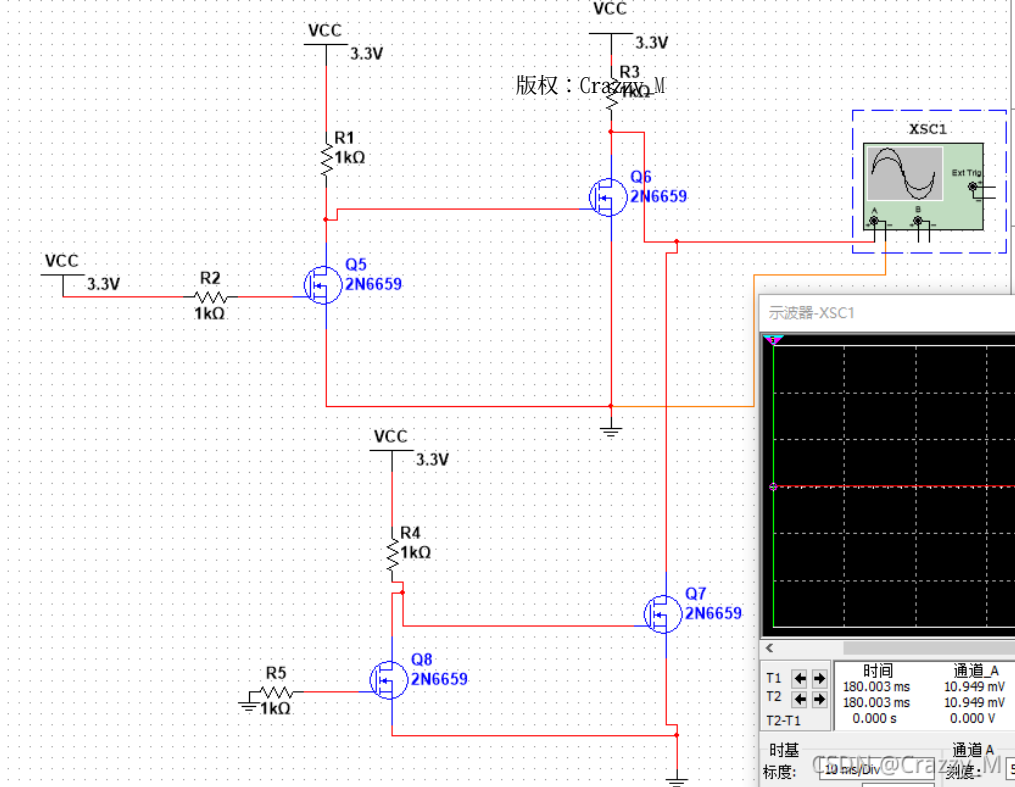


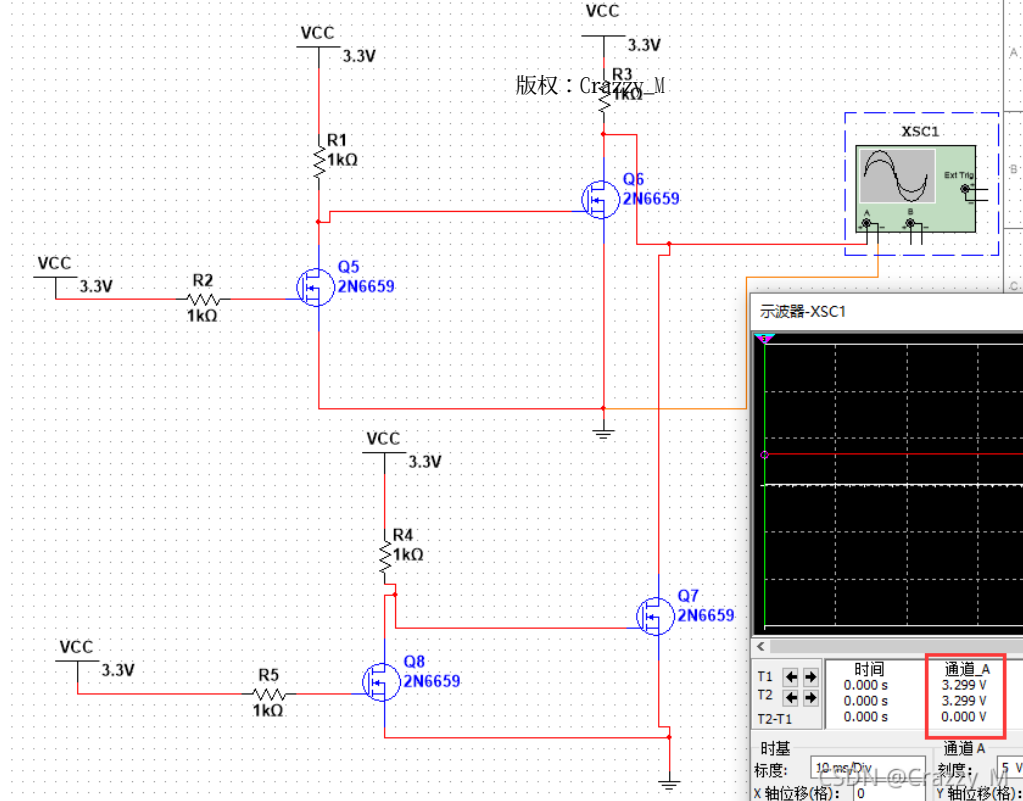
全1则为1



2.OD门（漏极开路输出）

原理和OC门一样的





四十一、数字IC设计流程、数字IC后端设计流程

1.数字IC设计流程

1.需求分析(制定规格书)。

分析用户或市场的需求，并将其翻译成对芯片产品的技术需求。

2.算法设计。

设计和优化芯片中所使用的算法。这一阶段一般使用高级编程语言（如C/C++），利用算法级建模和仿真工具（如MATLAB，SPW）进行浮点和定点的仿真，进而对算法进行评估和优化。

3.构架设计。

根据设计的功能需求和算法分析的结果，设计芯片的构架，并对不同的方案进行比较，选择性能价格最优的方案。这一阶段可以使用SystemC 语言对芯片构架进行模拟和分析。

4.RTL 设计（代码输入）。

使用HDL 语言完成对设计实体的RTL 级描述。这一阶段使用VHDL 和Verilog HDL 语言的输入工具编写代码。

5. RTL 验证（功能仿真）。

使用仿真工具或其他RTL 代码分析工具，验证RTL 代码的质量和性能。

6.综合。

未经允许，进制传播

从RTL 代码生成描述实际电路的门级网表文件。

7.门级验证（综合后仿真）。

对综合产生的门级网表进行验证。这一阶段通常会使用仿真、静态时序分析和形式验证等工具。

版权：Crazzy_M

8. 布局布线。

后端设计对综合产生的门级网表进行布局规划（Floorplanning）、布局（Placement）、布线（Routing），生成生产用的版图。

9.电路参数提取确定芯片中互连线的寄生参数，从而获得门级的延时信息。

10.版图后验证。

根据后端设计后取得的新的延时信息，再次验证设计是否能够实现所有的功能和性能指标。

11.芯片生产。

生产在特定的芯片工艺线上制造出芯片。

12. 芯片测试。

对制造好的芯片进行测试，检测生产中产生的缺陷和问题。

2.数字IC 后端设计流程

1. 数据准备。

对于Cadance 的SE 而言后端设计所需的数据主要有是Foundry 厂提供的标准单元、宏单元和I/O Pad 的库文件,它包括物理库、时序库及网表库,分别以.lef、.tlf 和.v 的形式给出。前端的芯片设计经过综合后生成的门级网表,具有时序约束和时钟定义的本脚本文件和由此产生的.gcf 约束文件以及定义电源Pad 的DEF (Design Exchange Format) 文件。(对synopsys 的Astro 而言, 经过综合后生成的门级网表,时序约束文件SDC 是一样的,Pad 的定义文件-.tlf 文件-technology file, Foundry 厂提供的标准单元、宏单元和I/O Pad 的库文件就与FRAM, CELL view, LM view 形式给出(Milkway 参考库and DB, LIB file)

2.布局规划。

主要是标准单元、I/O Pad 和宏单元的布局。I/O Pad 预先给出了位置,而宏单元则根据时序要求进行摆放,标准单元则是给出了一定的区域由工具自动摆放。布局规划后,芯片的大小,Core 的面积,Row 的形式、电源及地线的Ring 和Strip 都确定下来了。如果必要在自动放置标准单元和宏单元之后,你可以先做一次PNA(power network analysis) -IR drop and EM .

3. Placement -自动放置标准单元。

布局规划后,宏单元、I/O Pad 的位置和放置标准单元的区域都已确定,这些信息SE (Silicon Ensemble) 会通过DEF 文件传递给PC(Physical Compiler),PC 根据由综合给出的.DB 文件获得网表和时序约束信息进行自动放置标准单元,同时进行时序检查和单元放置优化。如果你用的是PC +Astro 那你可用write_milkway,read_milkway 传递数据。

4. 时钟树生成(CTS Clock tree synthesis)。

芯片中的时钟网络要驱动电路中所有的时序单元,所以时钟源端门单元带载很多,其负载延时很大并且不平衡,需要插入缓冲器减小负载和平衡延时。时钟网络及其上的缓冲器构成了时钟树。一般要反复几次才可以做出一个比较理想的时钟树。

5. STA 静态时序分析和后仿真。

时钟树插入后,每个单元的位置都确定下来了,工具可以提出Global Route 形式的连线寄生参数,此时对延时参数的提取就比较准确了。SE 把.V 和.SDF 文件传递给PrimeTime 做静态时序分析。确认没有时序违规后,将这两个文件传递给前端人员做后仿真。对Astro 而言,在detail routing 之后,用starRC XT 参数提取,生成的E.V 和.SDF 文件传递给PrimeTime 做静态时序分析,那将会更准确。

6. ECO(Engineering Change Order)。

针对静态时序分析和后仿真中出现的问题,对电路和单元布局进行小范围的改动。

7. filler的插入(pad filier, cell filler)。

Filler 指的是标准单元库和I/O Pad 库中定义的与逻辑无关的填充物,用来填充标准单元和标准单元之间,I/O Pad 和I/O Pad 之间的间隙,它

主要是把扩散层连接起来,满足DRC 规则和设计需要。

8.布线(Routing)。

Global route-- Track assign --Detail routing—Routing optimization 布线是指为满足工艺规则和布线层数限制、线宽、线间距限制和各线网可靠绝缘的电性能约束的条件下,根据电路的连接关系将各单元和I/O Pad 用互连线连接起来,这些是在时序驱动(Timing driven)的条件下进行的,保证关键时序路径上的连线长度能够最小。--Timingreport clear

9. Dummy Metal 的增加。

Foundry 厂都有对金属密度的规定,使其金属密度不要低于一定的值,以防在芯片制造过程中的刻蚀阶段对连线的金属层过度刻蚀从而降低电路的性能。加入Dummy Metal 是为了增加金属的密度。

10. DRC 和LVS。

DRC 是对芯片版图中的各层物理图形进行设计规则检查(spacing ,width),它也包括天线效应的检查,以确保芯片正常流片。LVS 主要是将版图和电路网表进行比较,来保证流片出来的版图电路和实际需要的电路一致。DRC 和LVS 的检查--EDA 工具Synopsys hercules/ mentor calibre/ CDN Dracula 进行的.Astro also include LVS/DRCcheck commands.

11. Tape out。

在所有检查和验证都正确无误的情况下把最后的版图GDSII文件传递给Foundry 厂进行掩膜制造。

四十二、芯片功耗问题

功耗问题一般分为静态功耗和动态功耗

1.动态功耗

发生在门开关（或状态翻转）的瞬间。由于对电容的充电、电源和地之间短暂电流通路造成的。它正比与开关频率。

降低动态功耗：

- 1.降低内核供电电压。降低电压会影响时序性能。为了弥补这一影响，一般采用流水线（pipelining）和并行（parallelsim）设计方法来提高设计性能。当然这样也会增加设计面积。
- 2.采用多电压设计，比如FPGA不同Bank设计不同的参考电压用于不同的功能。
- 3.降低电容负载和逻辑开关频率。

2.静态功耗

总是存在的，是由于电源和地之间的静态导通电流（或漏电流）引起的。

四十三、计算时钟频偏

先了解单位：ppm：百万分之一，即

10⁻⁶

例：有一个晶振，标称频率为200MHz，实际频率为200.0005MHz，那么它的时钟频偏是多少？

200.0005MHz - 200MHz = 0.0005MHz

频偏为：

0.0005MHz/200MHz = 0.0000025 = 2.5 × 10⁻⁶ 218 未经允许，进制传播